

电子科技大学
UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

硕士学位论文

MASTER THESIS



论文题目 _____

学科专业 _____

学 号 _____

作者姓名 _____

指导教师 _____

学 院 _____

分类号 _____ 密级 _____

UDC^{注 1} _____

学 位 论 文

(题名和副题名)

(作者姓名)

指导教师 _____

(姓名、职称、单位名称)

申请学位级别 **硕士** 学科专业 _____

提交论文日期 _____ 论文答辩日期 _____

学位授予单位和日期 _____

答辩委员会主席 _____

评阅人 _____

注 1：注明《国际十进分类法 UDC》的类号。

A Master Thesis Submitted to
University of Electronic Science and Technology of China

Discipline _____

Student ID _____

Author _____

Supervisor _____

School _____

独创性声明

本人声明所呈交的学位论文是本人在导师指导下进行的研究工作及取得的研究成果。据我所知,除了文中特别加以标注和致谢的地方外,论文中不包含其他人已经发表或撰写过的研究成果,也不包含为获得电子科技大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示谢意。

作者签名: _____ 日期: _____ 年 _____ 月 _____ 日

论文使用授权

本学位论文作者完全了解电子科技大学有关保留、使用学位论文的规定,同意学校有权保留并向国家有关部门或机构送交论文的复印件和数字文档,允许论文被查阅。本人授权电子科技大学可以将学位论文的全部或部分内容编入有关数据库进行检索及下载,可以采用影印、扫描等复制手段保存、汇编学位论文。

(涉密的学位论文须按照国家及学校相关规定管理,在解密后适用于本授权。)

作者签名: _____ 导师签名: _____

日期: _____ 年 _____ 月 _____ 日

摘要

随着边缘异构计算的快速发展，嵌入式系统面临严峻的数据安全挑战。机密虚拟机作为保障敏感数据与模型资产的核心技术已被广泛应用。然而，现有商用架构多存闭源局限，传统由 C/C++ 构建的底层固件易受内存破坏漏洞威胁；且在端侧受限环境下，实现安全的异构计算环境仍是机密计算领域的难题。为此，本文基于开源 RISC-V 架构，设计并实现了一套高安全、低开销的微内核可信为内核虚拟化方案 EasyAda-trust。本文主要研究与创新如下：

1. 提出基于 Rust 的内存安全底层固件。深度结合 Rust 所有权机制设计页分配器，从编译期根除二次释放等内存隐患；将 RISC-V 物理内存保护原语抽象为统一安全原语，构筑 CVM 与不可信组件间的物理级强隔离屏障，确立底层信任根。

2. 构建支持 CVM 全生命周期的微内核虚拟化架构。实施物理资源调度与安全鉴权的深度解耦。Hypervisor 仅负责宏观资源调度，而安全区域构建、二阶段页表隔离映射及机密流的两阶段上下文切换，均由可信安全管理器（TSM）隐匿执行；并结合本地证明机制实现了 CVM 的双向可信启动。

3. 设计防 DMA 攻击的机密异构计算环境。引入 RISC-V IOPMP 扩展在总线级实施设备动态鉴权；通过构建加速器管理 CVM，利用不可信共享内存与半虚拟化技术打通跨域通信，使机密应用旁路不可信软件中转栈，实现底层加速器在机密环境内零拷贝明文使用。

最后，在 QEMU 仿真环境下对系统开展了多维度评估。结果表明，EasyAda-SM 能够有效兼容现有 RISC-V TEE 方案且核心计算（Coremark 与 RV8）损耗均低于 2%；得益于静态内存预分配，CVM 在系统时钟与访存等核心原语上实现近似零损耗；同时，安全直通架构显著提升了异构 I/O 吞吐量。本文方案在内存安全、异构设备复用与总体计算效能之间实现了有效的架构折中。

关键词：Rust, RISC-V, 微内核虚拟化, 机密计算, 可信异构计算

ABSTRACT

With the rapid development of edge heterogeneous computing, embedded systems face severe data security challenges. Confidential Virtual Machines (CVMs) have been widely applied as a core technology to protect sensitive data and model assets. However, existing commercial architectures mostly suffer from closed-source limitations, and traditional low-level firmware built with C/C++ is vulnerable to memory corruption threats. Moreover, realizing a secure heterogeneous computing environment in resource-constrained edge scenarios remains a formidable challenge in the field of confidential computing. To address these issues, based on the open-source RISC-V architecture, this thesis designs and implements EasyAda-trust, a high-security and low-overhead trusted microkernel virtualization framework. The main research and innovations of this thesis are as follows:

1. Proposing a memory-safe underlying firmware based on Rust. By deeply integrating Rust's ownership mechanism, a page allocator is designed to eradicate memory hazards such as double-free vulnerabilities at compile time. Furthermore, RISC-V physical memory protection primitives are abstracted into unified security primitives to construct a strong physical-level isolation barrier between CVMs and untrusted components, thereby establishing the underlying root of trust.

2. Constructing a microkernel virtualization architecture supporting the full lifecycle of CVMs. A deep decoupling of physical resource scheduling and security authentication is implemented. The Hypervisor is solely responsible for macro-resource scheduling, whereas the construction of secure regions, the isolated mapping of two-stage page tables, and the two-stage context switching of confidential flows are all transparently executed by the Trusted Security Manager (TSM). Combined with a local attestation mechanism, a bidirectional trusted boot for CVMs is achieved.

3. Designing a confidential heterogeneous computing environment resistant to DMA attacks. The RISC-V IOPMP extension is introduced to implement dynamic device authentication at the bus level. By constructing an accelerator-management CVM and utilizing untrusted shared memory alongside paravirtualization technologies, cross-domain communication is bridged. This enables confidential applications to bypass the untrusted software relay stack, achieving zero-copy plaintext access to underlying

accelerators within the confidential environment.

Finally, a multi-dimensional evaluation of the system is conducted in a QEMU simulation environment. The results indicate that the proposed framework exhibits excellent ecological compatibility, with the performance overhead of core computations (Coremark and RV8 benchmark) kept below 2%. Thanks to static memory pre-allocation, the CVM achieves near-zero overhead on core primitives such as system clock and memory access. Meanwhile, the secure passthrough architecture significantly enhances the heterogeneous I/O throughput. The proposed framework achieves an ideal architectural tradeoff among ultimate memory safety, heterogeneous device multiplexing, and overall computing performance.

Keywords: Rust Language; RISC-V; Microkernel Virtualization; Confidential Computing; Trust Heterogeneous Computing

目 录

第一章 绪论	1
1.1 研究工作的背景与意义	1
1.2 国内外研究背景与现状	3
1.2.1 虚拟化技术研究现状	3
1.2.2 虚拟化与 TEE 融合架构及演进现状	5
1.2.3 异构计算环境下 GPU-TEE 安全隔离与防护技术研究	7
1.3 本文主要贡献和创新	8
1.4 论文结构	8
第二章 相关技术理论概论	9
2.1 RISC-V 背景知识	9
2.1.1 RISC-V 特权级介绍	9
2.1.2 RISC-V 中断与异常处理机制介绍	10
2.1.3 RISC-V 内存保护机制介绍	11
2.1.4 RISC-V 硬件虚拟化扩展 (H-extension) 介绍	12
2.2 可信执行环境 (TEE) 相关技术介绍	14
2.2.1 面向异构计算环境的 TEE	15
2.3 微内核操作系统 EasyAda 介绍	17
2.4 微内核虚拟化方案 EasyAda-Hypervisor 介绍	18
2.5 Rust 相关知识介绍	20
2.6 本章小结	21
第三章 基于 RISC-V 的微内核虚拟化方案设计	22
3.1 设计目标和需求分析	22
3.2 EasyAda-trust 工作流	22
3.3 威胁模型	23
3.4 可信微内核虚拟化方案设计	24
3.4.1 总体设计	24
3.4.2 内存安全的模块化安全监视器设计	26
3.4.3 安全特性设计	29
3.5 本章小结	38

第四章 内存安全的模块化安全监视器实现	40
4.1 内存安全固件	40
4.2 安全区域和内存管理层实现	40
4.2.1 内存管理原语	41
4.2.2 内存保护原语	44
4.2.3 安全内存管理原语	46
4.3 面向 CVM 的 Enclave 管理模块实现	49
4.3.1 Enclave 间内存安全	50
4.3.2 安全启动	51
4.3.3 中断和异常安全	52
4.3.4 IOPMP 管理	53
4.4 本章小结	55
第五章 基于微内核的可信虚拟化方案实现	56
5.1 安全启动流程实现	56
5.2 安全 CPU 虚拟化实现	59
5.2.1 安全上下文管理	59
5.2.2 安全异常处理	61
5.3 安全内存虚拟化实现	62
5.3.1 基于 TSM 接口的安全区域生命周期管理	63
5.4 安全 I/O 虚拟化实现	64
5.4.1 安全直通设备生命周期管理	64
5.4.2 基于 IOPMP 的设备侧防 DMA 攻击隔离	65
5.4.3 I/O 数据平面安全与端到端保护	66
5.5 机密异构计算环境实现	67
5.5.1 硬件加速器的保护	67
5.5.2 加速器管理 CVM 初始化	68
5.5.3 建立信道	69
5.6 本章小结	70
第六章 系统测试与分析	71
6.1 测试目标	71
6.2 测试软硬件环境	71
6.3 SM 生态兼容性测试	71

6.4 EasyAda-trust 测试	74
6.4.1 系统启动测试	74
6.4.2 CVM 安全启动测试	74
6.4.3 CVM 性能测试	77
6.5 本章小结	79
第七章 总结和展望	80
7.1 全文总结	80
7.2 后续工作展望	81
致 谢	82
参考文献	83
攻读硕士学位期间取得的成果	87

图目录

图 1-1	虚拟化技术分类	4
图 2-1	未引入硬件虚拟化扩展时 RISC-V 系统架构	9
图 2-2	引入硬件虚拟化扩展后的 RISC-V 系统架构	13
图 2-3	基于系统抽象层级与软件执行模型的 TEE 分类	14
图 2-4	微内核操作系统 EasyAda 总体架构	17
图 2-5	EasyAda-Hypervisor 总体架构	19
图 3-1	非机密虚拟机可信基	24
图 3-2	机密虚拟机可信基	25
图 3-3	EasyAda-SM 总体架构	27
图 3-4	EasyAda-SM 的内存管理模式	28
图 3-5	CVM 二阶段页表初始化	31
图 3-6	CVM 的 I/O 数据保护机制	36
图 3-7	硬件加速器管理 CVM 设计	38
图 4-1	安全区域和内存管理层实现	40
图 4-2	PageAllocator 结构	43
图 4-3	PMP 槽结构	45
图 4-4	PMP 运行时配置	48
图 4-5	PageAllocator 结构	48
图 4-6	不同类型 Enclave 与区域类型和分配器的关联关系	49
图 4-7	CVM 管理器结构	50
图 4-8	EasyAda -SM 中非安全/安全设备的 IOPMP 管理	54
图 5-1	EasyAda-SM 签名验证流程	56
图 5-2	机密虚拟机验证信息封装流程	57
图 5-3	EasyAda-trust 中双向验证流程	58
图 5-4	EasyAda-trust 下 hart 分区示意图	60
图 5-5	EasyAda-trust 下 CVM 的 I/O 密钥管理	67
图 5-6	引入安全设备后的 CVM 安全内存视图	68

图 6-1	原生环境与 EasyAda-SM 环境下 Coremark 的对比测试	72
图 6-2	原生环境与 EasyAda-SM 环境下 RV8 benchmark 测试结果对比	73
图 6-3	SBI 固件初始化	75
图 6-4	EasyAda-SM 初始化测试	76
图 6-5	可信 CVM 在可信平台上启动测试	76
图 6-6	不可信 CVM 在可信平台上启动测试	77
图 6-7	可信 CVM 在不可信平台上启动测试	77
图 6-8	LEBench 测试结果	78
图 6-9	安全设备直通架构与半虚拟化机制（不可信共享内存）下的网卡 I/O 对比测试结果	79

第一章 绪论

1.1 研究工作的背景与意义

近年来，物联网（Internet of Things, IoT）和嵌入式技术的迅速发展正在推动各行业向智能化方向升级。物联网通过将大量边缘设备纳入网络管理，实现对边缘数据的收集、传输和处理，并且已在智能家居、工业自动化、智慧城市和医疗健康等领域得到了广泛应用。作为物联网的核心，嵌入式系统在设备控制、传感器数据采集与监测、实时数据处理等方面发挥着关键作用，为各种智能设备提供技术支撑。

然而，随着物联网设备功能的不断扩展，嵌入式技术正面临更加复杂的应用需求和安全问题：（1）在场景需求方面，多样化的应用场景对嵌入式系统的实时性、可靠性和资源利用效率提出了差异化要求。在关键安全场景中，系统需要确保高实时性、可靠性和安全性；而消费电子产品则更关注低成本和高能效。传统嵌入式系统通常只运行单一操作系统，在需要兼顾多种功能需求时，往往依赖物理硬件的堆叠，从而导致系统成本、复杂性和功耗的显著提升；而基于 OpenAMP^[1]的多操作系统部署方案虽然可以在一定程度上兼顾多种功能需求，但无法很好地确保操作系统间的资源隔离与安全性。（2）在安全威胁方面，传统嵌入式系统通常针对特定场景高度定制化，软硬件高度固化，因此攻击面较小。然而，随着边缘计算和 IoT 的发展，嵌入式设备需要承担更复杂的计算任务，并依赖互联网实现设备控制和数据处理。系统代码量的膨胀和开放的网络环境导致嵌入式系统的攻击面急剧增大，对其安全性提出了更高的要求。

为了应对嵌入式硬件资源丰富化和应用场景复杂化的趋势，现代嵌入式系统正在从单一操作系统架构演变为混合关键性系统（Mixed-Criticality System, MCS），以提升系统资源利用率和安全性，并降低重复开发成本。现代 MCS 通过引入虚拟化技术对物理硬件进行抽象，实现在同一硬件平台上部署多个操作系统，从而应对不同应用场景的差异化需求；并且通过隔离不同运行环境间的内存、CPU 和 I/O 等资源，增强了系统的稳定性和可靠性。虽然虚拟化技术以虚拟机监视器（Hypervisor）作为可信计算基（Trusted Computing Base, TCB），可以为不同的虚拟机和 Hypervisor 划分出相互隔离的执行环境，但通用 Hypervisor 通常具有复杂的执行逻辑和庞大的代码量（例如 Linux KVM 大量复用了 Linux 的现有代码和功能），导致其攻击面较大，极易遭受攻击^[2]。因此，Hypervisor 虽然提高了系统资源的利用率，却难以有效保护用户的隐私数据。

为了确保数据得到有效保护，用于隔离敏感代码与数据的可信执行环境（Trusted Execution Environment, TEE）在过去十年间得到了广泛普及。目前，所有主流 CPU 架构均已推出或支持各自的 TEE 解决方案（如 ARM 的 TrustZone^[3]、Intel 的 SGX^[4]、AMD 的 SEV^[5]，以及 RISC-V 的 Penglai^[6] 和 Keystone^[7] 等），以提供一个安全的执行环境，该环境通常被称为飞地（Enclave）或安全世界（Secure World）。TEE 的应用场景广泛，可部署在云服务器、移动设备、IoT 设备、传感器以及硬件令牌等环境中。

目前，嵌入式场景下基于可信操作系统的 TEE 方案（例如基于 ARM TrustZone 扩展与 GP-TEE^[8] 规范的实现体系）虽然应用广泛，但仍存在部分缺陷：（1）闭源性限制：ARM 指令集的闭源特性导致开发者难以根据具体需求进行精细化定制；（2）扩展性不足：ARM TrustZone 将运行环境划分为安全世界（Secure World, SW）和普通世界（Normal World, NW），所有的可信应用（Trusted Application, TA）均共享同一个 SW，TA 间的资源隔离主要依赖 TEE-OS。这导致 TCB 较大，容易引入代码漏洞或被攻破；并且两个世界间的资源难以在运行时动态调整，系统灵活性较差；（3）开发成本高昂：在该框架下，厂商必须开发专用的 TEE-OS 并基于此系统开发 TA，开发门槛和难度较高。边缘计算的发展进一步放大了这一成本，端侧 AI 需要处理大量用户隐私数据，必须得到严格保护。然而，将庞大的 AI 模型完整移植到资源受限且运行专有 TEE-OS 的 TEE 中，或者仅使用 TrustZone 保护关键计算路径并通过复杂的软硬件逻辑保护上下文及 I/O 数据，都极大地增加了可信应用的开发难度。

为了保证可信应用安全性的同时充分复用现有软硬件生态，从而降低 TEE 的准入门槛与厂商的开发成本，亟需将虚拟化技术的灵活抽象能力与 TEE 的硬件级隔离能力相互结合。为此，主要 CPU 厂商均提出了基于虚拟机（VM）抽象的混合 TEE 方案（如 ARM CCA^[9]、Intel TDX^[10]、RISC-V CoVE 等）。通过将复杂的 Hypervisor 移出 TCB，并在 TEE 中直接运行未经修改的机密虚拟机（Confidential Virtual Machine, CVM），使得原有的可信应用能够无缝迁移至 TEE 中，极大地降低了厂商的开发成本。

然而，现有基于 VM 抽象的 TEE 方案主要面向服务器场景，无法直接扩展至资源和成本受到严格限制的嵌入式场景。此外，基于 VM 的 TEE 方案由于引入了完整的 VM，导致系统整体 TCB 剧增，显著增加了 TEE 内部的代码漏洞数量和攻击面，严重威胁系统的安全性和稳定性。目前披露的 TEE 代码漏洞中，多数源于非内存安全语言（如 C/C++）编写的程序因不安全的内存访问而引发的内存破坏^[11]。为了尽可能消除系统中的安全隐患，必须严格限制 TCB（尤其是运行在最高特权级的安全固件）中的不安全内存访问行为。

Rust^[12] 作为一种内存安全的编程语言，能够通过编译时的严格检查机制避免内存越界等问题，从而兼顾嵌入式场景对可靠性、安全性、实时性和性能的严苛要求。尽管目前已有部分研究^[13, 14] 采用 Rust 实现了 TA 以消除内存漏洞，但采用 Rust 构建底层软件 TCB 的相关研究工作依然处于空白阶段。

RISC-V 是一种基于经典精简指令集（RISC）原理的开源指令集架构（ISA）。得益于其开放与模块化的架构特性，RISC-V 能够在保持简洁性与可靠性的同时支持多种硬件平台。系统可根据场景需求集成可选的 RISC-V 标准扩展，甚至实现用户自定义的扩展指令，这使得 RISC-V 能够覆盖从嵌入式微控制器到高性能计算的广泛场景。近年来，RISC-V 在嵌入式领域得到了广泛应用，主流工具链和操作系统均已完善对 RISC-V 的支持。虽然目前 RISC-V 已发布硬件辅助的虚拟化扩展（H 扩展）并提供了支持构建 TEE 方案的硬件基础，但相关的系统级融合研究仍处于起步阶段。

综上所述，结合开源、可扩展的 RISC-V 指令集，实现支持运行机密虚拟机的 TEE 方案，并消除 TCB 中存在的内存漏洞，对于在确保嵌入式系统实时性与可靠性的同时，提升系统安全性与资源利用率、降低 TEE 开发成本及准入门槛，具有重大的学术价值与工程研究意义。

1.2 国内外研究背景与现状

1.2.1 虚拟化技术研究现状

虚拟化技术的逻辑起源可追溯至 20 世纪 70 年代，其核心本质在于通过构建硬件抽象层实现物理资源与执行环境的解耦。1974 年，Popek 与 Goldberg 提出了虚拟机监视器（VMM）的经典理论框架，界定了合格 VMM 须具备的等效性（Equivalence）、高效性（Efficiency）及资源控制（Resource Control）三大关键属性^[15]。

虚拟化的实现基石在于“陷入-模拟”（Trap-and-Emulate）机制，该机制深度依赖于指令集中特权指令与敏感指令的正交性逻辑：当硬件架构满足敏感指令集为特权指令集的子集时，即具备原生虚拟化能力。在此架构下，客户机操作系统（Guest OS）执行敏感指令将触发特权级陷阱（Trap），由处于最高特权级的 Hypervisor 捕获异常并实施语义模拟，从而在保障物理资源受控的前提下，实现多虚拟机间的强隔离与资源复用^[16]。根据实现路径及 Guest OS 的感知程度，虚拟化技术演进为两种主流范式：全虚拟化通过完全的硬件模拟支持未经修改的 Guest OS，具备极佳的兼容性；而半虚拟化则通过超级调用（Hypercall）接口建立 Guest 与 Hypervisor 的协同演进机制，以适度的非侵入式修改换取更高的交互效率。

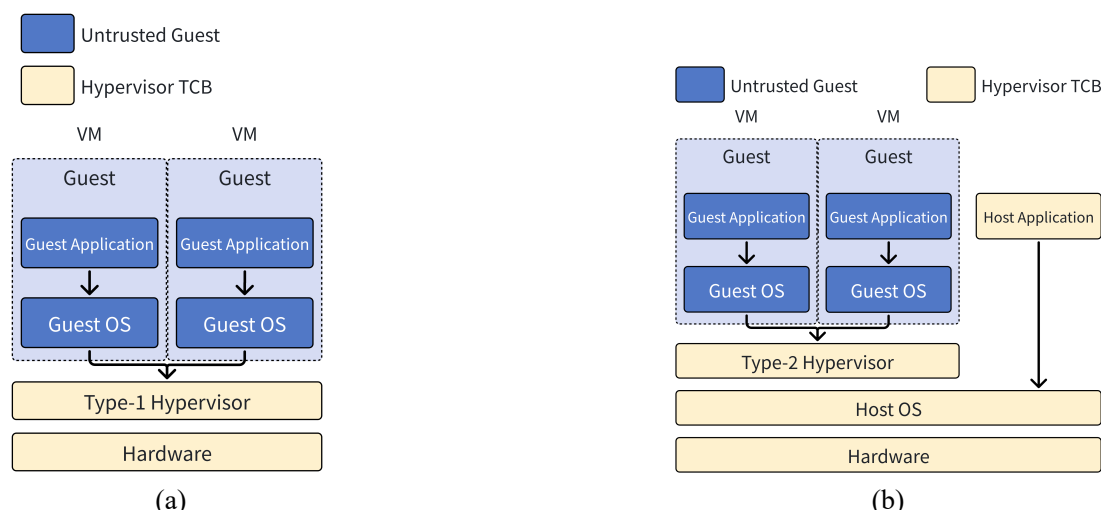


图 1-1 虚拟化技术分类

根据 Hypervisor 与宿主环境的依赖关系，其部署架构可分为 Type-1（裸金属型）与 Type-2（宿主型）两类，如1-1所示。Type-1 架构直接运行于物理硬件之上，拥有对底层资源的绝对控制权，因其访存路径极短且指令处理链路精简，在性能表现与硬件利用率上占据显著优势。相比之下，Type-2 架构寄生于宿主操作系统（Host OS），需跨越 Host OS 的抽象层间接访问硬件。尽管这种间接调用机制引入了额外的性能损耗，但其能够充分复用 Host OS 成熟的驱动体系与管理工具，展现出更强的环境适配性与部署灵活性。

面向传统云服务场景的虚拟机监视器（如 KVM^[17]、Xen^[18] 等）侧重于大规模租户的资源隔离。尽管已有研究尝试对其进行轻量化改造以向嵌入式场景迁移^[19, 20]，但受限于其宏内核架构下庞大的可信计算基（TCB），仍难以满足嵌入式环境对计算资源受限及严苛实时性的双重要求。为此，学术界与工业界相继提出了 Bao、ACRN、XtratuM 等针对高实时性与低功耗定制优化的轻量级方案^[21-23]。然而，此类方案在实际部署中仍面临资源动态调配率偏低及系统架构可扩展性受限等瓶颈。

在车载与工业控制等高实时性关键场景中，基于微内核的虚拟化技术凭借其极简的特权层级与卓越的系统响应性能，成为突破上述瓶颈的优选路径。依托 seL4 微内核构建的 CAmkES-VMM 框架，通过极致的架构精简与低延迟设计，确立了高实时与低开销需求的理想范式^[24]。同时，以 QNX 为代表的商用微内核方案，通过跨架构的强兼容性与关键任务（Mission-critical）调度策略，在自动驾驶等领域印证了微内核架构在兼顾强实时性与高安全性方面的固有优势^[25]。

综上所述，虚拟化技术已从早期的资源模拟演变为支撑现代复杂系统的核心基底。微内核虚拟化技术通过架构级的解耦与精简，在系统灵活性与执行效率之间取得了较好的平衡，契合现代嵌入式系统的复杂虚拟化需求，并已在严苛的实

时性场景中确立了显著的技术优势地位。

1.2.2 虚拟化与 TEE 融合架构及演进现状

随着机密计算技术的不断发展，如何平衡执行环境的安全性、灵活性与生态兼容性成为了学术界与工业界关注的核心议题。根据系统对信任边界的划分及 Enclave 抽象级别的不同，现代隔离技术主要经历了纯虚拟化方案、基于应用/OS 级抽象的 TEE 方案，以及基于机密虚拟机（CVM）抽象的软硬融合 TEE 方案三个演进阶段。

1.2.2.1 纯虚拟化 TEE 技术研究

主流指令集架构（ISA）均已引入成熟的硬件辅助虚拟化技术。传统的虚拟化技术侧重于资源复用与系统级隔离，使 Hypervisor 能够有效屏蔽底层异构硬件的差异。为提高系统的安全性，早期研究尝试通过极度精简 Hypervisor 或将其移出可信计算基（Trusted Computing Base, TCB）来构建安全隔离域。

为优化 Hypervisor 自身的稳健性，Steinberg 等人^[26]借鉴微内核设计理念提出了 NOVA 架构。该方案将资源管理等逻辑移出特权内核，使核心代码缩减至约 9KLoC，并为每个虚拟机分配用户态 VMM 实例以防范单点失效。Jia 等人^[27]提出的 HyperEnclave 架构则通过软件定义实现了与 SGX 兼容的 Enclave 抽象，成功将 TEE 保护能力与特定硬件微码解耦，并采用内存安全的 Rust 语言构建监视器。针对 ARM 架构，Pereira 等人^[28]利用静态分区技术构建了 Bao-Enclave，将安全功能从庞大的特权操作系统中剥离，实现了对硬件资源的精细化访问控制。

基于纯虚拟化的方案凭借硬件平台的广泛兼容性，能够在不依赖特定安全微码的前提下实现基础的物理级隔离。然而，庞大的通用 Hypervisor 一旦被攻破，将直接威胁所有虚拟机的核心数据安全。即使是经过极度精简的轻量级 Hypervisor，其作为特权层软件仍面临软件 TCB 冗余的问题，极易遭受复杂的侧信道分析，且难以防御针对物理内存的硬件探测攻击。纯虚拟化在底层硬件加密原语上的缺失，促使研究者转向架构级的硬件 TEE 方案。

1.2.2.2 基于进程与应用级抽象的细粒度 TEE 技术研究

为弥补纯虚拟化防御硬件攻击能力的不足，硬件厂商引入了具备芯片级内存加密与硬核隔离能力的专有扩展。早期架构级 TEE 主要聚焦于进程、应用或操作系统级别的细粒度抽象。这类方案仅将应用自身及必要的硬件纳入 TCB，以换取极高的理论安全性。

在 x86 架构中，Intel SGX（Software Guard Extensions）^[4]定义了极其严苛的

威胁模型，假定包括 OS 与 Hypervisor 在内的所有系统软件栈均不可信。然而，由于传统可信应用（Trusted Application, TA）多采用 C/C++ 构建，易受内存破坏攻击威胁（Wang 等人^[13]虽提出 Rust-SGX 框架予以缓解，但仍需对代码进行深度重构）。更关键的是，SGX 基于进程的隔离范式受限于受保护内存（EPC）容量与复杂的编程模型，导致深度学习等资源密集型应用面临巨大的迁移挑战。

在嵌入式主导的 ARM 架构中，机密计算常以基于可信 OS 的 TEE 方案作为基础（例如基于 ARM TrustZone 扩展与 GP-TEE^[8]规范的示例实现）。针对 TrustZone 安全世界资源独占的局限，Brasser 等人^[29]提出了 SANCTUARY 架构以动态划分用户态 Enclave；Wang 等人^[30]提出的 RT-TEE 确保了系统在遭遇拒绝服务（Denial of Service, DoS）攻击时的实时性；RusTEE^[14]则在编译阶段静态消除了 TEE 软件栈的内存隐患。

在新兴的开源指令集方面，Keystone 和 Penglai 是当前 RISC-V 主流的 TEE 方案。Penglai^[6, 31, 32]通过引入受护页表（GPT）等机制实现了高达 512GB 的动态隔离空间支持与高可扩展的 I/O 隔离，旨在解决云场景下的性能瓶颈；Keystone^[7]则引入了可编程的安全监视器，将核心功能模块化与可重用化。此外，Ozga 等人^[33]利用形式化验证框架确保了 RISC-V TEE 可信根的确定性。

纯粹的 TEE 技术虽然提供了芯片级的强隔离，但基于进程或 OS 抽象的方案在适配复杂生态时面临极高的开发门槛。开发者必须手动剥离敏感代码以划分 TCB、重写应用程序，或高度依赖特定的 TEE-OS。这种架构不可避免地导致了严重的生态割裂，显著抬高了现有应用向机密计算环境迁移的综合成本。

1.2.2.3 基于机密虚拟机抽象的软硬融合 TEE 技术研究

为了彻底打破纯虚拟化在数据安全性上的瓶颈，同时解决应用/OS 级 TEE 面临的开发门槛高与生态割裂困境，将虚拟化技术的灵活抽象能力与 TEE 专用安全指令拓展相互结合，成为了机密计算演进的必然选择。该范式提出了基于机密虚拟机（Confidential Virtual Machine, CVM）抽象的混合 TEE 架构。通过在硬件级加密的 TEE 中直接运行未经修改的 Guest OS 与大规模应用，最大程度地复用了现有操作系统生态，极大降低了厂商的开发与迁移成本。

为破解灵活性与安全性的双重瓶颈，主流硬件厂商相继推出了支持 CVM 抽象的体系结构。以 x86 架构为例，AMD SEV^[5]（Secure Encrypted Virtualization）利用硬件虚拟化扩展，为每个虚拟机分配唯一的地址空间标识符（ASID），并结合硬件级 AES 引擎确保内存数据以密文形式存储，实现了 Guest VM 与 Hypervisor 的语义脱钩；其增强版 SEV-SNP^[34]则引入反向映射表（RMP）机制，强制防止 Hypervisor 篡改内存权限。Intel 提出的 TDX^[10]架构引入了全新的 SEAM 特权层级

及专用模块，在架构层面彻底剥离了不可信 Hypervisor 对可信域生命周期的控制权。在 ARM 架构中，ARM CCA 同样引入了机密领域（Realm）的概念，以支持虚拟机级别的硬件强隔离。

与此同时，在开源的 RISC-V 生态中，传统多租户计算平台（Multi-tenant computing platforms）由于将平台固件、操作系统、虚拟机监视器及操作人员均纳入租户的可信计算基（TCB），导致其无法满足隐私导向型工作负载对 TCB 最小化的需求。针对这一局限性，Sahita 等人^[35]提出了面向 RISC-V 平台的机密虚拟机扩展（Confidential Virtual-Machine Extension, CoVE）架构。CoVE 充分利用了 RISC-V 架构无历史包袱（Clean slate）的优势，定义了全新的指令集（ISA）与非指令集（Non-ISA）规范，以及系统级芯片（SoC）要求。该架构通过提供具备硬件认证（HW-attested）的 TEE 来实现对使用中数据（Data-in-use）的保护，为构建具备可量化 TCB 的原生机密计算架构奠定了基础。

此外，机密计算不仅在隔离云端庞大的不可信代码中发挥着关键作用，更亟需向资源受限的嵌入式场景延伸。针对这一需求，Ozga 等人^[33]提出了面向嵌入式 RISC-V 系统的机密计算框架 ACE（Assured Confidential Execution）。作为一种开源且免授权费（Royalty-free）的技术方案，ACE 旨在将云端机密计算的安全能力下沉，以构建更安全、更可靠的任务关键型（Mission-critical）嵌入式系统，进一步完善了基于虚拟化扩展的 RISC-V 机密计算应用生态。

综上所述，TEE 架构的研究已从早期的纯软件隔离与进程级细粒度保护，全面转向兼顾高安全性与生态兼容性的虚拟机级保护，通过软硬协同的深度演进，逐步确立了现代机密计算的技术底座。

1.2.3 异构计算环境下 GPU-TEE 安全隔离与防护技术研究

随着端侧人工智能与云端大模型业务的爆发式增长，保障异构计算平台（如 CPU+GPU/NPU）的数据隐私与模型完整性已成为系统安全领域的研究热点。在架构实现上，高性能服务器场景通常采用独立显存的离散式 GPU，TEE 仅需隔离 GPU 的内存映射 I/O（MMIO）区域；而在端侧场景中，集成式硬件加速器通常与 CPU 共享物理内存，不仅需隔离 MMIO，更要求对 GPU 缓冲区实施动态物理隔离。

在基于 CPU 安全特性的监控机制研究中，Wu 等人^[36]提出的 GEVisor 利用现有 Intel SGX 构建了经过形式化验证的安全参考监视器。面向端侧统一内存架构，Deng 等人^[37]在 StrongBox 中利用 ARM TrustZone 技术，通过动态拦截 MMIO 与二阶段页表控制实现了高效隔离。为突破传统 TEE 频繁数据加解密导致的性能瓶颈，Fan 等人^[38]提出的 XpuTEE 框架引入微型监控器 XM，通过联合抽象与直接物理映射实现了“去加密化”监控，将大模型推理开销显著压缩至 2.5% 以下。

为应对新一代指令集架构与存量基础设施的兼容性挑战，机密计算研究逐渐转向软硬协同的深度解耦范式。Wang 等人^[39]提出的 CAGE 框架借助粒度保护检查（GPC）原语率先实现了 ARMv9 架构下的 GPU 硬件级隔离。此外，针对现有机密加速器高度依赖特定硬件更迭的问题，Wang 等人^[40]设计了具备强兼容性的 ccAI 系统。该方案在物理层动态截获并重定向 GPU 任务提交路径，平滑支持了 DeepSeek、Llama 等大模型的机密推理。

尽管异构隔离技术不断演进，Lu 等人^[41]在 MOLE 研究中揭示了新型硬件侧攻击面：GPU 内部集成的微控制器（MCU，如 Falcon/GSP）拥有超越系统 IOMMU 限制的特权级访问能力。攻击者可通过劫持 MCU 固件绕过 CPU 侧监控机制直接窃取 TEE 显存数据。该发现打破了“直通即安全”的传统假设，表明未来的异构机密计算亟需建立涵盖外设内部组件完整性度量的全路径防护体系。

1.3 本文主要贡献和创新

1.4 论文结构

第二章 相关技术理论概论

2.1 RISC-V 背景知识

x86 架构因其漫长的发展历史与向后兼容性负担，指令集日趋复杂庞大。尽管在服务器等高性能计算领域表现良好，却难以契合嵌入式场景对严苛功耗与成本的约束。

ARM 架构作为精简指令集（RISC）的代表，凭借 IP 授权模式赋予下游厂商一定的定制自由度，在嵌入式市场占据主导地位。然而，受限于其封闭的商业壁垒，硬件开发面临高昂的 IP 授权成本；同时，ARM 架构跨代演进（如 ARMv7 至 ARMv8）差异显著，导致了严重的指令集碎片化问题，系统高度依赖特定版本的编译工具链以维持跨平台兼容性与性能优化。

相比之下，新兴的 RISC-V 架构凭借开源开放与高度模块化的设计理念，与系统软件的解耦演进需求高度契合。硬件厂商可根据具体场景按需组合指令集扩展，软件生态亦能据此实现灵活伸缩，有效规避了跨代架构更迭带来的高昂代码移植成本。在云边端协同计算趋势下，资源受限的边缘节点可仅实现基础指令子集以满足极低功耗约束，而云端高性能设备则可集成复杂扩展以跃升算力。RISC-V 这种高度弹性的架构特性，为构建跨越异构硬件的通用、可扩展可信执行环境（TEE）提供了理想的底层硬件支撑

2.1.1 RISC-V 特权级介绍

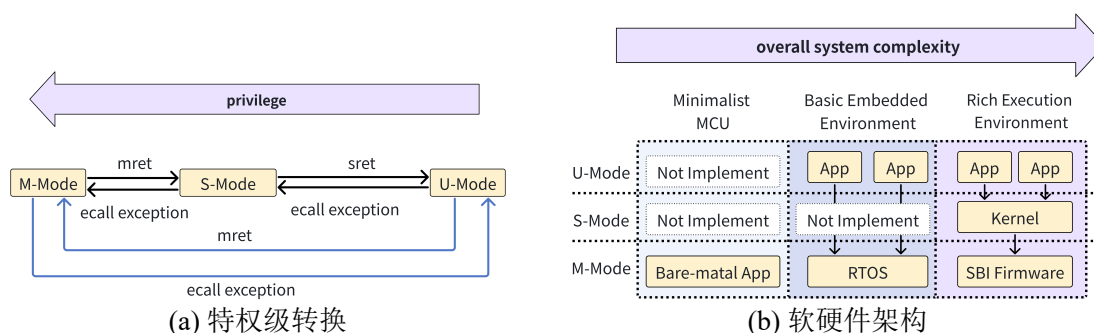


图 2-1 未引入硬件虚拟化扩展时 RISC-V 系统架构

RISC-V 架构将独立的 CPU 核心抽象为硬件线程（Hard Thread, hart）。在任何运行时态下，hart 均处于特定的特权级中。RISC-V 规范定义了由低至高的三种主要特权级：用户模式（U-Mode）、监管者模式（S-Mode）与机器模式（M-Mode）。如图2-1(b)所示，根据指令集标准，M 态为强制实现的基础特权级，而 U 态与 S 态

为可选扩展。实际商用硬件通常基于特定应用场景与功耗考量进行架构裁剪，如运行 RTOS 或裸机应用的 MCU 场景可以实现 U 和 M 态甚至仅实现 M 态。

各特权级之间具备严格的权限隔离边界^[42]：U-Mode 作为最低特权级，仅获准访问通用寄存器并执行基础指令，其访存操作严格受限于页表映射机制；S-Mode 在此基础上，进一步获准访问特定的控制与状态寄存器（Control Status Register, CSR）并执行相关特权指令，其访存操作同样受限于 MMU 页表机制；M 态作为系统最高特权级，拥有对所有物理硬件、CSR 及特权指令的无限制访问权，并可绕过页表直接寻址物理内存。任何低特权级尝试执行越权指令的行为均会触发硬件异常拦截。

在特权级状态转换机制上，系统上电重置时 hart 默认初始化为 M 态。控制流从高特权级向低特权级转移需显式执行 mret 或 sret 指令完成状态降级；反之，当系统发生外部中断或执行异常时，硬件将根据 CSR 中的委派（Delegation）配置，自动陷入（Trap）至 S 态或 M 态的异常向量入口。此外，低特权级软件可主动触发 ecall 或 ebreak 指令发起系统调用，以同步陷入的方式将控制权安全移交至高特权级执行环境。各特权级间的状态转换逻辑如 2-1(a) 所示。

2.1.2 RISC-V 中断与异常处理机制介绍

RISC-V 架构将所有打破正常程序控制流的突发事件统称为陷入（Trap）。在微架构层面，陷入被严格划分为两类：同步的异常（Exception）与异步的中断（Interrupt）。异常通常由当前执行的指令直接引发（如非法指令解码、缺页异常或主动发起的 ecall 系统调用），而中断则由独立于当前执行流的外部异步事件触发。RISC-V 手册规定中断按类型分为外部中断（External Interrupt）、软件中断（Software Interrupt）、时钟中断（Timer Interrupt）三种。

根据 RISC-V 基础特权级规范，系统发生的所有陷入默认均强制路由至最高特权级 M 态处理。然而，在部署了复杂操作系统或 Hypervisor 的场景中，频繁的 M 态陷入与上下文切换将引入不可接受的性能开销。为此，RISC-V 创新性地设计了陷入委派机制。通过精细化配置 M 态异常委派寄存器（medeleg）与中断委派寄存器（mideleg），底层安全固件可将特定类型的陷入事件（如 S 态下的缺页异常或时钟中断）显式旁路，使其直接触发 S 态的陷入响应而无需 M 态介入。这一机制极大地缩短了关键中断的响应延迟，是实现高效软硬件协同与机密虚拟化资源调度的核心架构支撑。

2.1.3 RISC-V 内存保护机制介绍

2.1.3.1 物理内存保护（PMP）机制

物理内存保护（Physical Memory Protection, PMP）机制是 RISC-V 架构标准提供的核心安全原语，也是在该架构上构建 TEE 的必要底层硬件支撑。在微架构层面，每个 hart 均集成了一组专用的 PMP 地址寄存器（`pmpaddrx`）与配置寄存器（`pmpcfgx`），且这些寄存器的读写权限被严格限制在 M 态。

在使能了 PMP 机制的硬件平台中，系统默认采用严格的隔离访问策略：在未显式授权的情况下，运行在更低特权级的软件被默认限制对任何物理地址的访问。通过在 M 态下对 PMP 寄存器组进行精细化配置，系统能够以 hart 为粒度，为其划分特定的连续物理地址空间，并显式授予相应的读、写及执行权限。简言之，PMP 机制从硬件底层提供了一种强隔离的访问控制手段，能够精准限制特定 hart 对任意物理内存区域的访存边界与操作能力。

2.1.3.2 增强的物理内存保护（Smepmp）机制

Smepmp（Supervisor Mode Execution Prevention and Machine-mode Physical Memory Protection）扩展是对 RISC-V 基础 PMP 机制的关键安全增强。在原生 PMP 架构中，锁定位（L 位）的语义存在天然的耦合局限：当 L=0 时，M 态默认拥有对全量物理内存的无限制访问权；当 L=1 时，M 态与低特权级将被强制绑定相同的 R/W/X 权限。这种粗粒度的设计导致系统无法在赋予 S 态合法访存权限的同时有效剥夺 M 态的对应权限，难以防御针对高特权级固件的混淆代理（Confused Deputy）攻击，亦无法从架构层面彻底杜绝 M 态意外执行驻留于低特权级内存中的恶意载荷。

为突破上述架构瓶颈，Smepmp 扩展创新性地引入了机器安全配置寄存器（`mseccfg`）。该寄存器通过三个核心控制位重塑了底层的物理访存控制逻辑：首先是机器模式锁定（MML）位，当 MML 被置位时，系统将彻底重载传统 PMP 配置中 L 位与 R/W/X 位的组合语义，实现了 M 态与更低特权级内存权限的深度解耦。例如系统可配置仅低特权级可读写、M 态不可访问的隔离区域，从而在物理硬件层面严格贯彻了最小特权原则。其次是机器模式白名单策略（MMWP）位，使能该位将逆转 M 态的默认访存行为，从原有的默认放行切换为严苛的默认拒绝（Default-Deny），强制要求 M 态的所有访存操作必须命中显式授权的 PMP 条目。最后是规则锁定旁路（RLB）位，它在维持整体隔离强度的前提下，允许授权的安全固件动态修改已锁定的 PMP 条目，极大提升了 TEE 运行时的内存管理弹性。

综上所述，Smepmp 机制在微架构层面弥补了原生 PMP 权限颗粒度粗糙的设

计缺陷，而原生 PMP 机制以最小的硬件开销构筑了坚实的宏观物理隔离边界，为嵌入式环境提供了兼具高实时性与高性价比的底层安全原语。

2.1.3.3 I/O 物理内存保护（IOPMP）机制

输入输出物理内存保护（Input/Output Physical Memory Protection, IOPMP）是 RISC-V 架构中用于保障系统级外设访存安全的关键硬件扩展。尽管原生的 PMP 机制能够有效限制 hart 的越权访存行为，但它仅作用于 CPU 侧，无法约束具备 DMA 能力的外设。恶意或受控的外设可轻易绕过 CPU 的 PMP 隔离边界，直接对受保护的物理内存发起读取或篡改操作（即 DMA 攻击），这构成了机密计算环境下的重大安全隐患。

为弥补这一架构级安全短板，RISC-V 引入了 IOPMP 扩展。在微架构实现上，IOPMP 通常部署于系统互连总线与外设控制器之间，充当所有 I/O 访存事务的硬件级安全网关。其核心访问控制逻辑基于设备唯一标识（Source ID, SID）展开：当外设发起总线级读写请求时，IOPMP 会首先捕获该请求携带的 SID，并通过内部的路由表将其映射至特定的内存域（Memory Domain, MD）。每个内存域内配置了一组类似于 PMP 的规则条目，用于精确界定该 SID 对应的设备所能访问的连续物理地址范围及其读写权限。任何未命中授权条目的 DMA 请求均会被 IOPMP 硬件直接拦截并上报异常。

2.1.4 RISC-V 硬件虚拟化扩展（H-extension）介绍

RISC-V 硬件虚拟化扩展（H-extension）是 RISC-V 指令集架构中专为提升 Hypervisor 运行效率而设计的关键硬件扩展。传统软件模拟的虚拟化方案面临高昂的陷入-模拟开销，而 H 扩展通过在处理器微架构层面引入新的特权级、寄存器副本以及二阶段地址翻译机制，从硬件底层为高效构建 Type-1 或 Type-2 型 Hypervisor 提供了系统级支撑。

如图2-2所示，使能 H 扩展后，原有的 S 态被扩展为 Hypervisor 扩展监管者模式（HS-Mode），同时新增了虚拟监管者模式（VS-Mode）与虚拟用户模式（VU-Mode）。在 HS 态下运行的 Hypervisor 拥有对物理硬件与虚拟机的完全控制权；而 Guest OS 及 Guest App 则分别运行在 VS 态与 VU 态下。这种设计确保了 Guest OS 无需修改核心代码即可无感运行，并在试图执行影响全局的敏感特权指令时安全地触发硬件异常并陷入 HS 态处理。

频繁的虚拟机退出（VM-Exit）是制约虚拟化性能的核心瓶颈之一。为此，H 扩展为 VS 态引入了一套完整的 S 态寄存器影子副本（如 vsstatus、vstvec、vsepc、vsscratch）等。当处理器运行在虚拟化环境（V=1）时，Guest OS 对常规 S 态寄存

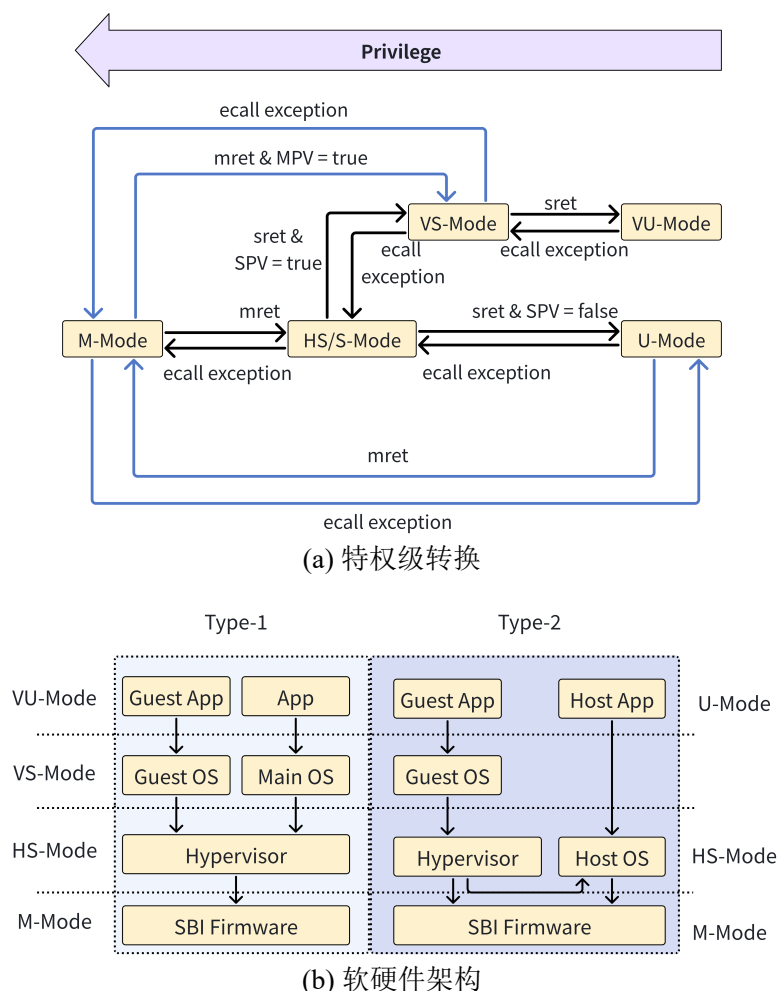


图 2-2 引入硬件虚拟化扩展后的 RISC-V 系统架构

器的读写操作会被硬件自动重定向至对应的虚拟控制状态寄存器中。这一机制使得客户系统在配置自身上下文时无需陷入 HS 态，极大地降低了寄存器访问带来的上下文切换开销。

为实现虚拟机间的物理内存强隔离，H 扩展引入了二阶段页表机制。第一阶段（Stage-1）完全由 Guest OS 控制，负责将虚拟地址（VA）翻译为客户物理地址（GPA），其页表基址由虚拟监管者地址转换与保护寄存器（`vsatp`）管理；第二阶段（Stage-2）由 Hypervisor 严密管控，负责将 GPA 强制翻译为真实的宿主物理地址（HPA），其页表基址由 Hypervisor 客户地址转换与保护寄存器（`hvatp`）管理。MMU 在处理 VS/VU-Mode 的访存请求时，会自动串行执行这两阶翻译，彻底阻断了虚拟机越权访问宿主物理内存或其他虚拟机地址空间的可能。

同时，H 扩展增强了 RISC-V 原有的委派机制。通过配置 Hypervisor 级的中断与异常委派寄存器（`hideleg` 与 `hedeleg`），系统可将原本默认陷入 HS 态的特定异常（如 VS 态系统调用或缺页异常）直接旁路委派至 VS 态，交由 Guest OS 内部的处理。

理程序直接响应。此外，H 扩展设计了完善的虚拟中断注入机制（如通过 `hvip` 寄存器配置 `VSSIP`、`VSTIP`、`VSEIP`），允许 Hypervisor 在软件层面灵活模拟虚拟硬件外设，并高效地向目标 VS 态挂起并注入虚拟中断信号。

2.2 可信执行环境（TEE）相关技术介绍

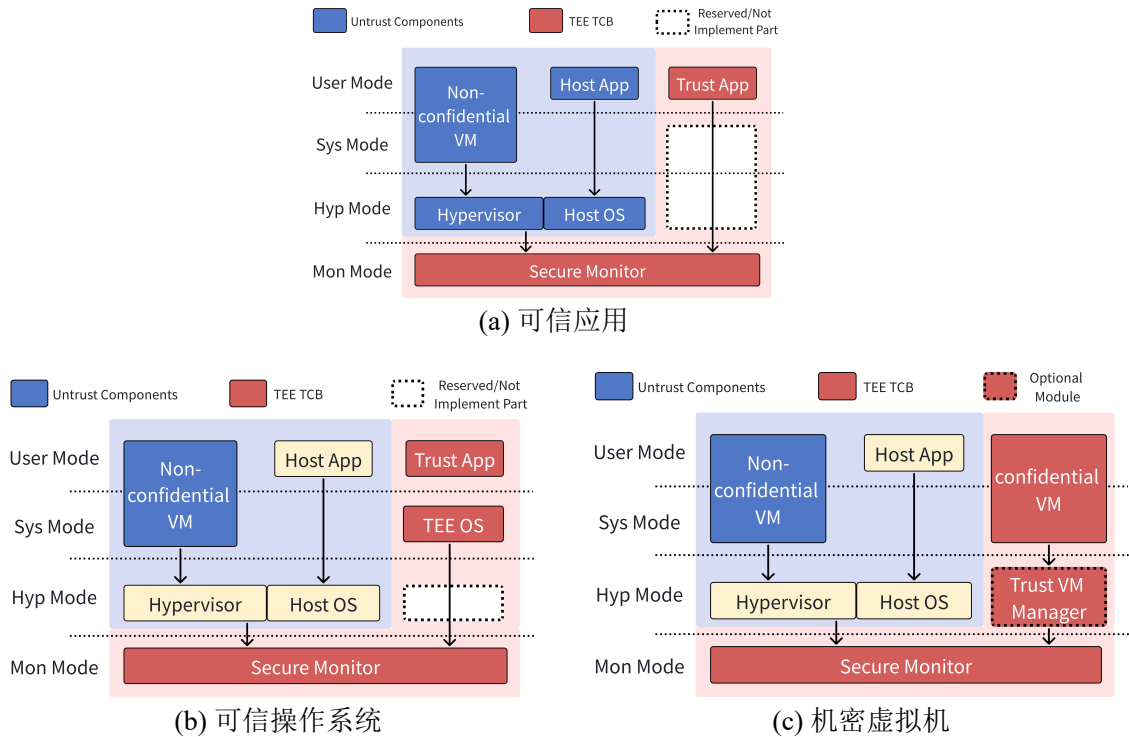


图 2-3 基于系统抽象层级与软件执行模型的 TEE 分类

TEE 通过在硬件微架构层面构建受保护的隔离执行空间（该隔离抽象在不同技术语境下命名各异，如 Intel TDX 的可信域、Intel SGX 的飞地、ARM TrustZone 的安全世界以及 IBM ACE 的可信虚拟机等），通过对传入和传出的数据进行加密，为敏感代码与执行状态确立了系统级的机密性与完整性保障。尤为关键的是，TEE 能够从指令集架构底层重塑信任边界，有效抵御来自系统中任何潜在恶意实体或高特权级系统软件（如不可信的宿主操作系统内核或虚拟机监视器）的越权访问与潜在攻击。相较于全同态加密或多方安全计算等纯密码学计算范式，TEE 凭借以下三大核心特性展现出显著的技术实用性与工程优势^[43]：

1. **通用可编程性（General Purpose Computation）**：TEE 内部所支持的执行功能通常表现为用户自定义程序，这赋予了其图灵完备的通用计算能力，使其能够广泛适配各类复杂的业务功能与算法形态。

2. **强机密性与执行完整性（Integrity and Privacy）**：TEE 在物理信任边界内同

时提供双重安全属性。机密性（Confidentiality）^[44] 确保了外部潜在攻击者无法从计算过程中窥探或获取任何敏感信息，而完整性（Integrity）则从底层机制上保证了目标程序的执行逻辑免遭篡改并被严格且正确地执行。

3. 近似原生的计算性能（High Performance）：TEE 的核心计算范式是在安全飞地内部将所有加密输入解密后，直接依托物理 CPU 在明文状态下执行目标自定义程序。因此，除了处理加密数据输入输出所产生的额外开销外，TEE 能够充分释放并利用 CPU 的原生运算速度，彻底规避了纯密码学方案中高昂的算力惩罚，具备极高的高性能优势。

根据系统抽象层级与软件执行模型的差异，如图2-3所示，当前业界的 TEE 架构主要可归纳为以下三种演进范式：

1. 面向可信应用（Trusted Application）的 TEE 架构：该架构下的 Enclave 主要承载深度定制的用户态安全应用。其执行逻辑高度依赖于特定平台提供的软件开发工具包（SDK）及底层安全固件，以此实现与不可信宿主的跨边界上下文交互。此轻量级隔离范式的典型代表包括 Intel SGX 以及 RISC-V 体系下的 Penglai 等。

2. 面向可信操作系统（Trusted OS）的 TEE 架构：该架构在 Enclave 内部引入了定制化的可信操作系统，能够支持多个可信应用的并发执行与精细化调度，并通过安全固件桥接与宿主环境的通信。其工业界标杆为 ARM TrustZone 技术及其衍生的 GP-TEE（GlobalPlatform TEE 规范）。其中，TrustZone 筑牢了包含信任根（RoT）、安全启动及物理内存强隔离在内的底层硬件原语；而 GP-TEE 则在软件抽象层标准化了功能组件模型与跨域交互协议。在 RISC-V 生态中，Keystone 框架亦提供了与上述架构高度对标的系统级隔离能力。

3. 面向机密虚拟机（Confidential VM）的 TEE 架构：传统的 TA 与 TOS 架构对上层业务软件存在较高的侵入性，要求开发者付出高昂的代码重构成本。为破解这一应用迁移瓶颈，现代主流指令集均演进出了基于 CVM 的隔离方案。该架构通过提升 Enclave 的系统级抽象维度，允许用户仅需将 VMM/Hypervisor 及相关依赖部署于可信环境中，即可透明地承载无修改的客户操作系统。同时，不可信宿主侧仅需极少量的底层适配，即可完整支持对 CVM 生命周期的安全管理。此架构代表了当前机密计算的前沿趋势，典型实现涵盖 Intel TDX、AMD SEV-SNP、ARM CCA，以及 RISC-V 架构下的机密虚拟化扩展（CoVE）等。

2.2.1 面向异构计算环境的 TEE

业界已经在 CPU 侧建立起了相对成熟的 TEE 机制（如 Intel SGX、ARM TrustZone 等），用于隔离和保护敏感数据。然而，随着机器学习、大数据处理和区块链等数据密集型应用的爆发，单核或多核 CPU 的算力已无法满足海量数据的高性

能计算需求。为了在利用 GPU、NPU 和 FPGA 等硬件加速器提供澎湃算力的同时，依然能够享受高强度的机密性、完整性与隔离性保护，将 TEE 的安全信任边界从主机 CPU 延伸至异构加速器设备（即构建加速器 TEE 或异构 TEE）成为了学术界与工业界的必然演进方向。这种信任边界的延伸不仅需要保护传统的明文数据和代码，还必须涵盖 AI 模型参数、输入输出特征、页表映射以及底层硬件环境状态。

根据加速器与主机 CPU 在物理架构与互连方式上的差异，当前异构计算环境的集成范式主要可划分为以下三类。不同的范式对底层安全隔离机制与 TEE 架构设计提出了截然不同的挑战：

1. 指令集扩展（ISA Extension）：该范式将 AI 或密码学运算逻辑直接嵌入 CPU 的执行流水线中，通过 RISC-V 等架构预留的编码空间实现自定义指令扩展（如矩阵运算或向量增强）。加速单元共享处理器的通用寄存器组或向量寄存器，加速指令由 CPU 译码后直接分发至执行单元。此模式消除了数据在异构组件间的移动开销，具有极高的指令响应速度和极低的上下文切换延迟。在 TEE 视角下，其信任边界完全收敛在 CPU 物理核心内部，复用原生的 CPU 特权级与内存隔离机制即可实现高安全性的保护。它适用于细粒度、高频次的张量运算，是追求极致单核计算密度及实时性场景的首选理论模型。

2. 片上协处理器（On-chip Coprocessor）：此范式下，加速器作为 SoC 内部独立的硬件 IP 存在，通过片上系统总线与 CPU 交互。加速器拥有独立的控制逻辑与局部存储空间，由 CPU 通过内存映射 I/O（MMIO）进行任务配置和启动，并通过 DMA 自主完成大规模数据的搬运。通过异步工作机制，CPU 仅负责高层调度，而加速器专注于流水化处理大体量数据。从安全维度来看，协处理器通常与 CPU 共享片上物理内存，其 TEE 设计需重点防范恶意的越权 DMA 攻击，高度依赖诸如 IOPMP 等总线级物理内存保护硬件来确立硬件级隔离边界。这种模式在能效比与系统灵活性之间进行了平衡，是当前主流嵌入式 AI SoC 的硬件范式。

3. 片外独立加速器（Discrete Accelerator）：加速器位于独立的物理芯片或插卡上，通过高速串行总线（如 PCIe 或 CXL）与主机互联。此类加速器通常具备独立的高带宽全局显存（如 HBM/GDDR 和复杂的层次化缓存结构。系统通过驱动程序管理设备内存空间，并采用命令队列模式进行大规模任务的批处理提交。其逻辑设计的复杂度不再受限于主机 SoC 的热设计功耗与封装面积，能够提供极高的峰值算力。然而在机密计算场景中，由于跨物理总线通信的存在，该范式不仅对主机侧的安全隔离机制提出了更高的时延容忍度要求，还必须引入端到端的内存加密引擎（MEE）与总线加密协议（如 TDISP），以彻底抵御总线探测、数据篡改与物理重放攻击。

尽管异构 TEE 研究在近年来呈现出爆发式增长，但其在真实物理设备上的大

规模部署仍面临严峻的工程与理论痛点。首先是系统架构的碎片化与通用性缺失，现存的大量加速器 TEE 方案往往强绑定于特定的“CPU-加速器”硬件组合，缺乏标准化的软硬件交互抽象，导致跨平台迁移极其困难。其次是可信计算基的急剧膨胀，为了支持复杂的异构加速器调度，传统方案往往被迫将体量庞大的设备驱动程序（如动辄百万行代码的 GPU 软件栈）直接纳入可信环境中，这严重违背了安全固件微内核化的极简设计初衷，剧增了系统的潜在攻击面。

2.3 微内核操作系统 EasyAda 介绍

EasyAda 是一款自研微内核操作系统，如图2-4所示，其设计理念遵循最小特权原则，旨在构建高安全性、高可靠性的操作系统内核，适用于对稳定性和安全性要求较高的嵌入式应用场景。EasyAda 将内核态精简为仅保留必要的特权功能模

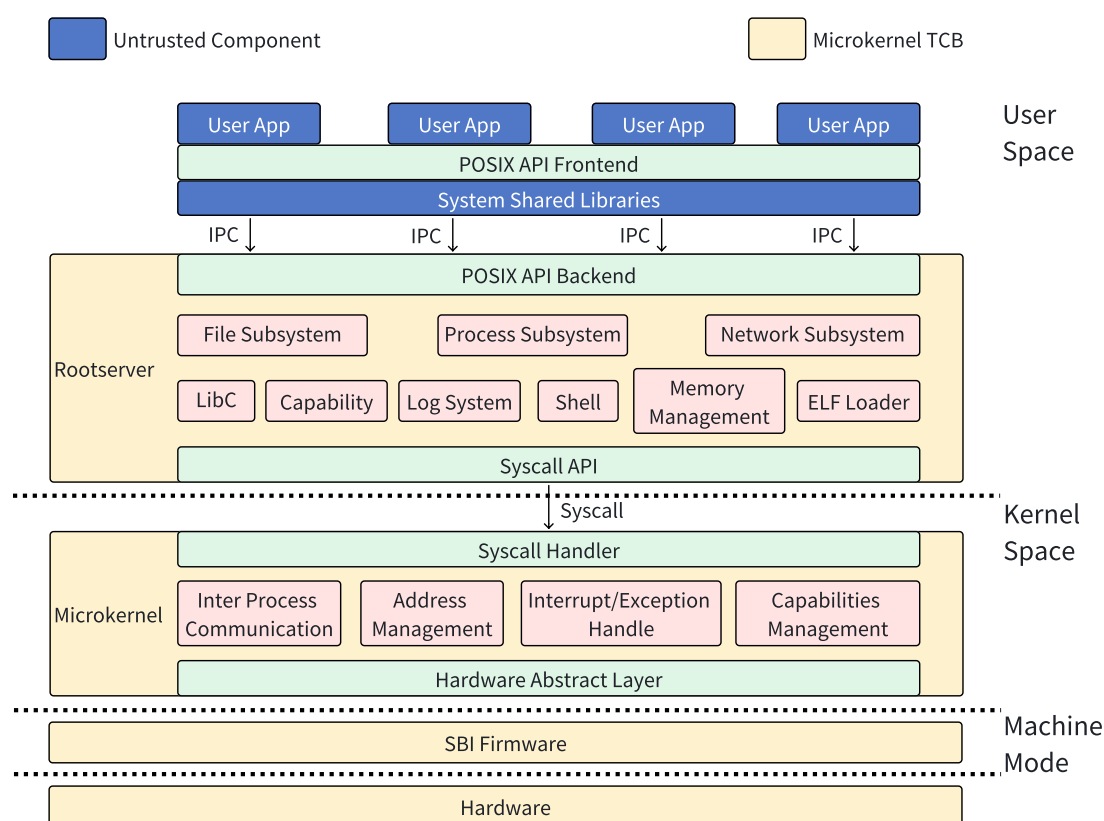


图 2-4 微内核操作系统 EasyAda 总体架构

块，如核间通信、权能管理、中断与异常处理、时钟管理、地址空间管理和内核对象管理等核心组件，内核将对外提供的服务封装为系统调用接口，仅暴露较小的攻击面。同时，通过硬件兼容层（Hardware Abstract Layer，HAL）抽象了操作系统依赖的硬件原语，如启动/暂停 CPU 核心，中断配置和处理，MMU 配置等，实现了统一的硬件操作接口，简化了操作系统在不同指令集和架构间移植的难度并

且有效降低了内核代码的复杂度，确保 EasyAda 可以在异构环境上对外提供统一的软件平台。

EasyAda 将除核心功能外的其余大部分服务迁移至用户态，在地址空间隔离的上下文中运行，并由根服务（Rootserver）统一管理。根服务作为一个运行在用户态的多线程进程，集成了多个功能子系统。这些子系统既包括独立的模块化组件，如文件系统、进程管理、网络协议栈等，也包括执行特定任务的独立线程，如 Shell 交互模块等。根服务将对外提供的服务封装为 RPC 接口，为操作系统提供统一、透明的服务访问接口，并且对 RPC 参数进行严格的检查以防范外部攻击。其他服务或用户应用可以通过系统共享库提供的 POSIX API 前端，将函数调用转换为 RPC 调用。同时，所有应用和系统服务的拥有独立的权能空间，仅能访问当前权能空间支持的 RPC 接口和系统调用，进一步缩小系统攻击面。

基于微内核的设计思想，EasyAda 相比 Linux 等宏内核操作系统具备更小的软件 TCB，而相比传统的 RTOS 如 FreeRTOS 等，EasyAda 又具备强大的可扩展性，可以通过扩展系统服务实现与宏内核对等的功能和性能。因此 EasyAda 通过灵活调整用户态系统服务配置，可以适应边缘到中高端嵌入式等多样化的场景和需求，为云边端场景提供通用的 OS 底座。

因为 EasyAda 被设计为面向多样化场景的通用操作系统，所以其应用和系统服务运行在 U 态，内核运行在 S 态（未开启虚拟化）/HS 态（开启虚拟化支持），并且通过 SBI 请求与 M 态 SBI 固件进行交互以实现硬件管理。

2.4 微内核虚拟化方案 EasyAda-Hypervisor 介绍

图2-5展示了支持管理 CVM 生命周期的基于 RISC-V 的 EasyAda-Hypervisor 设计。EasyAda-Hypervisor 仍然遵守了 EasyAda 微内核的设计哲学和思路：

1. 统一资源管理：主要系统资源由 EasyAda 负责管理和分配。充分复用微内核操作系统现有代码和能力，实现统一的资源管理并减少 EasyAda-Hypervisor 的代码量，缩小其 TCB。

2. Hypervisor 解耦：Hypervisor 被划分为运行在 EasyAda 用户态下的 VMM 和内核态下的 Hypervisor 驱动，以尽可能复用 EasyAda 现有代码和功能。前者负责主要的 VM 管理工作，包括管理 VM 生命周期和软硬件资源，模拟虚拟化设备后端，以及管理 VM 服务线程等；后者作为屏蔽不同架构硬件虚拟化扩展实现的抽象层，对外提供基于虚拟化依赖的硬件原语抽象的统一接口，完成对硬件虚拟化扩展相关寄存器进行配置，以及接收和处理 VM 中断和异常等。

3. 静态分区：Hypervisor 采用静态分区策略，虽然该策略缺乏灵活性，但是因

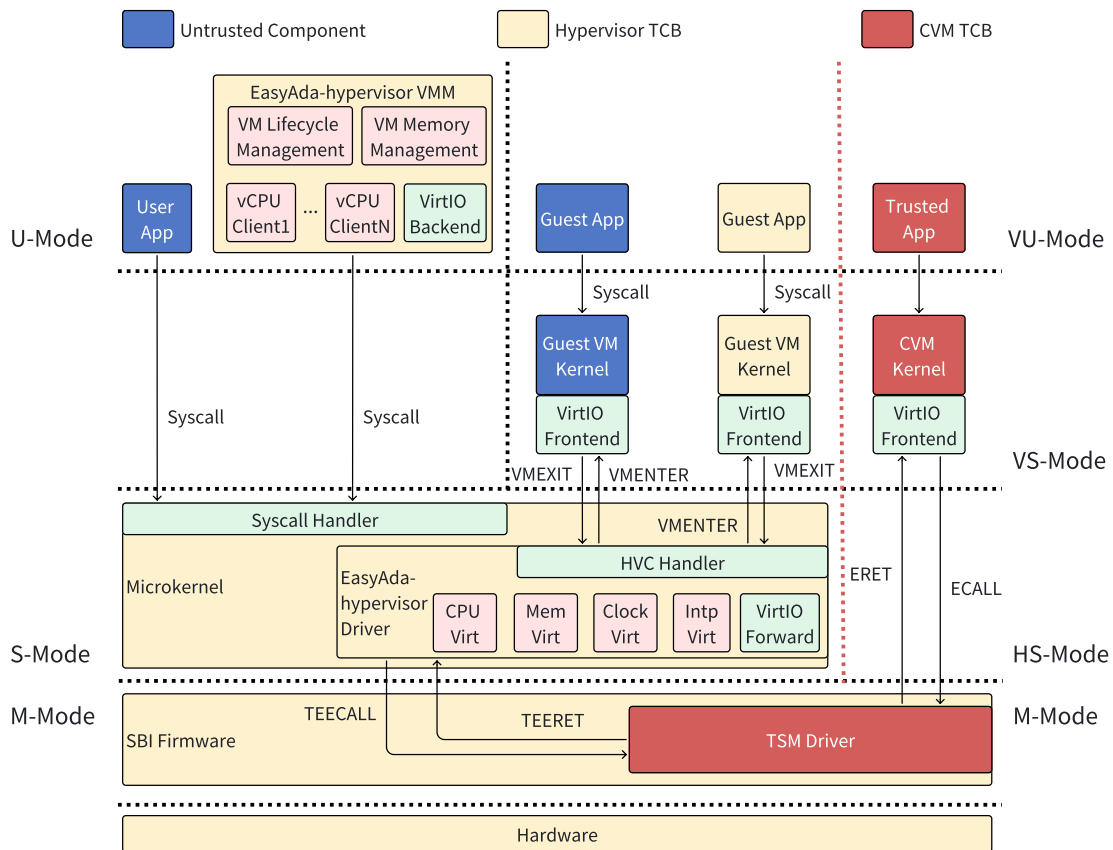


图 2-5 EasyAda-Hypervisor 总体架构

为嵌入式场景应用高度定制化的特点，静态分区策略足够满足其功能需求。并且静态分区策略可以简化代码逻辑，缩小 Hypervisor 的攻击面，这一点在引入 CVM 支持之后更加重要，因为标准 RISC-V CoVE 扩展要求 SM 实现数十个 SBI 接口以实现对 CVM 资源的细粒度控制，显著增大了 TCB 的攻击面，并且导致 hypervisor 需要数个 SBI 调用才能完成一个功能，严重影响了系统的实时性和性能。

EasyAda-Hypervisor 通过在 EasyAda 中引入两个核心组件实现基于硬件虚拟化扩展的 VM 生命周期管理：（1）运行在内核态（HS 态）的 Hypervisor 驱动，负责抽象虚拟化硬件原语并对上提供虚拟化能力接口。其虚拟化能力主要可以分为以下五类：（1）CPU 虚拟化：目标是基于虚拟化专用特权级（RISC-V 的 VS/VU/HS 态），为 VM 提供符合其运行要求数量的 vhart 并在隔离的硬件环境中运行 VM 上下文，在实现操作系统无感迁移到虚拟化环境中运行的同时，避免 VM 访问和篡改 Hypervisor 或其他 VM 的上下文信息；（2）内存虚拟化：目标是基于硬件虚拟化提供的二阶段地址翻译能力，通过动态配置和管理 VM 的二阶段页表，实现细粒度的，VM 无感的强制内存隔离；（3）中断虚拟化：目标是基于硬件提供的中断控制器（包括本地中断控制器和平台中断控制器等），通过中断注入或中断直通等手

段, 实现 VM 中断管理的同时, 避免 VM 接收和配置不属于当前 VM 的中断; (4) 时钟虚拟化: 目标是基于虚拟硬件时钟, 通过虚拟化计时器和虚拟时钟补偿计时器, 实现 VM 无感退出/恢复; (5) I/O 虚拟化: 目标是基于共享物理内存和虚拟中断, 通过为不同 VM 模拟虚拟设备, 在保证 VM 无感迁移的同时, 提升 VM 的 I/O 性能和安全性。

用户态 VMM 主要包括两个部分: VM 资源和生命周期管理模块和 vhart 服务线程, 负责提供完整的虚拟机管理与运行支持。VM 资源和生命周期管理模块负责虚拟机的生命周期管理, 包括创建、运行、暂停、恢复及删除虚拟机等功能, 并且通过 Shell 和 RPC 接口对外提供交互式接口, 简化用户对虚拟机实例的管理。

vhart 服务线程可以视为 vhart 的调度实体和 VM 在 host 侧的用户接口, 作为 VirtIO 后端负责响应 VM 的 I/O 请求。以实现 I/O 虚拟化和中断处理的高效执行。vhart 服务线程运行于 VMM 进程上下文中, 与 VMM 及内核态虚拟化组件协同工作, 优化虚拟机的计算与 I/O 任务调度。

2.5 Rust 相关知识介绍

Rust^[12] 是一种旨在兼顾可靠性与执行效率的系统级编程语言。为了在内存安全 (Memory-safety) 与线程安全 (Thread-safety) 这两个核心维度上实现可靠性, Rust 提供了以下机制: (1) 确立每个数据对象的所有权 (Ownership); (2) 自动检查每个对象的读写权限 (即可变性 (Mutability)); (3) 强制执行针对所有对象的生命周期管理 (Lifetime management); (4) 禁止不安全的类型转换以保障类型安全 (Type-safety); (5) 禁用危险的裸指针 (Raw pointer) 操作, 例如指针别名 (Pointer aliasing) 或悬垂指针 (Dangling pointers)。

在程序编译阶段, 若代码违反了 Rust 的任何安全准则, 编译器将抛出异常并生成详尽的错误诊断信息, 以辅助开发者进行相应的代码修正。除了显著提升代码安全性外, Rust 还带来了其他多项工程优势, 例如高效的并行处理能力、对开发者友好的编译提示, 以及成千上万个用于支撑各类开发需求的 Crate (功能类似于 C 语言中的 Library)。

尽管 Rust 默认被设计为遵循严格的安全准则, 但为了确保其具备编写任何底层系统程序的能力, 它同样提供了 unsafe 关键字, 允许开发者显式地注入非内存安全的代码片段。Rust 保留此 unsafe 选项主要基于两方面核心考量: (1) 允许开发者实现某些无法通过编译器默认静态检查的“特殊”功能逻辑; (2) 允许代码直接与底层操作系统或物理硬件组件进行交互。被标记为 unsafe 的代码域能够绕过 Rust 的内置安全检查器, 因此可能会执行存在潜在脆弱性的操作, 例如对不可变

变量发起写操作、执行非标准的数据类型转换，或是直接操作并解引用裸指针。在 Rust 开发中，使用 `unsafe` 代码段的一个典型场景是必须调用基于 C 语言构建的底层函数，该机制在 Rust 中被严格定义为外部函数接口（Foreign Function Interface, FFI）。然而，`unsafe Rust` 在带来底层控制力与能力扩展优势的同时，亦不可避免地引入了安全隐患，`unsafe Rust` 极易为系统引入潜在的安全风险漏洞。

2.6 本章小结

本章系统性地阐述了构建异构机密计算底座相关的核心技术理论。首先，详细分析了 RISC-V 架构的特权级模型与陷入委派机制，探讨了包含 PMP、Smepmp 和 IOPMP 在内的物理内存保护扩展机制，并论述了硬件虚拟化扩展在微架构层面提升虚拟机运行效率的实现原理。其次，归纳了 TEE 的基本概念、核心安全特性与主流架构演进范式。针对异构计算场景，分析了信任边界由主机 CPU 向外部硬件加速器延伸过程中存在的硬件隔离与架构兼容性问题。随后，介绍了当前微内核操作系统 EasyAda 及其虚拟化方案 EasyAda-Hypervisor 的架构设计和实现，论证了其基于最小特权原则与统一资源抽象在降低 TCB 及提升系统可扩展性方面的架构优势。最后，阐述了 Rust 语言在保障内存安全与线程安全方面的核心机制，并客观分析了其在底层系统编程中的技术优势与潜在安全隐患。

第三章 基于 RISC-V 的微内核虚拟化方案设计

3.1 设计目标和需求分析

根据前文的介绍，基于 RISC-V 的微内核虚拟化方案——EasyAda-trust 主要面向支持硬件虚拟化的中高端嵌入式场景，设计目标为在提供可信计算能力，在 CPU 侧和 I/O 侧保护用户隐私代码和数据的同时，尽可能降低因为内存隔离等保护机制引入的额外性能开销，以及降低 TEE 的开发难度和准入成本，降低端侧 AI 模型等安全应用供应商的开发成本和难度。

基于以上设计目标，本文认为 EasyAda-trust 应当满足以下需求。

1. 安全性：安全性包括功能安全和数据安全两个方面，从功能上来说，整个虚拟化方案中上层代码中出现的漏洞或不安全行为不应影响更高特权级软硬件的状态，同时单一应用或 VM 的崩溃不应扩散影响到其它同层应用或 VM；从数据保护的角度，每一个软件实体应当遵循最小权限原则，能且仅能访问当前实体所依赖的资源和服务等，即使是更高特权级的软件也不应当拥有无限制访问上层数据和代码的权力。

2. 可扩展性：可扩展性同样分为硬件和软件两个方面。从硬件角度，虚拟化方案应当支持用户根据需求引入和整合定制化的硬件，或者引入更高级的指令集扩展（如向量计算扩展等）；从软件角度，虚拟化方案应当支持灵活的部署方案，并且允许用户开发定制化应用。

3. 硬件兼容性：虚拟化方案应当实现对现有商用硬件的广泛兼容性，并且能够向前兼容更先进的硬件平台。

4. 易用性：虚拟化方案应当兼容主流基础软件以及相关工具链，包括普通 VM 和 CVM，降低开发人员开发难度。

3.2 EasyAda-trust workflow

在进一步讨论 EasyAda-trust 的设计之前，为了便于阐述其威胁模型，本节首先定义 EasyAda-trust 的开发流程和其中涉及的角色。EasyAda-trust 根据嵌入式场景的需求简化了生命周期中包含的角色并将其分为四大类：

- (1) 芯片 IP 供应商：负责芯片 IP 设计。
- (2) 硬件供应商：完成硬件平台设计并根据芯片 IP 和硬件平台设计完成硬件制造，采购以及最终的组装，并对上提供 RoT，硬件密钥对和 SM 开发，编译和配置等。

(3) 平台供应商：负责操作系统、Hypervisor 等基础软件的研发与配置。

(4) 平台用户：基于平台供应商提供的软硬件生态和相关 SDK，根据业务场景开发部署定制化的 VM/CVM 和应用。

以典型的智能驾驶场景为例，ARM 等公司负责设计芯片 IP，高通/联发科等公司基于芯片 IP 完成硬件平台的设计，制造和组装并提供 RoT 和 SM。智驾供应商则负责批量采购硬件平台，并基于硬件平台完成 EasyAda-trust 的基础组件开发，如微内核操作系统，Hypervisor 和 SDK 等，对上层提供统一的开发平台；上层业务部门则可以根据不同汽车厂商的需求选择不同的 EasyAda-trust 配置和实例化方案，并基于 SDK 二次开发 VM/CVM 以及其上的应用（这一阶段中可能还有第三方开发人员开发的侧载应用）。最终作为一个统一的智能驾驶系统对驾驶人员提供服务。在实际场景中，多个角色可能由同一实体或组织承担。

3.3 威胁模型

根据上一节对 EasyAda-trust 全生命周期中的角色的分析，其威胁模型如下所示。硬件层面，不考虑供应链攻击，本文假定方案运行的硬件环境实现符合标准且无缺陷；系统上下文切换时，所有微架构层的运行轨迹均被正确清除。

软件层面，本文假定软件层攻击者会尝试攻陷受保护的 CVM，其攻击方式可以分为（1）篡改 CVM 的控制流或数据流执行逻辑，破坏其运行完整性；（2）窃取 CVM 中受保护的数据；（3）冒充被攻击的 CVM 向其所有者发起交互。

因为嵌入式系统的特性，基础软件通常是可信的，不会主动尝试攻击其他软件实体以及窃取和篡改用户隐私数据，但是可能存在会被恶意应用的代码漏洞；普通 VM 上可能运行不可信的第三方侧载应用，并且普通 VM，微内核操作系统和 Hypervisor 的根权限可能因为代码漏洞被利用或受到网络攻击而被完全控制；同时，本文认为运行在最高特权级的 SM 可信但是同样可能存在代码漏洞的。

同时，攻击者可能尝试运行恶意的 CVM，还可通过 Hypervisor 管控的寄存器、共享内存缓冲区、虚拟输入输出设备，或 CVM 初始镜像状态、扁平设备树（Flattened Device Tree, FDT）等，对被攻击 CVM 的全生命周期及输入数据实现完全控制。

本文的威胁模型排除针对内存、总线及处理器的物理攻击。因为已有成熟的防护方案可应对此类攻击，例如锁步故障检测机制、基于纠错码的内存保护、总线加密等。针对面临物理攻击风险的目标系统，可单独部署上述技术而无需对 EasyAda-trust 的实现进行修改。

3.4 可信微内核虚拟化方案设计

3.4.1 总体设计

如图3-1和3-2所示，如果将整个 EasyAda-trust 视为两层 TEE 的组合：

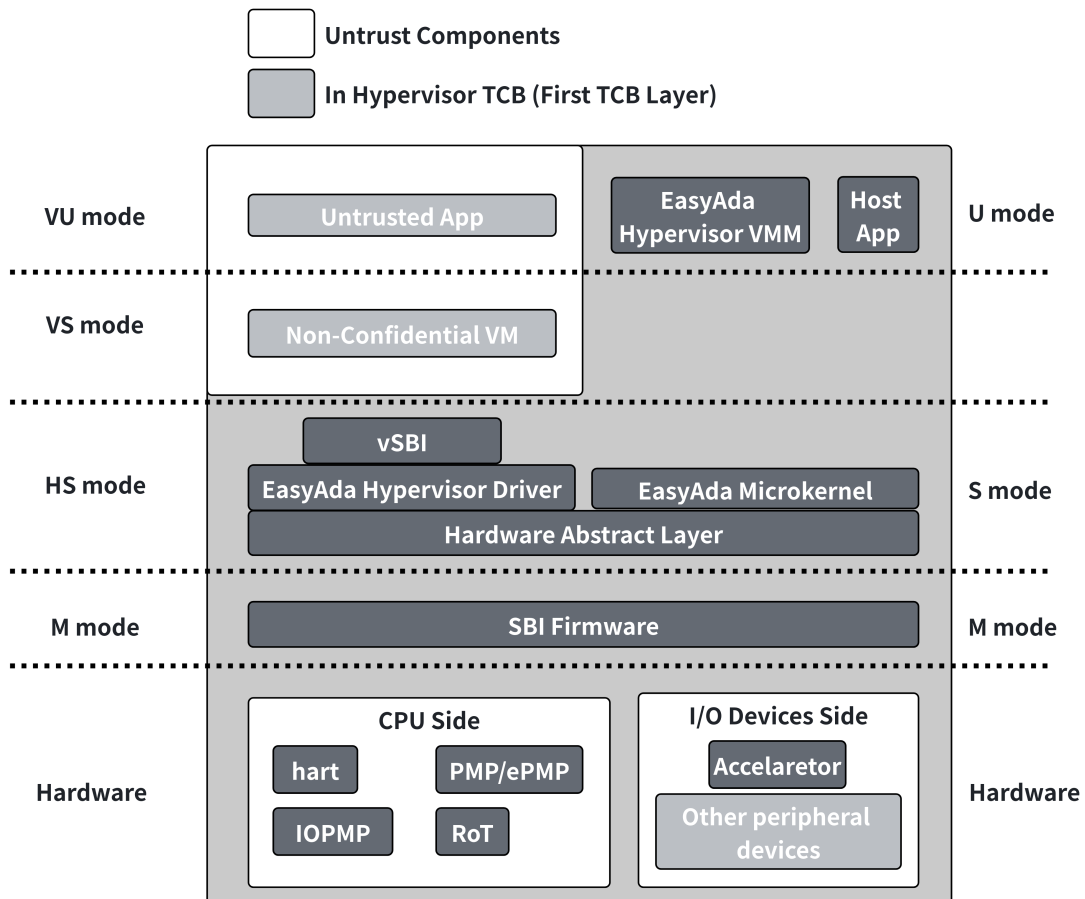


图 3-1 非机密虚拟机可信基

1. 非机密虚拟机 TCB：如图3-1所示，主要包括不可信组件和微内核操作系统 EasyAda 以及微内核虚拟化方案 EasyAda-Hypervisor。不可信组件包括第三方开发者开发的侧载应用和 VM，可能包含恶意代码并尝试攻击 Hypervisor 并窃取和篡改其它 VM 的敏感数据；EasyAda 和 EasyAda-Hypervisor 由 EasyAda-trust 平台供应商提供，通常被视为可信但是其实现中可能存在代码漏洞。不可信 VM 负责为上层应用提供统一的基础软件，降低应用开发成本和难度；EasyAda-Hypervisor 负责管理所有 VM 的资源 and 生命周期，对不可信 VM 提供虚拟 SBI 固件实现，并复用 EasyAda 现有功能以实现统一的资源管理并减少代码量；EasyAda 是整个系统的核心，负责管理和调度所有的硬件资源，并且允许对性能和安全性要求更高的应用直接部署在 EasyAda 上以避免虚拟化对性能和实时性的负面影响。

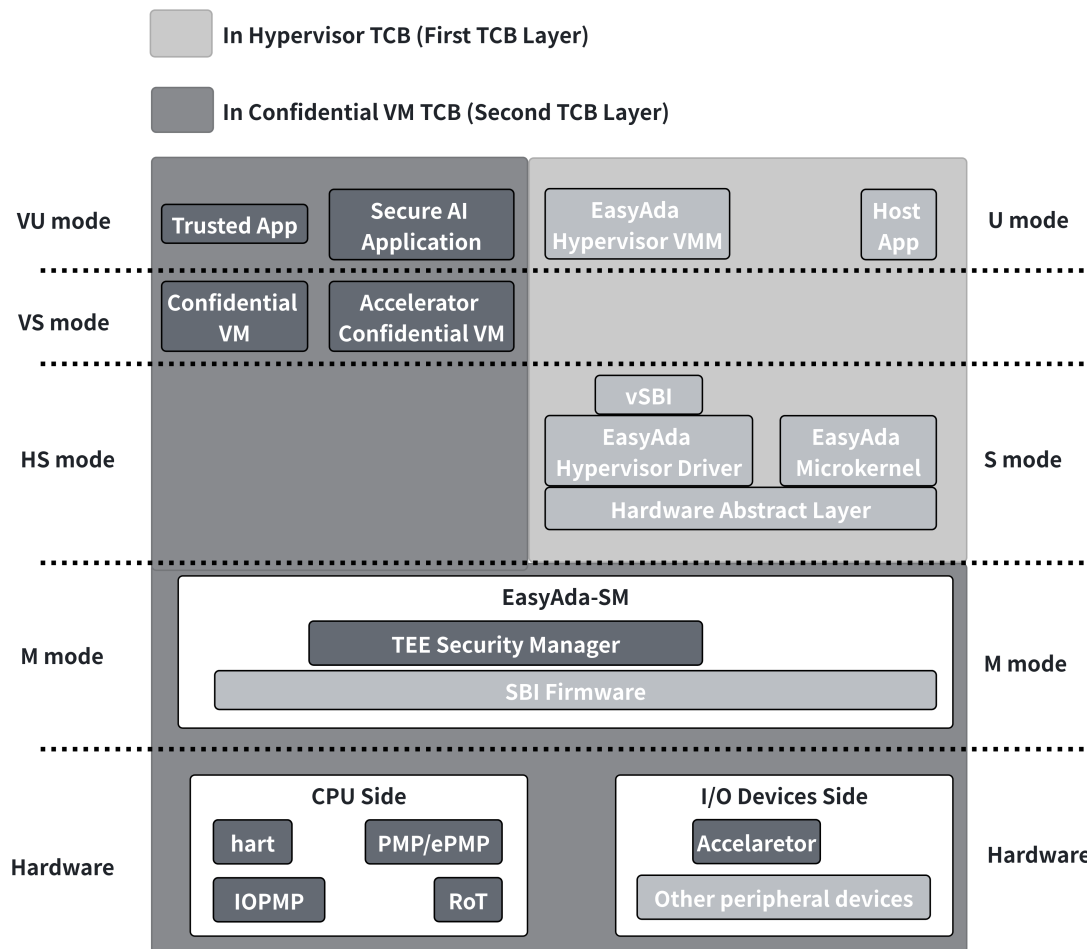


图 3-2 机密虚拟机可信基

2. 机密虚拟机 TCB: 该部分为 EasyAda-trust 新引入的 TEE 架构，用于弥补非机密虚拟机 TCB 在防御不可信 VM 攻击时攻击面过大和安全性不足的缺陷。该部分是可信且不可被攻破的组件，包括运行在最高特权级的安全监视器 EasyAda-SM，实现在 SM 中的 TEE 安全管理器（TEE Security Manager，TSM），运行在相互隔离的运行环境（EasyAda-trust 统一称为 Enclave）中的 CVM 以及其上运行的各类 TA。EasyAda-SM 是实现执行环境间隔离的核心组件，EasyAda-Hypervisor 通过 SM 提供的接口实现对 CVM 生命周期和资源的管理。CVM 对其上的 TA 提供了统一的开发平台，允许开发者透明的使用机密虚拟机保护开发的安全 AI 应用。

EasyAda-trust 为了兼顾的兼容性和安全性，要求硬件环境至少应当实现 H 扩展，PMP 和 IOPMP。边缘用户可以在一个相对轻量化和精简的硬件环境上运行 EasyAda-trust，而需要更高性能或依赖定制化硬件的场景则可以引入增强计算能力的指令扩展和定制化硬件。

EasyAda-SM 并非专为 EasyAda-trust 设计的安全固件方案，其设计目标是在

不同硬件平台上针对不同 TEE 场景提供统一的安全底座。通过模块化和分层设计, EasyAda-SM 支持用户根据使用场景复用已有模块设计实现新的 Enclave 管理模块。本文基于 EasyAda-SM 设计实现了 CVM 管理模块。为了避免不稳定的中间状态增加 EasyAda-SM 攻击面增加, 本文仅在 CVM 管理模块中针对 CVM 定义了三种无中间状态的动作: 转换 (promote), 执行 (run) 和销毁 (destory) 用于 EasyAda-Hypervisor 管理 CVM 生命周期。因为嵌入式场景对功耗和成本的要求, 通常不会实现 IOMMU 用于限制具备 DMA 的恶意设备绕过 CPU 侧的内存隔离访问安全区域, 本文在 CVM 管理模块中引入了 IOPMP 管理模块用于防御 DMA 攻击^[45]。

虽然 EasyAda-trust 将 EasyAda-SM 和 CVM 纳入了 CVM 可信基中, 但是可信并不等于安全。用户在开发过程中可能因为缺乏测试等原因引入破坏 TEE 安全性的代码漏洞, 尤其是不安全的内存访问导致的内存泄漏等问题。针对用户可能引入的代码漏洞, EasyAda-SM 和 CVM 分别采取了不同措施以尽可能减少或隔离代码漏洞, 避免单一漏洞导致整个 TEE 被攻破。本文开发了内存安全的 EasyAda-SM, 并通过分层设计将不安全的内存操作进行封装和限制以降低测试和形式化验证的难度。同时 CVM 管理模块为每个 CVM 启用了硬件虚拟化扩展, 通过二阶段地址翻译隔离不同 CVM 的执行环境, 避免单一 CVM 的安全漏洞影响 TEE 整体稳定性和安全性。具体设计和实现将在后文详细介绍。

3.4.2 内存安全的模块化安全监视器设计

考虑 RISC 精简和灵活的特点, RISC-V 下所有 TEE 方案均依赖运行在最高特权级的安全固件 (通常称为安全监视器, Secure Monitor, SM) 结合指令集拓展对外提供 TEE 相关原语。但是现有 SM 存在以下问题: (1) 不安全的内存管理: 现有 SM 主要基于非内存安全语言 C/C++ 实现, 容易在代码中引入不安全的内存访问, 影响 TEE 整体的安全性; (2) 拓展性不足: 面向不同场景的 TEE 间 SM 设计和实现差别较大, 无法为不同场景提供统一的安全底座。

为了解决以上两个问题, 如图3-3所示, EasyAda-SM 重新设计了运行在最高特权级的 SM, 与 Penglai 和 Keystone 将 Enclave 管理模块作为 SBI 拓展引入 SBI 固件不同, EasyAda-SM 采用了相反的逻辑, 将 SBI 固件作为 SM 的静态链接库并引入 SBI 能力抽象层, SM 可以调用 SBI 固件中的函数获取其提供的能力, 并且可以通过切换使用的 SBI 固件具体实现 (如不同硬件供应商提供的 SBI 固件实现等) 充分利用不同 SBI 固件的差异化能力。

EasyAda-SM 将 TEE 所需的操作, 如内存隔离, 密钥管理, 镜像度量和安全异常处理等抽象为独立的子模块, 每个子模块对外提供统一的操作原语以屏蔽底

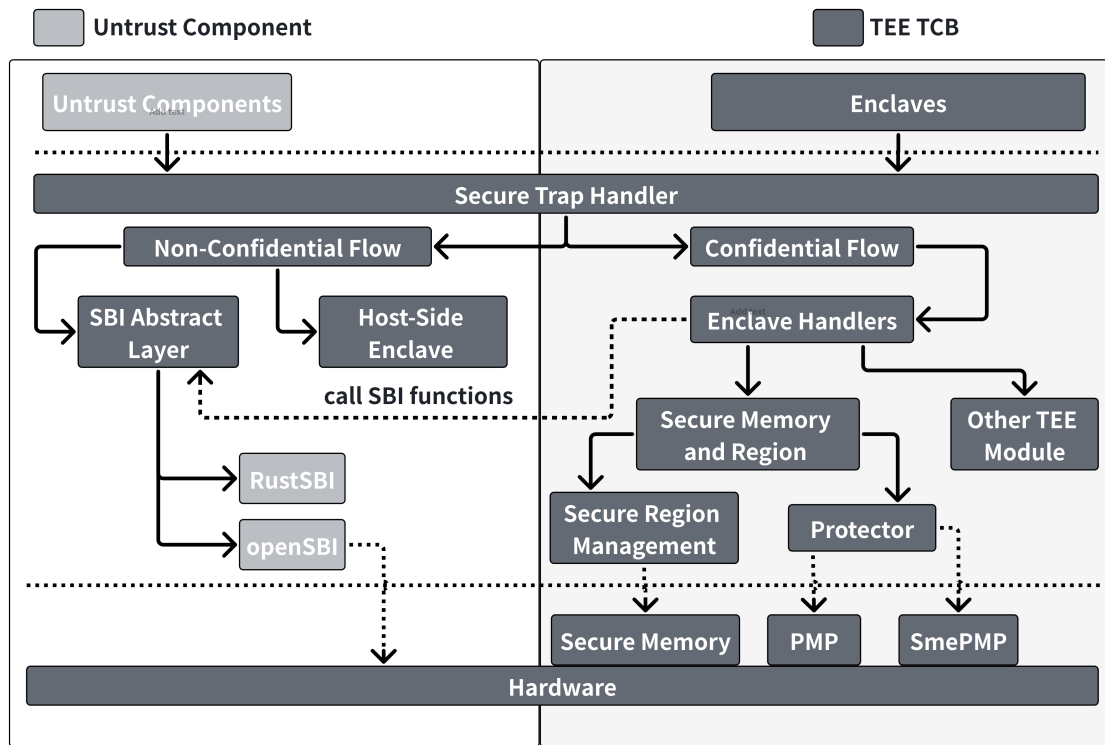


图 3-3 EasyAda-SM 总体架构

层硬件和指令集差异。如图3-3所示，EasyAda-trust 开发者可以针对不同的 TEE 方案开发独立的 Enclave 管理模块，也可以对特定模块进行修改以适配专有指令和硬件。根据最小特权原则（Principle of Least Privilege, PoLP），每个模块仅被授予完成其特定功能所必需的最小权限集合，从而限制潜在安全漏洞的影响范围。

同时，通过对现有 RISC-V 主流 TEE 方案进行深入分析，本文发现不同 TEE 方案对安全内存管理，使用和隔离机制存在较大差异，在安全启动和中断，I/O 隔离等方面的机制和策略则较为共通，如 Penglai 的 SM 被设计为实现更灵活和细粒度的安全内存管理，因此需要 SM 实现安全内存分配能力；Keystone 的 SM 则被设计为一次性内存分配，SM 不进行动态安全内存分配，仅负责进行保护；ACE 中 TSM 为了适应 CVM 二阶段页表的构建，实现了专用的页分配器，充分利用 Rust 的所有权机制消除了重复分配的隐患。进一步的，EasyAda-SM 将内存划分为以下几类并据此设计内存隔离方案。

- (1) 通用区域和通用内存：任何未被 SM 标记和保护的上/外内存都属于通用区域下的通用内存。为了提高系统整体的内存利用率，默认情况下所有内存都是由不可信的 host 负责管理的通用内存。
- (2) 安全区域：被 SM 标记和保护连续内存区域被称为安全区域。安全区域由通用区域划分而来，SM 会为每个安全区域分配唯一的硬件单元用于

实现内存隔离。

- 1) 安全内存：被 SM 标记和保护，但是尚未被 Enclave 使用的安全内存，由 SM 负责分配和管理。
- 2) Enclave 内存：被 SM 标记和保护并且被 Enclave 使用的安全内存。Enclave 内存由 SM 负责配置和保护，但是在 Enclave 释放之前 SM 无权进行释放或再分配。

图3-4展示了 EasyAda-SM 的内存管理模式。EasyAda-SM 将通用内存视为初始状态（未初始化），Enclave 内存视为最终状态（被使用），安全内存作为两者转化的桥梁，处于中间状态（初始化但未使用）。EasyAda-SM 引入安全内存作为 Enclave 内存和通用内存转化的中间层，目的是平衡缩小 SM 攻击面和提高系统整体内存利用率的两种需求。

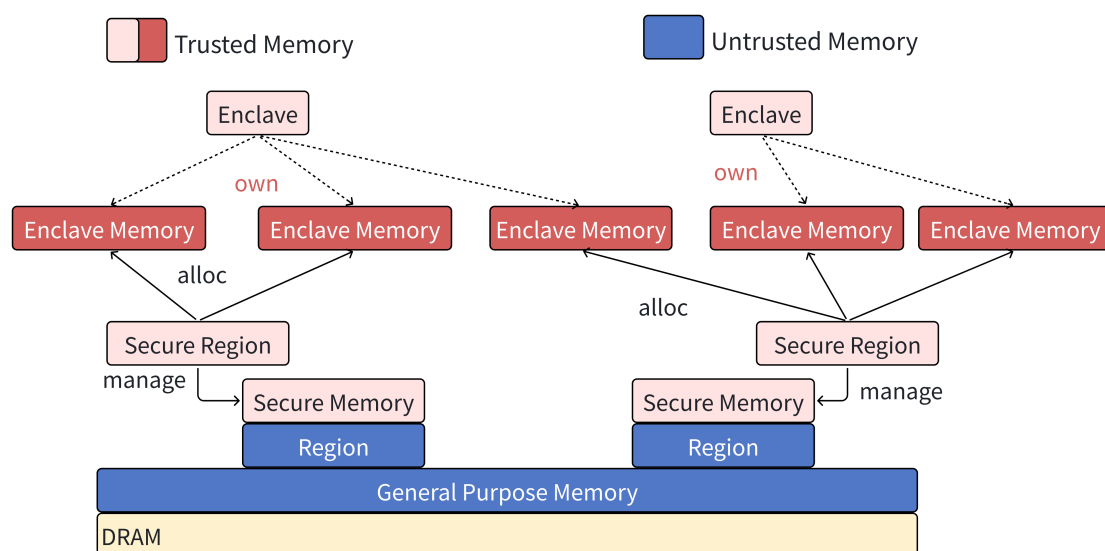


图 3-4 EasyAda-SM 的内存管理模式

Keystone 没有安全内存的中间状态，通用内存直接转化为 Enclave 内存，优势是内存管理简单，攻击面较小，劣势是难以在 Enclave 运行期间动态伸缩内存；ACE 则在系统启动时静态划分安全内存和通用内存区域，SM 负责从安全内存中为可信 VM 分配 Enclave 内存，优势是可以在可信 VM 运行期间动态伸缩内存，劣势是安全区域在系统初始化时静态划分，容易降低系统整体内存利用率；而 RISC-V 的 CoVE 扩展则要求 host 以物理页为单位捐赠内存并由 SM 直接转化为 Enclave 内存，优势是提高了系统整体内存利用率，劣势是 SM 需要实现多个 SBI 接口并且完全依赖 host 侧实现内存管理，显著增加了 SM 的攻击面。

EasyAda-SM 允许 SM 从 host 侧动态获取内存并转化为相互隔离的安全区域或归还安全区域到通用内存，然后由 SM 负责从安全内存中动态分配 Enclave 启动

和运行时需要的 Enclave 内存。在提高系统内存利用率的同时减少对 host 侧内存管理的依赖，缩小 SM 的攻击面，实现两种需求的平衡。

进一步的，EasyAda-SM 引入了统一的安全区域和内存管理层（以下简称安全区域管理层）将安全内存相关操作，如创建/销毁安全区域，申请/释放安全内存，隔离安全内存等封装为统一安全内存管理原语。同时，安全区域管理层将安全内存管理原语依赖的硬件操作划分为内存管理和内存保护两个部分，并且抽象出了统一的内存管理原语和内存保护原语，开发者可以根据硬件环境和业务需求提供具体的原语实现。通过提供统一的内存安全原语，安全区域和内存管理层简化了开发 Enclave 模块的难度；通过抽象统一的内存管理和保护原语，安全区域管理层实现了不同 Enclave 管理模式间基础代码的复用和可扩展性，开发者对上层透明地复用已有原语或根据场景需求和硬件环境实现新的原语。

3.4.3 安全特性设计

接下来将介绍 EasyAda-trust 针对 CVM 需求设计安全的虚拟化方案。

3.4.3.1 安全启动流程设计

安全启动的目标是构建可信链（Trust Chain），确保只有由授权用户开发的，未经篡改的可信组件可以加载到 TEE 中运行，并且最终的安全应用仅能在经过验证的平台上运行并获取敏感 I/O 数据。

安全启动依赖两对密钥对：（1）硬件供应商提供的签名密钥对；（2）每个设备独有的设备密钥对。签名密钥对作为硬件厂商信任背书，签名私钥由硬件供应商离线保管，用于生成 SM 的数字签名，而签名公钥则存储在每个可信设备的信任根（Root of Trust, RoT）中，用于在系统启动时验证 SM 的数字签名。设备密钥对则作为每个设备的唯一标识，设备公钥对用户和开发者开放，用于加密用户验证信息或秘密，而设备私钥存储在设备 RoT 中，用于解密用户加密信息。

RoT 是可信链建立的起点。根据 EasyAda-trust 的威胁模型，EasyAda-trust 将硬件供应商提供的可信硬件（如 TPM）和固件（存储在 eFUSE 和 ROM 等中的代码和数据）视为 RoT。硬件供应商在开发，配置和编译 SM 时，需要生成 SM 的数字签名。在 RoT 启动安全启动流程后，需要生成 SM 的当前度量值并与数字签名进行匹配，由 SM 进行下一阶段的安全启动流程。

平台供应商可以根据业务场景使用特定的 SM 配置，因为 Hypervisor 和不可信 VM 不在 EasyAda-trust 的 TCB 中，因此默认 SM 配置下不对 Hypervisor 进行验证，平台和硬件供应商可以根据需求自行拓展支持对 Hypervisor 和不可信 VM 进行验证。

Hypervisor 需要在系统运行时动态部署 CVM。因为恶意软件攻击者可能尝试拉起恶意的 CVM，同样恶意的硬件环境和 SM 可能绕过 CVM 验证进行强制部署，因此 CVM 部署时必须完成 CVM 与硬件环境的双向验证。为了避免恶意硬件环境强制部署和访问 CVM 中的敏感信息，EasyAda-trust 要求 CVM 开发者自行提供并使用密钥对敏感代码和数据进行加密，确保只在可信的硬件环境进行解密和部署。

针对验证方式，云服务厂商通常采取远程证明（Remote Attestation），通过远程的可信设备验证当前硬件环境和代码是否可信且未经篡改，只有经过验证的环境和 CVM 才能成功部署。部分嵌入式环境可能长期部署在网络访问受限的环境中，因此 EasyAda-trust 默认使用本地证明^[46]（Local Attest）实现 CVM 与硬件环境的双向验证。

3.4.3.2 安全 CPU 虚拟化设计

EasyAda-Hypervisor 能够确保 VM 间 vhart 的隔离，VM 间不会泄漏上下文信息，但是无法实现 EasyAda-Hypervisor 与 VM 间的上下文隔离，因此 EasyAda-trust 对 hart 引入了机密流和非机密流以实现不同执行环境间上下文的隔离。

RISC-V 的硬件虚拟化扩展引入了虚拟化专用特权级以实现 VM 间以及 VM 和 host 间 CPU 上下文的隔离。对于不可信 VM，SM 会将大多数系统级异常委派到 Hypervisor 进行处理以监控 VM 的任何敏感行为。但是对于 CVM，将异常委派到 Hypervisor 进行处理会导致 VM 运行上下文泄露给 Hypervisor，因此需要 TSM 拦截和监控 CVM 的敏感行为。

与普通 VM 一样，CVM 的 hart 分配和 vhart 与 hart 的关联由 Hypervisor 完成。为了避免不可信组件访问和篡改 CVM 上下文，在切换到 CVM 上下文后会配置其原本由 Hypervisor 处理的异常直接陷入 M 态中，通过轻量上下文切换到 TSM 进行处理。若异常需要由 Hypervisor 处理，则由 TSM 进行完整的安全上下文切换，清理所有上下文并切换到 Hypervisor 上下文进行处理。

3.4.3.3 安全内存虚拟化设计

内存虚拟化主要涉及为 VM 分配运行内存，并建立和维护 VM 运行内存和虚拟设备地址的第二阶段地址映射表。前者用于容纳 VM 运行时的代码和数据，后者用于实现设备直通和 VM 间通信等。

不可信组件与 CVM 间内存隔离：为了实现 CVM 与不可信组件间的内存隔离，CVM 无法直接使用 Hypervisor 分配的不可信内存作为运行时内存，因此 EasyAda-trust 引入了安全内存和安全区域的概念。安全内存指由 SM 配置，使用硬件实现强制内存隔离，不可信组件无法直接访问的内存区域，任何用于存储 CVM 敏感数

数据和代码的内存必须使用安全内存以确保其机密性和完整性。EasyAda-trust 以安全区域为单位管理安全内存，每个安全区域标记了一片物理连续的内存区域，由绑定的硬件和 SM 进行保护，安全区域内的内存分配由 SM 负责。安全内存由不可信内存转化而来，Hypervisor 可以申请一块连续的不可信内存捐献给 SM，由 SM 进行保护，并将其标记为安全内存，之后任何不可信组件都无法直接访问这片内存区域。

CVM 间内存隔离：虽然 CVM 被视为 EasyAda-trust 软件 TCB 的一部分，但是 CVM 中的代码漏洞可能破坏 TEE 提供的安全保护，并且控制了 Hypervisor 的恶意攻击者可能运行恶意的 CVM 以尝试攻击受害 CVM，因此 CVM 间同样需要实现内存隔离。ACE^[33] 中所有 CVM 共享一个安全区域，通过硬件虚拟化提供的二阶段地址翻译实现 CVM 间内存隔离。虽然这提供了足够的安全防护，但是难以根据需求对安全内存进行动态收缩和扩容。EasyAda-trust 支持增加或回收安全区域，多个 CVM 可以共享一个安全区域以提高内存利用率。CVM 间的内存隔离则通过二级段地址翻译实现隔离。

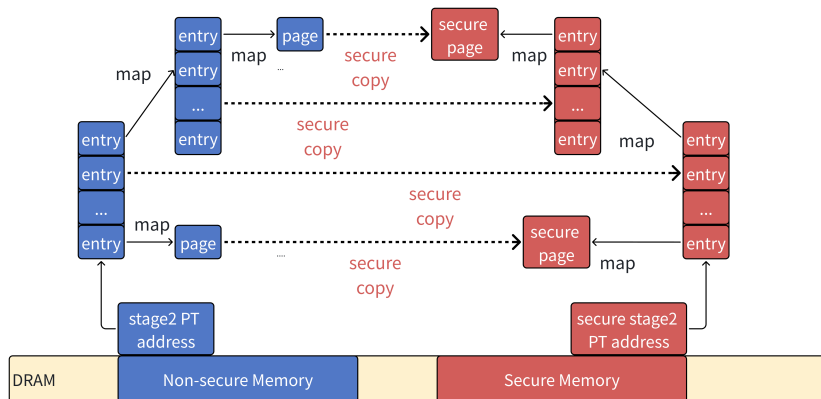


图 3-5 CVM 二阶段页表初始化

CVM 内存虚拟化：CVM 在创建 VM 实例阶段与不可信 VM 类似，由 Hypervisor 完成内存申请和二阶段页表初始化等。当用户需要将 VM 转换为 CVM 时，Hypervisor 驱动通过 TSM 对外提供的 promote 接口，将二阶段页表和 FDT 的 IPA 作为参数交给 TSM。如图3-5TSM 首先会遍历由 Hypervisor 为当前 VM 维护的二阶段页表，申请安全内存用于构建完全一致的页表作为 CVM 的初始二阶段页表，同时按序申请安全内存用以拷贝不可信内存中的数据，避免后续 Hypervisor 篡改初始状态。

然后 TSM 将按序遍历 CVM 的二阶段页表并度量 FDT，kernel 和 initrd 的初始度量值（具体 CVM 度量流程设计将在后续安全启动小节详细说明），并与 CVM 开发者提供的初始度量值进行对比，若度量值校验通过则说明当前 VM 符合预期，

由可信的开发者开发并且没有被 Hypervisor 篡改，允许被拉起并访问开发者提供的用于后续加解密 I/O 数据的密钥。

安全 MMIO 区域隔离：RISC-V 采用统一编址模式，设备的 MMIO 区域以内存区域的方式进行管理。因此 EasyAda-trust 基于安全区域与不可信组件的隔离实现了设备 MMIO 区域与不可信组件的隔离，并且通过为安全区域引入额外的标识符，标识当前安全区域类型是否为 MMIO 区域，并且可以进行进一步细分实现具体设备类型的标识。

不可信共享内存：除了存储数据和代码的安全内存，CVM 与不可信组件间需要额外的共享内存以实现高效的数据交换，尤其针对磁盘等块设备的 I/O 虚拟化场景。EasyAda-trust 支持 CVM 向不可信 host 请求共享内存。

不可信共享内存由 host 分配和管理，主要用于 VM 间，VM 与 host 间，以及不可信组件与 CVM 间的数据交换。host 完成内存分配后，需要由 Hypervisor 映射到 VM 地址空间内，或由 TSM 映射到 CVM 地址空间内，确保组件可以在当前地址空间下直接访问共享内存。因为不可信共享内存由 host 管理，因此 CVM 在读写这类内存时，需要进行严格的安全检查并主动对敏感信息进行加密避免泄露。

EasyAda-trust 并不支持在 CVM 间分配可信共享内存。因为（1）根据角色划分，VM 和 CVM 均由同一角色——EasyAda-trust 用户负责开发，应用功能在开发时已经相对确定，不需要也不应该将需要大量数据交换的多个功能拆分到多个 CVM 中实现（2）CVM 和 TSM 作为 TCB 应当尽可能保证精简，在 CVM 间支持可信共享内存会进一步增加 TCB 的大小和复杂度（3）少量的数据交换可以通过不可信共享内存和加密信道实现 CVM 间的数据交换。综上，EasyAda-trust 的设计并未考虑使用可信共享内存实现 CVM 间通信。

3.4.3.4 安全中断虚拟化设计

中断安全分为两个部分：中断发送的安全和中断接收的安全。两者都要求目标的中断不应被不可信组件获取和篡改。

对于发送中断，RISC-V 要求所有中断必须在 M 态下发送。对于不可信 VM，发送中断首先会陷入到 Hypervisor 驱动中，由 Hypervisor 驱动决定实际的接收 hart，再通过 SBI 调用陷入 SBI 固件进行中断发送。在这个流程中 Hypervisor 对 VM 的中断有绝对的控制权。但是在 CVM 场景下，不可信组件无权控制 CVM 发送的中断，需要直接陷入到 TSM 中发送，并且需要 TSM 为 CVM 维护接收方到物理 hart 的映射关系。

对于接收中断，SM 会将除外部中断外的所有中断全部委派到 HS 态由 Hypervisor 进行处理，因为 EasyAda-Hypervisor 采用静态分区策略，Hypervisor 可以进一

步将时钟中断直接委派到 VS 态由 VM 直接处理，避免频繁陷入 Hypervisor 带来的性能开销。其他中断如软件中断等需要陷入到 Hypervisor 中进行处理，由 Hypervisor 决定是否注入中断或切换到 vhart 服务线程上下文运行，无需对 CPU 当前上下文进行清理，相关中断配置由 Hypervisor 负责。

但是对于 CVM，除时钟中断外的其他中断必须绕过 Hypervisor 直接陷入到 TSM 中进行处理并决定是否注入中断。如果中断涉及进一步的上下文切换，需要由 TSM 负责清理 CVM 上下文避免泄露任何敏感数据。中断相关配置均由 TSM 负责。

除了中断发送/接收安全外，还需要额外注意拒绝服务攻击（Deny of Service, DoS），包括 CVM 对 host 侧的 DoS 攻击，以及 host 对 CVM 侧的 DoS 攻击。前者由于 host 具有最高调度权，可通过 TSM 强制销毁恶意 CVM 来解决；而针对后者（host 攻击 CVM），则需要进一步区分攻击模式：一种攻击模式是 host 生成大量伪造的中断请求，迫使 CVM 频繁陷入 TSM，导致 CVM 无法及时响应其他正常请求，这种攻击可以通过 TSM 对来自 host 侧的中断进行监控和计数，进而对中断请求进行管控和屏蔽实现防御；另一种攻击模式则是 host 拒绝响应来自 CVM 的任何请求，因为 host 被设计为负责管理系统资源，所以不针对这种攻击进行防御。

3.4.3.5 安全 I/O 虚拟化设计

EasyAda-Hypervisor 原有的安全 I/O 虚拟化方案能够实现：（1）VM 间数据流和控制流的隔离（2）确保同一时刻一个设备仅能划分给一个软件实体（VM 或 Hypervisor）直接访问。

为了确保 CVM 使用的外部设备的安全性，EasyAda-trust 需要额外实现以下安全能力：

- （1）平台和硬件的可信性验证：根据当前设备的安全级别，在 CVM 使用安全设备之前对设备进行可信验证，防止 CVM 被恶意外设攻击导致敏感数据的泄漏和篡改。
- （2）CVM 生命周期中 I/O 数据的安全性
 - 1) CPU 侧数据安全性：保护 CVM 访问外设过程中输入输出数据的机密性和完整性，包括通过设备独占，数据加密和校验等方案实现。
 - 2) 设备侧数据安全性：具备 DMA 能力的设备可以绕过 CPU 侧的内存隔离机制直接访问物理内存，恶意设备可以利用这一点直接读写安全区域，因此需要根据外设安全级别限制外设可访问的内存区域。

针对以上安全需求，EasyAda-trust 根据外设的使用特点将设备划分为两类：

- （1）普通 I/O 设备：这类 I/O 设备包括串口，网卡，磁盘等设备，这些设备的

特点是设备对数据内容不敏感，无需或仅需少量计算。

- (2) 安全设备：这类设备重视安全性，对输入数据内容敏感或需要进行大量计算，如指纹采集器，摄像头和硬件加速器等。

EasyAda-trust 根据设备类型采取了不同的可信性验证方案。对于由 host 管理的普通 I/O 设备，EasyAda-trust 不对其进行可信验证。而对于安全设备，EasyAda-trust 采用了 host 分配，TSM 鉴权和管理逻辑，并在安全启动流程中引入了可信设备注册，使用和回收的可信链。以系统开发时硬件供应商提供的可信硬件配置文件作为可信链起点，在 host 未将可信设备提升为安全设备时，仍然可以正常控制该设备，同时 host 可以在系统运行过程中向 TSM 请求提升设备为安全设备，由 TSM 根据硬件配置文件对设备进行可信验证，只有通过验证的外设会被视为可信设备并提升为安全设备。因为 CVM 在启动时会对硬件环境进行验证，未通过验证的硬件环境无法对 CVM 核心的机密代码和数据进行加解密，因此由 TSM 负责对外设进行可信验证是符合可信链建立流程的。

完成设备的可信验证后，EasyAda-trust 需要进一步保护机密 I/O 数据，对于 CPU 侧的保护，有三种思路可以用于保护 CVM 的输入输出数据：

1. 基于纯加密的数据保护：如图3-6(a)所示，因为系统设备主要用 host 负责管理，最简单的方案是 CVM 使用开发者提供的密钥对所有 I/O 数据进行加解密和度量，并通过不可信的共享内存和 VirtIO 协议栈实现与不可信组件的数据交换。最终由 host 侧的 vhart 服务线程调用实际外设驱动完成对外设的读写。

这种保护模式适用于纯粹的 I/O 请求保护，如网卡设备，串口，磁盘块设备等。由 CVM 作为数据发送者对数据进行度量和加密，用于加解密的密钥仅在安全内存中才会以明文形式存储和使用。而数据使用者通过配对的密钥进行解密和度量，阻止信道上任何潜在的攻击者进行窃听，可以有效保障数据的机密性。但是一方面，因为需要由不可信组件读写数据，需要额外的重传机制保障数据的完整性；另一方面，多数计算任务要求数据以明文形式提供，对数据进行加解密会引入额外的性能开销。

2. 基于临时设备授权的数据保护：如图3-6(b)所示，如果将设备管理和授权剥离，将设备授权提升到 SM 进行管理，由 SM 在不可信组件和 CVM 间实现设备的分时授权，确保同一时刻一个设备仅能授权给一个 CVM 或不可信组件使用，当一个实体获得授权之后，其他实体无权对设备进行任何读写和配置操作。

这种保护适用于全局唯一设备的 I/O 请求保护。在软件实体获得授权时可以获得设备的完全访问和配置权限，当 CVM 获取授权时，可以直接使用明文数据读写设备而不用担心不可信组件和其他 CVM 劫持设备或窃取和篡改数据；同样的，当不可信组件获取设备授权时，CVM 也无法劫持或访问设备；当解除权限时，由

SM 或软件实体自身清理设备状态，避免泄露敏感信息。这种模式可以在保护数据机密性和完整性的同时，避免加解密带来的开销。但是这种保护方式需要额外引入复杂的时分复用逻辑，尤其对于在内核态下直接进行管理的设备，需要对内核进行修改以确保 SM 可以正确监控系统对外设的使用，避免因调整设备授权导致当前实体对外设的访问异常中断。

3. 基于硬件分区的数据保护：如图3-6(c)所示，若当前硬件环境中存在冗余或专为安全应用设计的设备，参考静态分区的思想，可以对设备进行分区，将部分设备固定划分给可信组件使用的同时，避免影响不可信组件对外设的访问。如 ARM 下安全世界拥有独立的 UART，GPIO，flash 和 RAM 等设备专供安全世界使用。

这种保护模式适用于专用安全设备或冗余外设的 I/O 请求保护，如指纹/声纹输入设备，冗余串口/flash 等。因为 RISC-V 采用统一内存编制，硬件分区可以基于内存隔离机制实现，CVM 可以以明文形式直接读写 TEE 外设，无需额外的加解密工作；同时 CVM 和 host 侧因为访问独立的外设，I/O 流程完全隔离，对外设的访问互不干扰，在保证数据完整性和机密性的同时具备更高的效率，且无需在 host 和 CVM 间实现复杂的时分复用逻辑。但是一方面该方法对硬件环境有较高要求，在硬件资源受限的环境中难以使用；另一方面可扩展性不足，如果存在多个 CVM 实例，CVM 间仍然需要针对设备实现时分复用逻辑。

对于普通 I/O 设备，EasyAda-trust 允许 CVM 不保护 I/O 数据（如串口输出）或参考3-6(a)方法保护 I/O 数据，设备直接授权给 host 使用，CVM 和不可信 VM 需要通过不可信的共享内存与 host 通信，由 host 完成对外设的实际操作。简化设备的管理逻辑，同时确保设备可以扩展给更多组件使用。

而对于安全设备，EasyAda-trust 参考3-6(c)方法，由 CVM 直接独占设备进行管理，SM 负责实现设备的数据缓冲区和 MMIO 区域与不可信组件间的隔离。CVM 仍然可以通过共享内存的方式与其他组件间进行交换数据。为了避免复杂的运行时设备发现，EasyAda-trust 要求开发者在开发阶段对完成可信设备的标记并将可信的硬件配置存储在硬件 RoT 中或直接固化在 TSM 中，用于对设备身份进行验证。

对于设备侧的 I/O 数据保护，EasyAda-trust 基于 RISC-V 的 IOPMP 扩展实现。为了避免 IOPMP 条目的快速耗尽和频繁的 IOPMP 切换，EasyAda-trust 对安全设备仅开发所有由 host 分配的安全区域的访问权限，而非安全设备仅能访问不可信的内存区域，具体 IOPMP 管理方案实现将在安全监视器章节详细介绍。

同时，必须考虑到部分端侧硬件环境出于功耗和成本考虑，仅实现了 RISC-V 硬件虚拟化扩展而未实现 IOMMU，并且外部设备本身不具备地址管理单元（Memory Management Unit, MMU）。因此 EasyAda-trust 在部分硬件环境中无法使用 IOMMU

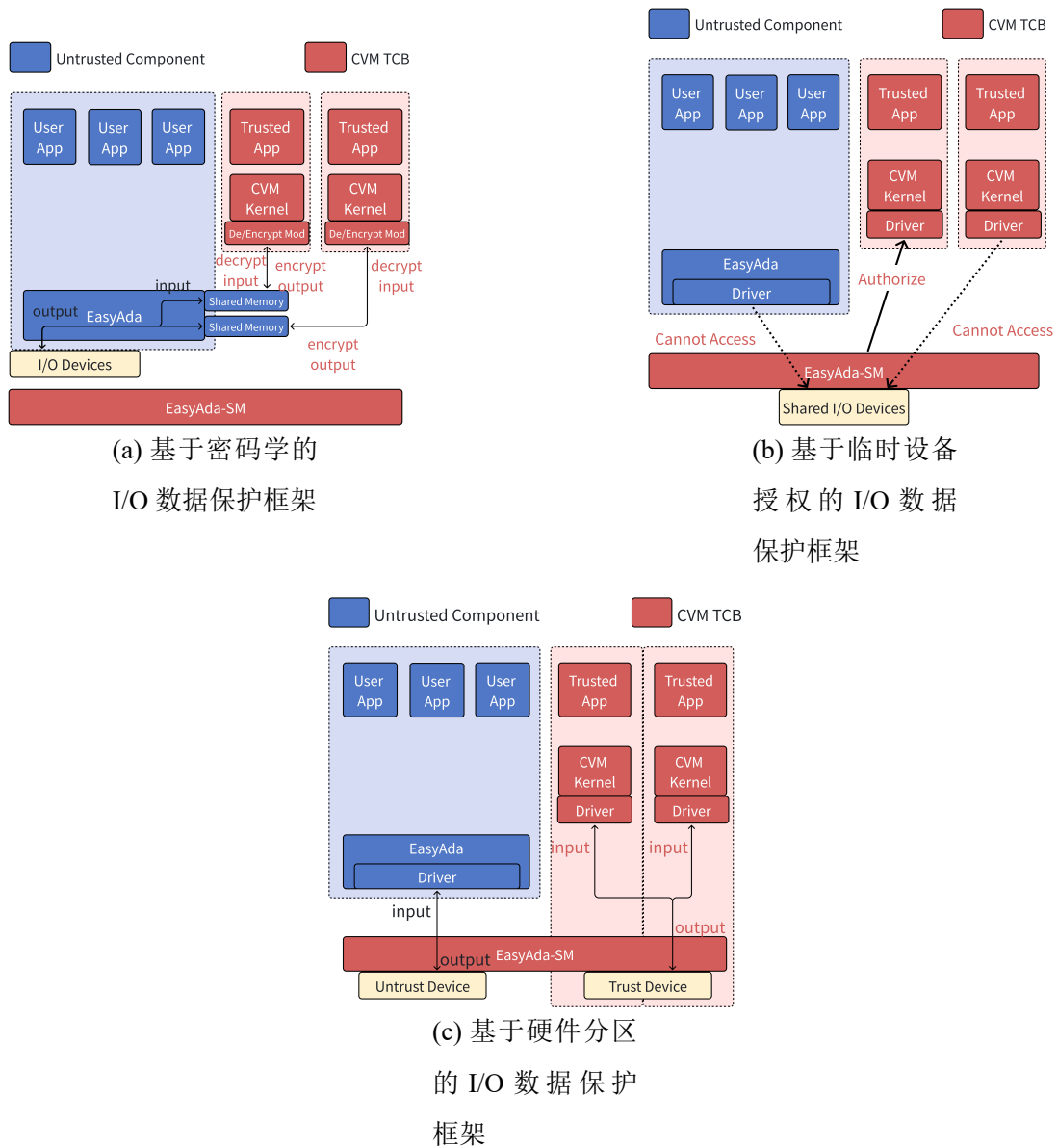


图 3-6 CVM 的 I/O 数据保护机制

实现 I/O 虚拟地址 (Input/Output Virtual Address)，使用 IPA 直接配置 DMA 会导致外设的内存访问触发未定义的行为，必须要通过软件实现 IPA 到 HPA 的地址翻译。一种方案是在此类环境上整体回退到全虚拟化设备，CVM 仍然具备完整的设备驱动，但是任何对 MMIO 设备的访问都需要陷入到 TSM 中完成地址翻译和硬件操作，对于性能敏感场景会引入无法接受的时延和性能开销。因此 EasyAda-trust 在保留了全虚拟化支持的基础上，允许 CVM 显式向 TSM 发起请求批量实现 IPA 到 HPA 的翻译，仅需在配置 DMA 地址时触发一次下限，并且可以通过合并 I/O 请求进一步降低地址翻译的开销。

为了实现 CVM 使用硬件加速器计算时数据的机密性和安全性，下面将详细介

绍基于 EasyAda-trust 的安全异构计算环境的设计。

3.4.3.6 安全异构计算环境设计

与其他 I/O 设备相比，硬件加速器丰富的软硬件计算和存储资源增加了 I/O 数据保护的难度；同时，因为硬件加速器承担了大量的计算任务，对性能非常敏感；而且与其他设备类似，需要考虑跨执行环境的设备复用。前文已经介绍了通过数据加密和独占设备的 I/O 数据保护，但是这些措施在实现 CVM 数据保护时存在一定的局限性：

- (1) 纯密码学方案的吞吐开销：对于资源稀缺的嵌入式场景，安全性并非硬件加速器的第一考量，因此通常并不具备硬件加解密能力，需要通过软件方案加解密，如果采用 3-7 (a) 的方法进行保护，对于 AI 训练中的大规模梯度同步，密码学操作会迅速占满显存控制器的带宽或 CPU 的计算周期，导致严重的计算延迟并显著降低带宽利用率。Lee^[47] 等人的研究显示这种开销导致 DDP 训练迭代时间平均增加了 8 倍，极端情况甚至达到 41.6 倍。同时，密码学基于块的计算方式与 GPU 非连续的内存访问间粒度不一致，会进一步放大加解密的负面影响。
- (2) 独占设备难以满足复杂场景：复杂嵌入式场景下通常需要部署多个模型以应对不同的使用场景，如车载场景下座舱 AI 和自动驾驶模型通常位于不同的执行环境中，均需使用硬件加速器进行计算。因此 EasyAda-trust 被设计为支持在不可信组件和 CVM 间共享的硬件加速器。

根据硬件加速器与 CPU 的耦合程度，可以将其划分为以下三类：(1) 指令集扩展，将 AI 运算逻辑直接嵌入 CPU 流水线中，集成度最高；(2) 协处理器，与 CPU 共享物理内存并通过 MMIO 进行控制，集成度次之；(3) 片外加速器，拥有独立显存，自由度最高，通常在服务器等高性能场景下使用。端侧场景下为了控制功耗和成本，主流硬件平台中硬件加速器通常以协处理器的模式集成在 SoC 中，以统一内存架构 (Unified Memory Architecture, UMA) 的方式进行管理。因此除了隔离硬件加速器的 MMIO 区域外，还需要额外隔离从内存中动态分配的数据缓冲区。

如图3-2所示，EasyAda-trust 为了降低用户安全管理硬件专用加速器的难度，实现为不同的硬件加速器提供统一的保护方案，引入了专用加速器管理 CVM (Accelerator CVM)。得益于 CVM 的引入，开发者可以实现对现有硬件加速器驱动的复用而无需进行代价高昂的移植。

对于 CPU 侧，与其它安全设备类似，有 TSM 通过安全硬件实现硬件加速器 MMIO 区域与不可信组件的隔离。而对于硬件加速器使用的动态数据缓冲区，因

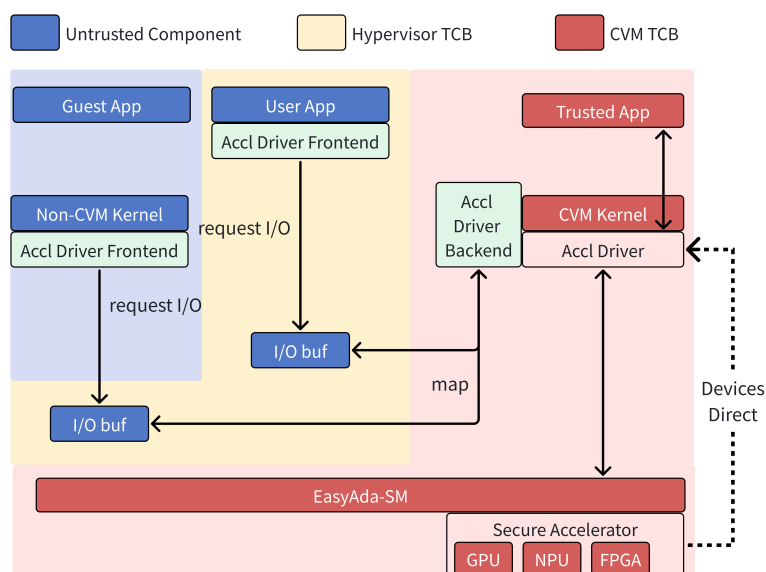


图 3-7 硬件加速器管理 CVM 设计

为数据缓冲区由加速器驱动从 CVM 的安全内存中进行分配，同样可以复用安全区域提供的内存隔离保障。对于外设侧，TSM 会通过 IOPMP 授予且仅授予硬件加速器访问管理 CVM 的使用的安全区域的访问权限，由 TSM 确保不会因为错误配置 DMA 导致硬件加速器 I/O 地址异常。得益于 I/O 数据保护，CVM 可以直接以明文数据形式与硬件加速器进行交互，避免了高昂的加解密开销。

为了实现硬件加速器在不同执行环境间的复用，加速器管理 CVM 可以作为半虚拟化设备后端，为其它不可信组件提供虚拟硬件加速器设备。不可信组件和加速器管理 CVM 间的虚拟设备发现同样在开发时通过配置硬件配置文件静态实现。因为 EasyAda-trust 的首要目标是确保 CVM 内机密应用和数据的安全性，对于不可信组件则不进行任何保障，因此不可信组件和加速器管理 CVM 间使用不可信的共享内存进行数据交换，并且不强制要求任何数据加密和校验。

3.5 本章小结

本章系统论述了基于 RISC-V 架构的微内核虚拟化方案 EasyAda-trust 的架构设计与核心机制。该方案以 EasyAda 和 EasyAda-Hypervisor 微内核虚拟化方案为底层支撑，通过引入 Rust 编写的安全监视器 EasyAda-SM 及内存原语抽象，有效缩减了系统的 TCB。在安全机制方面，建立了涵盖机密虚拟机全生命周期的防护体系，利用本地证明、机密流隔离及二阶段页表等技术，保障了 CPU 状态与内存空间的强隔离性，并构建了抗攻击的安全中断处理路径。此外，针对端侧异构计算需求，方案结合 IOPMP 技术设计了专用加速器管理机制，在提升设备复用效率

的同时确保了数据的机密性。

第四章 内存安全的模块化安全监视器实现

4.1 内存安全固件

为了消除 SM 中不安全的内存访问，EasyAda-SM 使用适合嵌入式场景的内存安全语言 Rust 实现，通过静态的编译时内存检查消除代码中可能存在的不安全内存访问，避免了昂贵的运行时内存检查和回收。EasyAda-SM 基于 RustSBI^[48] 而非基于内存不安全语言 C 的 SBI 固件（如 OpenSBI），进一步消除了 SBI 固件中不安全的内存访问。

4.2 安全区域和内存管理层实现

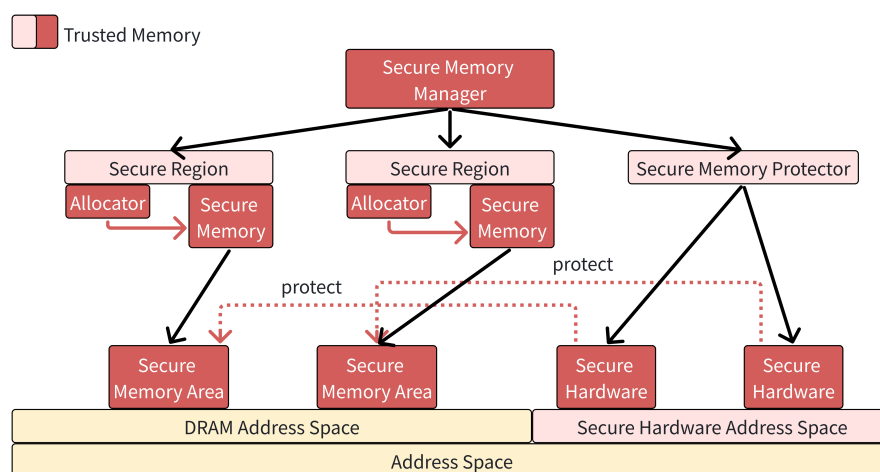


图 4-1 安全区域和内存管理层实现

安全区域和内存管理层的目标是实现不可信组件与安全内存间隔离的同时提高系统整体的内存利用率。如图4-1所示，安全区域和内存管理层以安全内存管理器作为安全内存的顶级管理单元，对外提供安全内存管理接口，每个安全内存管理器包含多个安全区域（Secure Region）和一个安全内存保护器。

每个安全区域关联一块连续内存区域和一个内存分配器（allocator），内存分配器对应一套内存管理算法实现。严格来说安全区域管理的内存存在被强制隔离之前并不属于安全内存，因为不同 TEE 方案对于安全内存的保护时机存在较大差异，因此 EasyAda-SM 将安全内存保护器和 allocator 分离，由安全内存管理器自行决定内存的保护时机，为了方便叙述，后文以安全内存代指安全区域管理的内存区域。

安全内存保护器则负责管理用于隔离安全内存的安全硬件如 PMP，并对外提

供内存保护接口。安全内存保护器的保护粒度是安全区域，主要目标是隔离不可信组件和安全内存，并辅助实现 Enclave 间内存隔离。

为了在同一 Enclave 管理模式提供统一的安全内存视图，每个 Enclave 管理模块通常使用一个全局的安全内存管理器实现统一的安全内存管理，并使用每个安全区域独立的 allocator 实现精细化的内存管理。

根据上一章中介绍 EasyAda-SM 的分层设计思想，安全内存管理器抽象了安全内存管理原语，allocator 抽象了内存管理原语，而安全内存保护器则抽象了内存保护原语，三者共同实现对安全内存生命周期的管理。下面将按序说明每个原语的具体实现。

4.2.1 内存管理原语

表 4-1 EasyAda-SM 内存管理原语

原语	功能
new	创建一个空的 allocator（此时无安全内存需要管理）
init	使用指定连续内存区域初始化一个 allocator
alloc	从当前安全内存中申请内存
free	释放安全内存到当前安全区域中
available	获取当前安全内存剩余可分配内存
total	获取当前安全内存总大小

如表4-1所示，内存管理原语抽象了安全内存管理所需的操作。包括初始化安全内存，申请/释放安全内存等。为了简化实现并避免重复安全检查引入的系统性能开销，内存管理原语不进行任何安全检查，如不在 free 原语中检查释放内存是否属于当前安全区域以避免破坏内存管理元数据等。

本文分析了不同 Enclave 抽象的安全内存使用特点：

- （1）基于应用的 Enclave 抽象（App Enclave）：此类轻量化 Enclave 通常直接运行在用户态，无权访问和控制任何内核态资源，如 MMU 和中断配置等。因此 TA 的虚拟地址空间和物理地址空间在 Enclave 运行时不会进行任何修改。
- （2）基于 OS 的 Enclave 抽象（Rt Enclave）：此类 Enclave 更加复杂，拥有完整的用户态和内核态资源，TA 的虚拟地址空间在运行时可能修改；但是因为直接运行在与 host 相同的特权级，为了保证高效的内存隔离，Enclave 的物理地址空间在运行时不会进行修改。
- （3）基于 VM 的 Enclave 抽象（VM Enclave）：因为开启了硬件虚拟化，此类

Enclave 兼具前两种 Enclave 的特点：在虚拟化特权级下拥有完整的用户态和内核态资源，又可以在运行时无感可控地调整 Enclave 物理地址空间并且不破坏当前内存隔离。

EasyAda-SM 针对不同类型 Enclave 使用安全内存的特点，提供了三种参考内存管理原语实现，下面将介绍其具体实现。

4.2.1.1 基于空分配算法的原语实现

该实现为安全区域的内存管理器的空实现，仅用于安全区域初始化，因为部分关键安全区域（如 SM 自身代码和数据占用内存区域）仅需要进行隔离，不允许进行任何分配，需要使用空分配算法作为分配器占位符。在空分配算法上尝试调用任何初始化或分配操作都会导致异常。

4.2.1.2 基于伙伴分配算法的原语实现

Buddy 分配器基于对 RustSBI 现有系统堆管理算法的复用，使用 buddy 算法管理安全内存，支持多个 Enclave 共享相同的安全区域。该实现针对 App Enclave，以 2 的幂次为粒度分配和管理安全内存，足以满足 App Enclave 的内存需求，可以较好的兼顾分配效率和内存利用率，同时内存管理元数据直接存储在安全内存中，对 SM 本身的内存依赖较少。劣势是为了简化管理逻辑需要预留一部分安全内存用于存储内存管理元数据，在大片内存分配场景下容易产生外部碎片。

该算法可以确保内存分配行为的安全，即同一内存块同一时刻能且仅能属于一个 Enclave 内存区域。但是对内存块间的隔离不提供任何保障，依赖 Enclave 管理模块配置 MMU 实现隔离，因此不可用于 Rt Enclave 的内存管理。

4.2.1.3 基于一次性分配器的原语实现

一次性分配器将当前安全区域内所有安全内存作为一个内存块进行分配和释放，同一安全区域仅供一个 Enclave 使用。该实现专门针对 Rt Enclave 和安全设备的内存管理。因为 Rt Enclave 在内存管理上具备两个特点：（1）可以在 Enclave 内部进行内存分配，利用预留的空闲内存区域（2）可以配置 MMU，因此不能通过 MMU 实现 Enclave 间内存隔离。而一个安全设备的内存区域应当完全由一个实体进行管理，不应进行拆分。一次性分配器仅对 Enclave 内存需求和类型进行简单判断，如果类型匹配且安全内存满足 Enclave 需求，则直接完整分配所有安全内存为 Enclave 内存，并返回内存信息，由 Enclave 在内部进行二次内存管理；同时复用安全内存保护器提供的安全区域间隔离实现 Enclave 内存间的隔离（因为此时 Enclave 独占安全区域）。

该算法可以确保任意类型 Enclave 间的内存隔离，并且分配逻辑简单，开销较低。但是因为没有对内存进行任何拆分，对于不具备内存管理能力的 Enclave 容易引入内部碎片降低内存利用率；同时，安全内存保护器可以使用的安全硬件数量较为有限（如 RISC-V PMP 条目数量多为 16，最多不超过 64 个），该方法会快速耗尽可用的安全硬件，因此不应大量使用。

4.2.1.4 基于页分配器的原语实现

页分配器（Page Allocator）是专为虚拟机飞地（VM Enclave）设计的内存管理组件。该分配器充分利用 RISC-V 架构多级页表支持特定映射粒度（如 4KB、2MB 和 1GB）的特性，并深度结合 Rust 语言的所有权机制。与依赖不安全内存操作的传统管理方案相比，该设计从根源上消除了内存管理元数据遭破坏的安全隐患。如

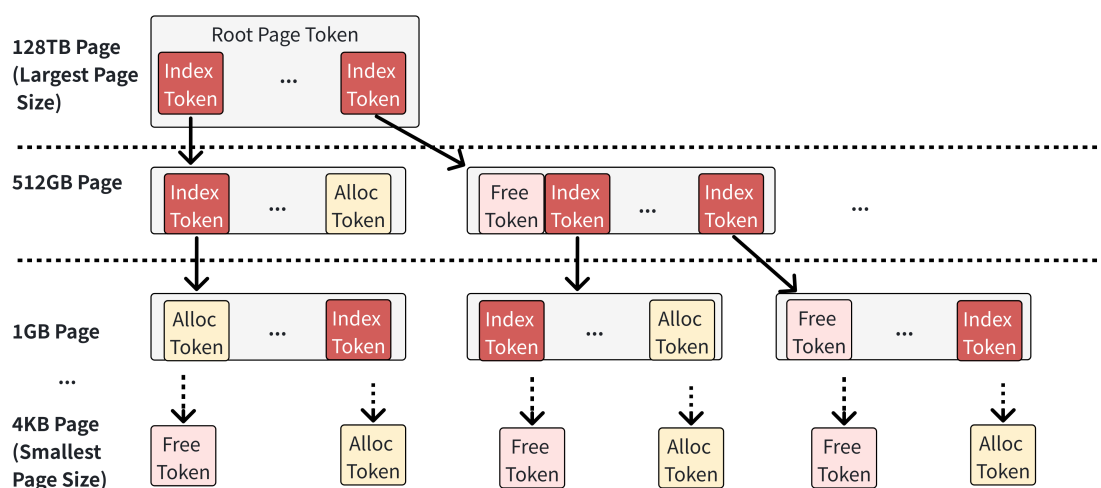


图 4-2 PageAllocator 结构

图4-2所示，页分配器采用类似多级页表与 B 树结合的层级拓扑来索引与管理物理内存块。其层级划分严格对齐 RISC-V 硬件支持的物理页尺寸。分配器中的基本管理单元定义为页令牌，每个页令牌封装了对应物理页的基址与跨度范围，并包含一个子令牌数组，用于指向当前物理页进一步拆分后的次级物理页令牌。依据状态与功能，页令牌可划分为以下三类：

- (1) 索引页令牌：作为路由与索引节点，该令牌不直接参与物理内存分配，仅负责标识其管辖的次级物理页已被分配或使用。索引页令牌内部动态维护当前内存区域经分割后所能支持的最大单次分配粒度（即最大连续物理页尺寸），从而加速分配时的容量校验；同时，它会为其子令牌数组实际分配内存，以持久化存储次级令牌状态。
- (2) 空闲页令牌：作为可供直接分配的叶子节点，该令牌标识其映射的物理

页处于完全空闲状态。为优化内存开销，空闲页令牌不会为其子令牌数组预分配实际内存空间。

- (3) 已分配页令牌：用于标识对应物理页已完成分配且不可用。在工程实现上，已分配页令牌并非实体数据结构，不占用任何元数据内存。依托 Rust 的所有权转移语义，一旦某空闲页令牌被分配，其所有权即被消耗，原有资源被自动释放。此机制在编译期确保了该令牌无法被重复引用，彻底杜绝了非法的二次分配。

分配器初始化阶段：系统首先对目标安全内存区域进行合法性校验，强制要求其基址对齐至 4KB 物理页边界。为避免直接将理论地址上限（如 128TB）作为根节点所导致的内存空间浪费与搜索深度膨胀，分配器会依据实际管理区域的容量动态界定根页令牌的物理页尺寸。随后，分配器遍历安全内存以构建页令牌树。为最小化元数据开销，系统通过计算当前基址所能对齐的最大物理页尺寸来实例化空闲页令牌，并递归查找或建立其父级索引页令牌以完成挂载。遍历结束即标志分配器初始化完成。

页令牌生命周期管理：在分配器初始态，系统中仅存在空闲页令牌与索引页令牌。当处理物理页分配请求时，系统执行如下操作流：（1）自根页令牌发起广度优先搜索，定位首个容量达标的空闲页令牌；（2）若该令牌映射的物理页尺寸与请求严格匹配，则直接消耗此空闲页令牌（逻辑上转为已分配页令牌）并返回物理地址；（3）若其尺寸溢出请求大小，则将该空闲页令牌分裂为若干次级空闲页令牌，原节点降维转化并驻留为索引页令牌，随后对新生成的次级空闲页令牌递归执行上述匹配与分裂逻辑。

在处理物理页释放请求时，系统执行逆向回收逻辑：（1）自根页令牌起始，通过计算目标物理页在安全内存中的绝对偏移量，将其映射为各级索引页令牌的子节点索引；（2）寻址至位移与尺寸均匹配的空缺位置后，利用该物理页重构出新的空闲页令牌并重新挂载入树；（3）执行碎片合并检查，若某索引页令牌下的所有次级节点均已恢复为空闲页令牌，则触发层级合并，消耗所有次级节点并将其父级索引页令牌向上聚合提升为单一的宏观空闲页令牌。

4.2.2 内存保护原语

如表4-2所示，内存保护原语抽象了隔离安全区域所需操作，包括安全硬件管理和限制/授权指定内存区域访问。

因为不同的安全硬件和内存隔离机制间存在较大差异，如 PMP 机制下每个条目直接保护连续内存区域，而 STT 下的 MPT 条目则存储地址空间的权限表，两者的管理模式完全不同。因此安全内存保护器不对安全设备管理进行约束，仅要求每

表 4-2 EasyAda-SM 内存保护原语

原语	功能
alloc	申请一个安全硬件用于隔离安全区域，返回 ID
free	根据 ID 释放一个安全硬件
disable	根据 ID 关闭指定安全硬件
enable	根据 ID 使能指定安全硬件
grant_access	在指定 CPU/hart 集合中使用指定安全硬件授权访问指定内存区域
retrive_access	在指定 CPU/hart 集合中使用指定安全硬件限制访问指定内存区域
is_protectable	检查指定内存区域是否可以被保护

种实现显式告知安全区域和内存管理层安全区域与硬件的关联关系（alloc/free 原语），以及根据关联关系实现对指定内存区域的访问权限控制（grant/retrive）。

因为不同执行环境间通常拥有各自独立的 hart，因此 RISC-V 下硬件内存保护机制通常以 hart 为单位，每个 hart 拥有独立的硬件且仅负责配置当前 hart 内存保护视图。为了实现全局统一的内存保护视图，访存控制配置需要同时同步到所有相关 hart，以安全区域为例，SM 需要同时完成在不可信组件使用的 hart 集合中限制访问安全内存和在 Enclave 使用的 hart 集合中授权访问安全内存，因此 grant 和 access 以 hart 集合为单位进行配置，不同 hart 间通过核间中断和原子操作确保配置的一致性。同时，具体的 hart 集合由更上层的 Enclave 管理模块管理，内存保护原语仅负责同步配置到指定 hart 集合。

因为 EasyAda-trust 主要使用 RISC-V PMP 机制保护安全内存，因此下面主要介绍基于 PMP 的内存保护原语实现。

4.2.2.1 基于 PMP 的内存保护原语实现

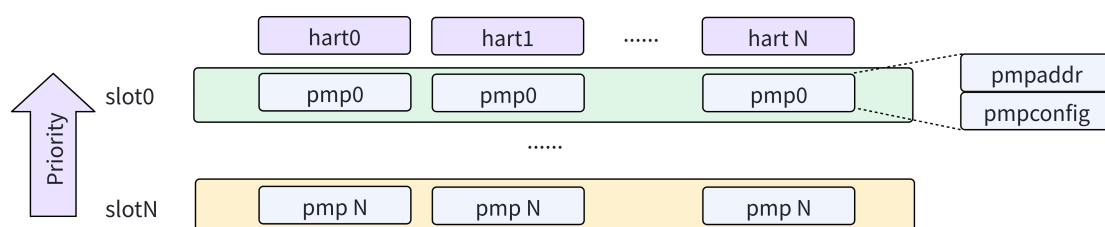


图 4-3 PMP 槽结构

PMP slot: RISC-V 下每个 hart 有独立的 PMP 条目和相关配置，仅控制当前 hart 内存访问。为了实现所有 hart 拥有统一的安全内存视图，如图4-3所示，PMP

管理器以 PMP slot 为单位管理 PMP 硬件，每个 PMP slot 是所有 hart 上索引相同的 PMP 条目的集合。

PMP 管理器：EasyAda-SM 定义的 PMP 管理器使用位图管理 PMP slot，为了提高灵活性，位图中包含三个掩码：（1）当前管理器管理的 PMP slot 索引的掩码，标记当前 PMP 管理器管理的所有 PMP slot 的索引，运行时不可变；（2）当前管理器可分配的 PMP slot 索引的掩码，标记当前 PMP 管理器总共可分配给安全区域的 PMP slot 索引，运行时不可变；（3）当前管理器已分配 PMP slot 索引的掩码，标记当前 PMP 管理器中已分配 PMP slot 索引，运行时动态修改。

在初始化时，开发者初始化当前管理器可使用和管理的 PMP slot 索引掩码（两者的区别在于，可使用的 PMP slot 可以动态分配，用于隔离安全区域，而被管理的 PMP slot 同时包括可使用和开发者预留的 PMP slot，如最高权限的 PMP slot 被用于保护 SM，虽然该 slot 可以被配置，但是不能被使用）。在配置时，安全区域可以选择需要配置的 hart 的集合，并在这些 hart 上配置关联的 PMP slot 限制/授权访问安全内存。

授权/限制访问内存：上层模块可以自由决定本次授权/限制操作覆盖的 hart 范围。当前请求权限控制时，上层模块需要传入需要控制的内存区域，需要操作的 hart 集合以及被使用的安全硬件 ID。PMP 管理器会对安全硬件可用性进行基本检查，然后在当前 hart 上配置对应的安全硬件，如果涉及多个 hart，则需要通过 RISC-V 下软件中断向远端 hart 发起请求，确保配置的原子性和一致性。

4.2.3 安全内存管理原语

表 4-3 EasyAda-SM 安全内存管理原语

原语	功能
new	创建新的安全内存管理器
init	初始化安全内存管理器
deinit	重置软硬件状态并销毁安全内存管理器
extend	创建一个新的安全区域
reclaim	回收一个空闲安全区域
alloc_em	从符合 Enclave 类型的安全区域中申请 Enclave 内存
free_em	释放 Enclave 内存到对应安全区域中
grant_access	在指定 hart 集合上授权访问指定内存区域
retrieve_access	在指定 hart 集合上限制访问指定内存区域

前文介绍了安全区域和内存管理层对内存管理和保护原语的抽象和实现，本节将介绍安全区域和内存管理层对安全内存管理原语的抽象和实现。如表4-3所示，

安全内存管理原语总体上可以分为安全内存管理器生命周期管理，安全区域生命周期管理，Enclave 内存管理和内存保护四类。

安全内存管理器生命周期管理：安全内存管理器作为安全区域和内存管理层的顶级管理单元，负责统筹安全内存资源和安全硬件资源，并使用安全硬件保护安全内存。在 `init` 阶段，安全内存管理器预期完成安全硬件的初始化和关键安全区域的初始化（尤其是 SM 本身的代码和数据），此时安全内存管理器中不存在可以被用于分配 Enclave 内存的安全区域，后续可以在安全内存管理器中动态扩展/回收安全区域。在 `deinit` 阶段，安全内存管理器会验证所有安全区域的空闲状态。为避免中断 Enclave 的正常执行，系统强制要求 Enclave 管理模块在发起 `deinit` 请求前，主动完成所有 Enclave 实例的清理。同时，因为内存清理已经在 Enclave 内存释放时完成，因此无需二次清理以避免敏感信息泄漏。然后安全内存管理器会消耗所有安全区域的所有权并记录安全内存信息归还到不可信 host。

安全区域生命周期管理：安全区域的初始化分为三个部分：内存分配，安全内存隔离，以及内存分配器初始化。内存分配通常由不可信的 host 完成；在扩展安全区域时，安全内存管理器会首先检查当前安全硬件是否支持保护安全区域，以及是否与已有安全区域重叠，确保不会破坏 Enclave 间内存隔离。然后安全内存管理器会尝试从安全内存保护器中申请一个安全硬件，使用该安全硬件隔离安全内存和不可信组件，此时安全内存仅能被安全内存管理器访问，最后使用安全内存初始化安全区域，设置当期区域为未使用，完成安全区域初始化，但是内存分配器初始化时机由安全内存管理器自行决定，本文将在后文展示延迟初始化的实现；在回收安全区域时，安全内存管理器会检查安全区域是否是未使用状态，Enclave 管理模块必须主动销毁使用当前安全区域的 Enclave，然后才能安全回收内存。

Enclave 内存生命周期管理：安全内存管理器最终的目标是管理 Enclave 分配，当 Enclave 管理模块申请 Enclave 内存时，需要告知安全内存管理器当前 Enclave 类型以选择合适的安全区域及内存分配器，安全内存管理器会遍历所有安全区域查找符合类型和内存需求的安全区域，然后尝试申请 Enclave 内存，如果申请成功则返回内存信息和安全区域编号，否则继续重复该操作直到遍历所有安全区域；当释放 Enclave 内存时，Enclave 管理模块需要提供 Enclave 内存信息，由安全内存管理器负责遍历所有安全区域并查找 Enclave 内存属于的安全区域，然后通过安全区域的内存分配器释放 Enclave 内存。

内存保护：如图4-4所示，本段以 PMP 为例介绍不安全上下文和 Enclave 上下文间切换是内存保护配置。对于不可信的上下文，SM 会通过 `retrive` 限制 hart 访问任何安全区域，其它内存可以正常访问；而对于可信上下文，SM 会通过 `grant` 授予 hart 访问 Enclave 内存所属的安全区域访问权限。因为复杂 Enclave 可能使用多

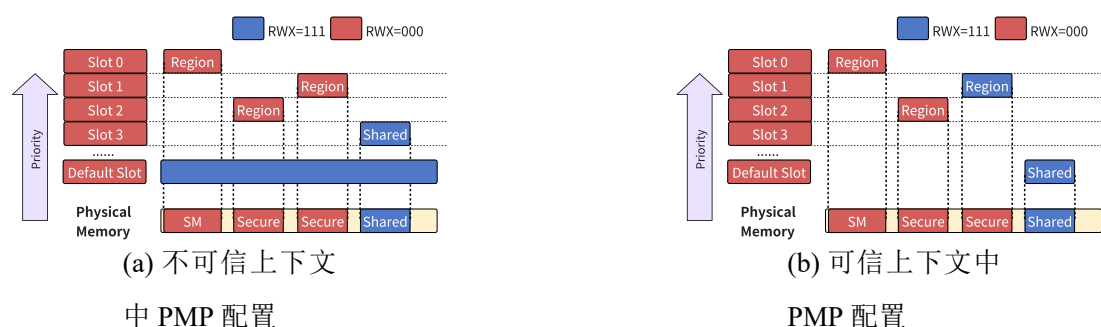


图 4-4 PMP 运行时配置

个不连续的 Enclave 内存和多个 hart，需要在每个 hart 上授权访问多个安全区域，并通过 hart 生命周期管理接口实现其它 Enclave 使用的 hart 上下文的安全切换。

这里需要额外关注 default slot，该 slot 用于赋予任何 hart 默认访问所有内存区域的权限，避免因为保护安全内存导致 hart 无法访问不可信内存，图4-4(b)展示了 OS Enclave 的管理方式，因为其具备配置 MMU 的能力且直接运行在内核态下，需要修改 default slot 禁止访问所有物理内存空间，而对于 App 和 VM Enclave 则不强求，因为 Enclave 与 MMU 相互隔离。

下面将详细介绍 EasyAda-SM 针对多种 Enclave 抽象提供统一管理的安全内存管理原语实现。

4.2.3.1 面向通用 Enclave 部署场景的安全内存管理原语实现

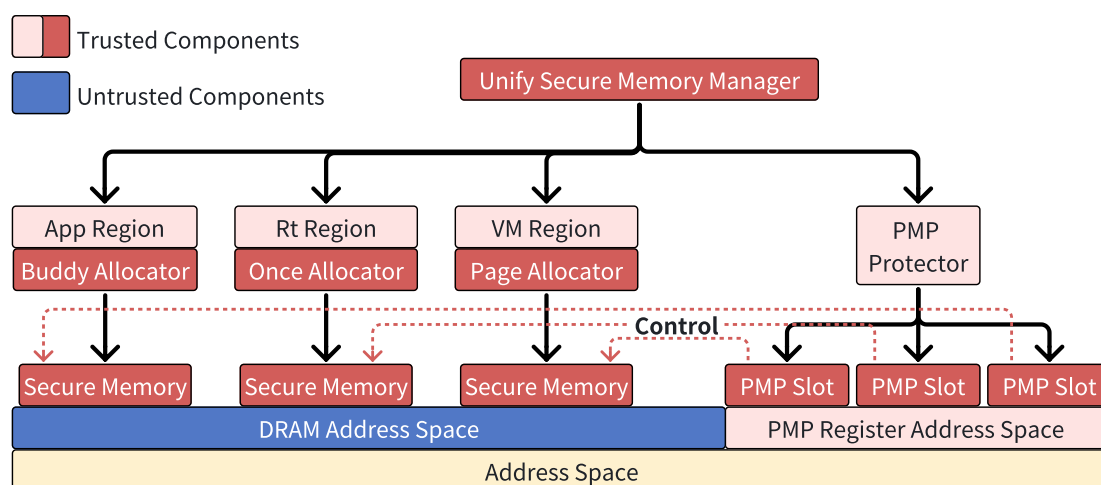


图 4-5 PageAllocator 结构

为了支持通用的 Enclave 管理，降低用户开发难度，EasyAda-SM 实现了如图4-5所示的通用安全内存管理器（简称为通用内存管理器），将 Enclave 和安全区域划分为一一对应的五类：（1）Application（2）Runtime（3）VirtualMachine（4）Devices

(Devices 会进一步划分为串口, 硬件加速器等设备类型) (5) None (仅用于保护的安全区域), 并且如图4-6所示, 为每类 Enclave 绑定了特定的内存分配器。具体

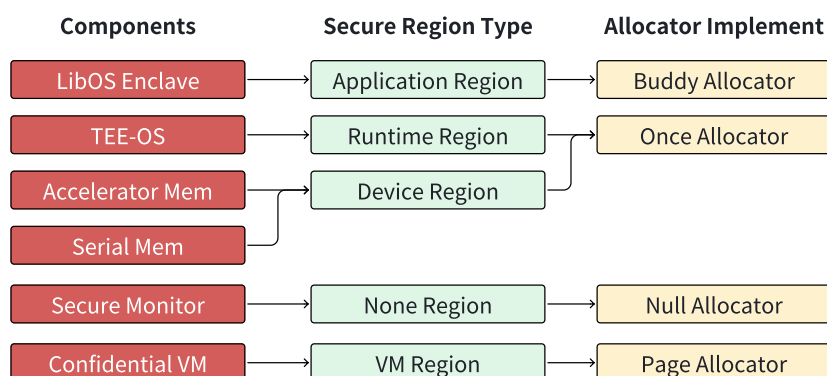


图 4-6 不同类型 Enclave 与区域类型和分配器的关联关系

来说, 不同 Enclave 内存与不可信组件的隔离机制相同均基于 PMP 实现, 区别在 Enclave 内部内存管理和 PMP 权限配置不同。因此 EasyAda-SM 引入了通用安全区域表示多种安全区域状态, 为了符合 Rust 的类型检查要求, EasyAda-SM 定义了包含所有分配器实现的枚举 (类似 C/C++ 中的 union), 安全区域通过在不同的分配器枚举间切换实现状态切换。并且状态切换按照以下状态机进行。

安全区域状态转换: 初始状态下, 所有安全区域均为未使用状态, 使用空分配器管理内存, 仅实现内存隔离, 无法进行任何内存分配; 当申请 Enclave 内存时需要显式指定 Enclave 类型, 通用内存管理器会遍历查找类型匹配的安全区域并尝试申请安全内存, 若当前类型匹配则直接分配, 或者当前区域未被使用则转化为目标 Enclave 类型区域, 初始化内存分配器并尝试分配内存, 否则继续查找符合条件的区域; 当释放 Enclave 内存时, 通用内存管理器会遍历查找包含 Enclave 内存的安全区域并尝试释放内存, 若释放后安全区域无内存分配, 则消耗当前安全区域所有权, 转换为未使用安全区域。

除内存管理外, 不同 Enclave 抽象在安全区域保护逻辑上存在较大差异, 但是该部分不属于通用内存管理器的职责范围, 下一节将介绍如何基于通用内存管理器实现面向 CVM 的 Enclave 管理模块。

4.3 面向 CVM 的 Enclave 管理模块实现

如图4-7所示, CVM 管理模块包括四个主要模块:

- (1) 通用安全内存管理器: 如前文所说, 通用安全内存管理器目标是实现安全内存与不可信组件间的隔离, 负责统一管理安全设备和安全内存区域。
- (2) CVM 保护器: CVM 保护器的目标是实现 CVM 间的地址空间隔离。因

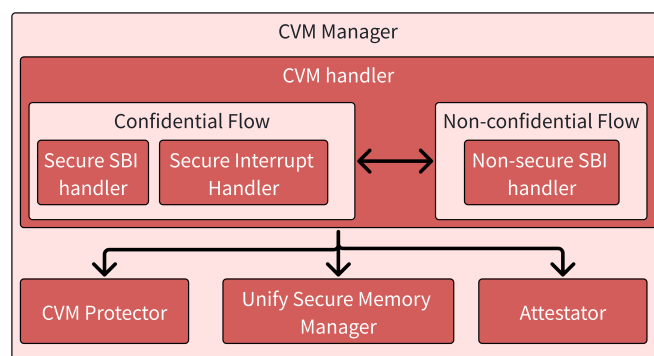


图 4-7 CVM 管理器结构

为 CVM 管理模块使用二阶段地址翻译机制在 CVM 间实现地址空间隔离的，因此该组件实际负责 CVM 的二阶段页表管理。

- (3) CVM 证明器：CVM 证明器的目标是实现 CVM 的可信启动，负责完成 CVM 初始状态度量，TAP 解密和密钥管理。
- (4) CVM 处理程序：CVM 处理程序是 CVM 管理模块的统一入口，负责实现对不可信组件和 CVM 所有异常和中断的拦截和处理，机密处理流和非机密流间敏感数据的隔离和安全上下文切换。

可以将该模块视为 TSM 的一种实现，后文直接以 TSM 代指 CVM 管理器。

4.3.1 Enclave 间内存安全

Enclave 间内存安全的核心是隔离不同 Enclave 的地址空间，安全内存管理器实现了不可信组件和安全内存间的隔离，如果直接基于其实现 CVM 间隔离会快速耗尽可用安全硬件，减少可管理的 CVM 数量。因此 TSM 利用 CVM 基于硬件虚拟化扩展运行的特点，引入了 CVM 保护器基于二阶段页表管理和实现 Enclave 间内存安全。

CVM 保护器的核心职责是维护当前 CVM 的二阶段页表，包括初始化 CVM 二阶段页表，映射/解映射安全内存和不可信内存以及使用二阶段页表翻译得到物理地址。为了避免 CVM 保护器耗尽 SM 的堆内存，CVM 保护器构建二阶段页表使用的内存主要来自于从安全内存管理器中申请的内存。为了提高系统内存利用率，CVM 保护器允许从多个安全区域中申请，因为二阶段地址翻译和页分配器保证同一物理页仅能分配给一个 CVM 且 CVM 仅能访问指定的地址空间，因此不会破坏 CVM 间内存隔离。

在 CVM 初始化时，CVM 保护器使用安全内存根据不可信 VM 的二阶段页表构建完全对等的 CVM 初始二阶段页表，并将所有物理页中的内容拷贝到安全内存中，该操作会耗费大量内存和 CPU 运行周期，但是嵌入式场景下 CVM 部署之

后通常会持续运行直到下一个电源周期，而且安全性是 EasyAda-SM 的首要考虑目标，因此 EasyAda-SM 认为在系统启动时该操作带来的延迟和性能降低是可以接受的。在系统运行时，CVM 保护器会负责二阶段页表的动态维护。

4.3.2 安全启动

安全启动的核心是确保只有被验证证明了可信且未被篡改的软件实体才能访问敏感资源。CVM 管理模块引入了验证器模块用于管理密钥并对 CVM 进行度量。

在系统开发阶段，EasyAda-trust 的硬件平台供应商会将硬件密钥固化在 RoT 中，并对上提供硬件公钥，可以进一步使用硬件公钥对 EasyAda-SM 的预期度量值进行签名。在系统初始化阶段，RoT 负责对 SM 进行度量并结合硬件密钥与数字签名进行对比，如果度量成功则转移控制权到 SM，并传递硬件密钥给 SM。该私钥后续会由 TSM 用于 CVM 的安全启动。

在系统初始化时，CVM 管理模块会初始化验证器模块用于存储硬件私钥，并作为根密钥用于派生其他密钥。因为此时 SM 所在内存区域已经通过硬件实现隔离，因此硬件密钥无法被不可信组件窃取。

内存布局度量：当 Hypervisor 请求提升 VM 为 CVM 并陷入 TSM 时，TSM 会尝试从安全内存中为 CVM 的 hart 分配元数据区域，用于存储 CVM 的 hart 上下文信息，并使用安全内存创建 VM 内存布局的安全副本，包括页表本身和物理页中的内容。然后 TSM 将内存布局副本和 FDT 传递给验证器，由验证器遍历内存布局副本并根据 FDT 对内存中的关键数据进行度量（包括 kernel，initrd 和 FDT 等）并生成运行时度量值，验证器并不关心非关键区域的内容，在度量过程中验证器会负责清理所有非关键内存区域内容，避免不可信组件在其中存储恶意数据和代码。在内存布局度量完成后，验证器会将度量值存储到 CVM 在安全内存中的元数据中。

CVM 上下文度量：完成内存布局度量后，验证器会进一步度量 CVM 元数据区域中存储启动 hart 的初始上下文信息，包括通用寄存器和控制寄存器信息。因为除了启动 hart，其他 hart 的启动均由启动 hart 通过 hart 状态管理扩展（Hart State Management，HSM）扩展进行管理，初始上下文由 TSM 进行设置和保护，所以无需进行度量。验证器同样会将初始上下文度量值存储在 CVM 对应的元数据中。

对比预期度量值：在系统初始化阶段，TSM 从 RoT 中获取了硬件私钥。验证器完成对 CVM 的度量之后，使用硬件密钥解密 CVM 开发者提供的锁盒，然后使用解密得到的对称密钥解密 TAP，得到 CVM 开发者提供的预期度量值和开发者密钥。如果本地计算生成的 CVM 上下文和内存布局度量值与 TAP 解密得到的预期度量值或数字签名匹配，则证明当前 CVM 可信。

密钥管理：除了硬件私钥外，验证器还需要保护和管理 CVM 开发者提供的密钥。这些密钥通常会用于（1）CVM 解密机密应用或根文件系统等（2）作为根密钥直接或衍生密钥对网络，串口和磁盘块设备等 I/O 数据进行加解密。CVM 可以在运行时通过 SBI 请求从 TSM 获取密钥。

4.3.3 中断和异常安全

表 4-4 CVM 管理模块在机密流支持的句柄类型

类别	功能描述
标准 SBI 扩展	包括（1）探测 SBI 基础扩展支持，获取硬件基本信息（2）发送处理器间中断，远程刷新指令缓存和虚拟内存地址等（3）Hart 状态管理，启动、停止、挂起机密 hart，查询 hart 运行状态等（4）系统控制，请求系统重置或关机
I/O 虚拟化扩展	（1）MMIO 服务请求，向不可信 host 发起 MMIO 读写请求（2）不可信的共享内存管理，向不可信的 host 请求共享内存，用于实现 I/O 半虚拟化（3）安全设备管理，请求直接映射/移除安全设备物理内存地址到 CVM，用于实现设备直通。
密钥管理	从 TSM 中获取 CVM 的特定密钥等机密数据
机密异常处理	处理 CVM 二阶段地址访问异常，虚拟指令等

表 4-5 CVM 管理模块在机密流支持的句柄类型

类别	功能描述
标准 SBI 扩展	所有 RustSBI 实现的标准 SBI 扩展，由 CVM 处理程序调用 RustSBI 的接口实现。
CVM 生命周期管理	实现不可信 host 对 CVM 生命周期的管理，包括将普通 VM 提升为 CVM，运行 CVM 和销毁 CVM。
I/O 虚拟化管理扩展	（1）MMIO 请求回复，回复 CVM 的 MMIO 请求处理结果。（2）共享内存请求回复，回复 CVM 共享内存请求处理结果以及分配的共享内存信息。
安全区域管理扩展	管理安全区域的创建和回收，不可信 host 通过此扩展捐赠内存创建新的安全区域/回收空闲安全区域使用内存

中断和异常安全的核心是确保机密流和非机密流之间的敏感数据互不泄露，并且阻止不可信组件通过中断或异常攻击 CVM。

在标准流程中，M 态中断和异常的入口函数由 SBI 固件实现和初始化。而在

EasyAda-SM 中，CVM 管理模块会修改 M 态陷入入口函数以确保 CVM 管理模块负责响应和监控所有异常和中断，再根据需要调用 RustSBI 固件提供的能力。这样的好处是无需在执行环境切换时重新配置入口函数以实现 CVM 中断和 SBI 请求的接管和确保 CVM 管理模块可以监控所有敏感操作。

对于非 TEE 环境，即标准的 Hypervisor 和不可信 VM 流程，中断和异常默认委派到 HS 态进行处理（需要由 Enclave 管理的设备的外部中断必须在 M 态下进行处理），Hypervisor 可以进一步委派中断到 VS 态进行处理。但是对于 TEE 环境，默认情况下 CVM 管理模块会将所有 VS 态中断委派到 CVM 中，但是其它中断均需要陷入 M 态。而对于机密流下的异常，大部分会委派到 CVM 中进行处理，而 GUEST_PAGE_FAULT 等异常则直接陷入到 M 态进行处理。

不管是异常还是中断，最后都会进入 CVM handler 进行统一处理。CVM handler 实现了所有 CVM 管理模块的对外提供的服务接口，如表4-4和4-5所示，根据 hart 当前状态划分为机密流和非机密流两类，当前 hart 如果运行在 Enclave 中，则会进入可信流处理，反之则进入不可信流处理。

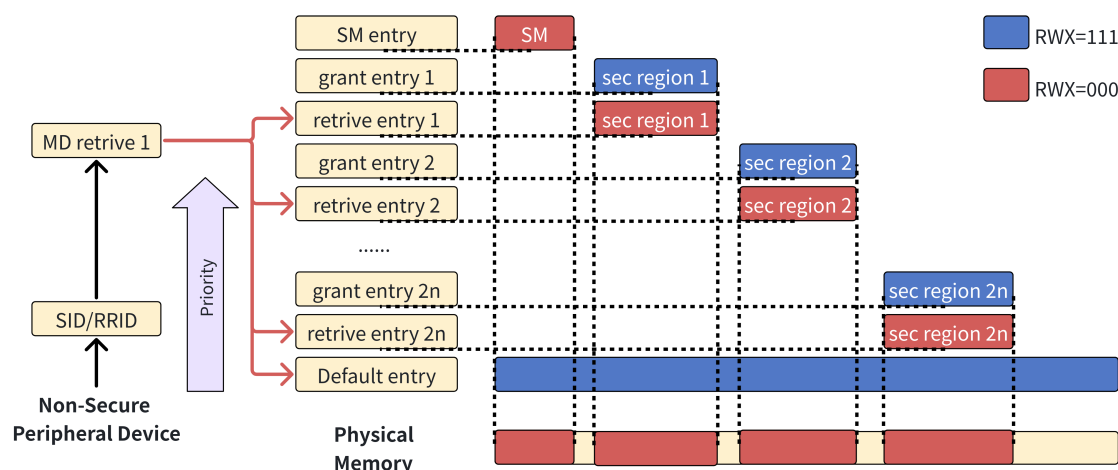
4.3.3.1 机密流和非机密流

因此，CVM handler 的核心工作之一是实现不可信组件和 CVM 上下文间的安全切换。EasyAda-SM 将其 hart 的上下文切换分为普通上下文切换和安全上下文切换两个阶段。CVM/Hypervisor 进入 SM 时首先会执行普通上下文切换，保存当前执行必须的通用寄存器和配置寄存器，然后进行处理。如果需要切换执行环境，即跨处理流处理时，CVM 管理模块会进行安全上下文切换，将 hart 当前所有执行状态保存到安全内存中，同时加载目标上下文的执行状态，并且，CVM 管理模块还会通过通用内存管理器执行关键的内存访问权限调整和清理操作（如 TLB），避免当前软件实体的敏感信息泄漏给将要执行的一方，并确保正确配置将要执行的软件实体相关内存资源的权限。

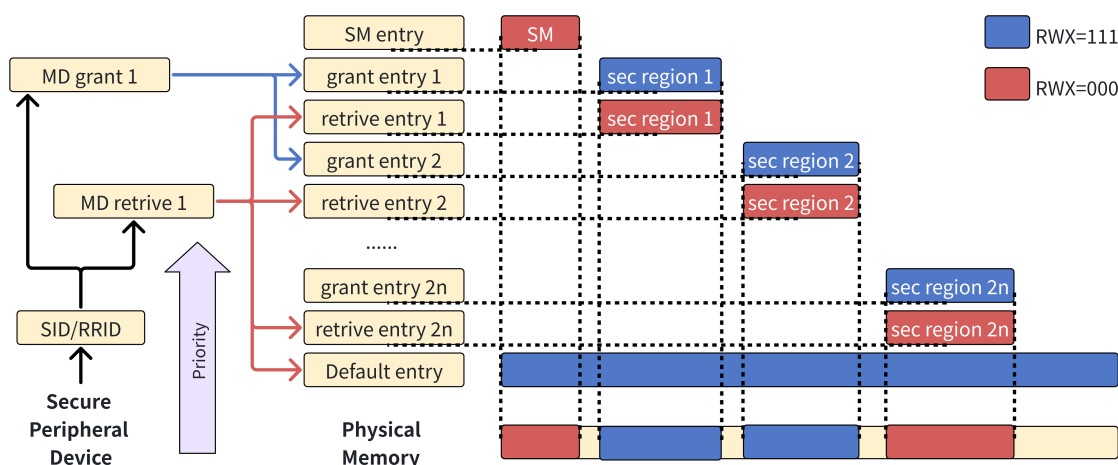
4.3.4 IOPMP 管理

为了防止支持 DMA 的恶意外设绕过 CPU 的内存隔离直接访问安全内存，EasyAda-trust 使用 IOPMP 扩展限制了每个硬件可以访问的内存区域。在开发时，硬件供应商需要为可信设备静态分配 SID 作为唯一标识符号。

在系统启动时，TSM 创建安全区域是会同时完成对最高特权级的 IOPMP 条目（SM 条目）的配置，限制任何设备对 SM 内存区域的读写，同时配置最低优先级的条目（default 条目）赋予设备默认访问所有不可信内存区域的权限。此时设备可以正常使用，但是无法读写 SM 中的代码和数据。同时，TSM 会为所有设备



(a) 非安全设备的 IOPMP 配置



(b) 安全设备的 IOPMP 配置

图 4-8 EasyAda -SM 中非安全/安全设备的 IOPMP 管理

的 SID 创建，绑定并启用一个用于限制访问的 MD，初始状态下每个 MD 中仅包含 SM 和 default 条目。

图4-8展示了 EasyAda-trust 使用 IOPMP 控制设备访问安全区域的流程。在系统运行时，EasyAda-trust 以 IOPMP 条目对的形式管理 IOPMP 硬件，每当一个安全区域创建时，TSM 会按序申请第一对空闲的 $2n-1$ 和 $2n$ 表项， $2n-1$ 用于授权访问安全区域的， $2n$ 则用于限制访问安全区域。如图4-8(a)所示，对于不安全的设备，每当 TSM 创建新的安全区域和 IOPMP 表项时，会同步当前配置到所有设备的限制 MD 中，默认限制所有设备对安全区域的读写。

同时，Hypervisor 可以在任何时刻请求将可信设备提升为安全设备，TSM 会通过可信的硬件配置文件验证设备可信，并创建安全区域隔离安全设备的 MMIO 区域，以及为安全设备创建未初始化的，用于授权访问的 MD。当安全设备实际分配给 CVM 时，TSM 会将 CVM 关联的所有安全区域的授权条目加入设备的授权

MD 中，并切换安全设备的 MD 为授权 MD。如图4-8(b)所示，此时安全设备仅能访问 CVM 关联的安全区域，杜绝了安全设备将敏感数据写入不可信内存的隐患。

TSM 仅在设备提升为安全设备时，或从安全设备回退为不安全设备时进行重新配置，避免频繁切换 IOPMP 配置时上下文切换和缓存一致性带来的性能开销。

4.4 本章小结

本章系统论述了基于微内核的可信虚拟化方案的工程实现。在 CPU 虚拟化层面，通过重构 vhart 分区架构，并由 TSM 接管两阶段上下文切换，实现了 CVM 运行状态对不可信环境的彻底隐匿。在内存虚拟化层面，实施了资源分配与安全管控的深度解耦：Hypervisor 仅负责物理资源的宏观调度，而 TSM 全权把控安全区域的生命周期与二阶段页表的隔离映射。在 I/O 虚拟化层面，确立了主机分配、TSM 鉴权的安全直通设备管理流程，依托 IOPMP 扩展构筑了防御 DMA 攻击的物理屏障，并结合动态密钥供应实现了数据平面的端到端保护。最后，面向端侧异构计算需求，构建了专用的硬件加速器管理 CVM，基于不可信共享内存与半虚拟化技术安全打通了跨域通信信道，实现了加速器的机密部署与安全复用。

第五章 基于微内核的可信虚拟化方案实现

5.1 安全启动流程实现

EasyAda-trust 下 RoT 是安全启动的起点。如图5-1所示，硬件供应商需要提供验签密钥对。验签公钥对 SM 预期度量值生成数字签名，并将数字签名存储在 RoT 中。验签私钥则用于在开发时生成可信 SM 的数字签名。在系统启动流程中，RoT 会计算 SM 的实际度量值，并根据 RoT 中的验签公钥对比预存的数字签名完成验证，验证通过后 RoT 会将控制权转交给 SM 并进入安全启动的下一阶段。除了硬件供应商统一的验签密钥对，每个设备拥有独有的，无法篡改的设备私钥对，设备公钥向平台供应商和开发者开放，用于加密验证信息或敏感数据，而设备私钥存储在设备 RoT 中，用于在运行时进行解密。

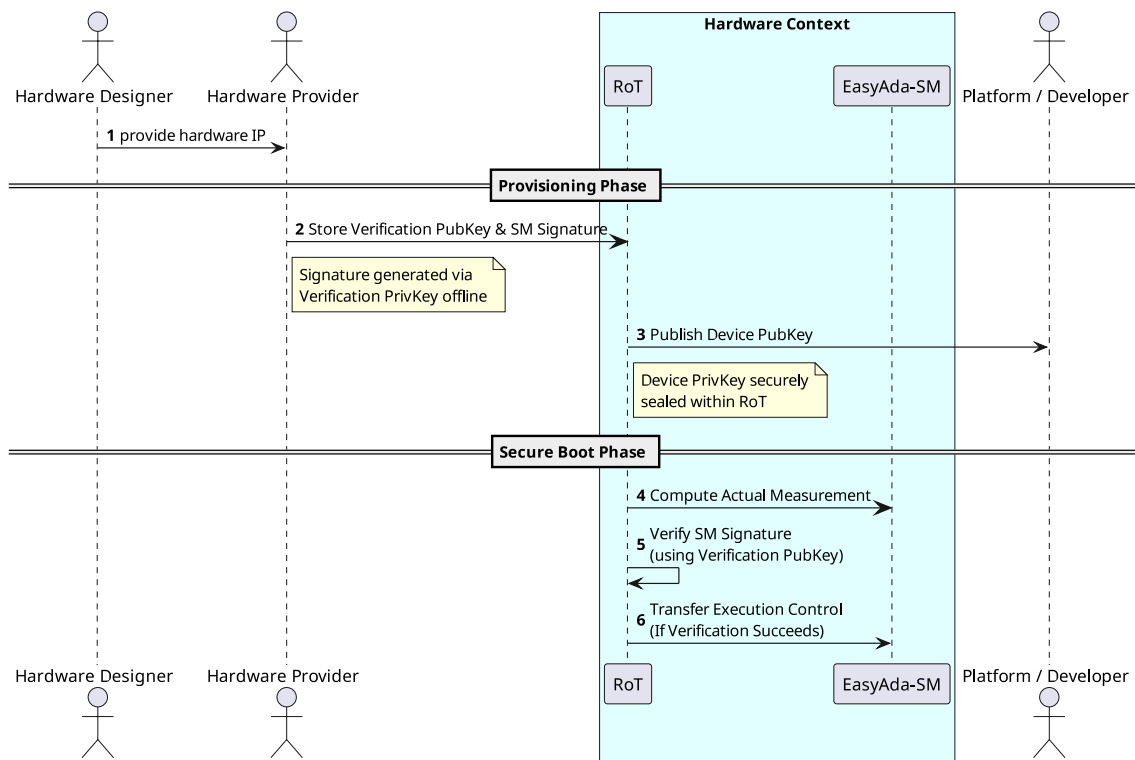


图 5-1 EasyAda-SM 签名验证流程

SM 初始化流程分为两个部分：TSM 初始化和 SBI 运行环境初始化。因为 SBI 固件被视为不可信或实现可能存在漏洞，因此 SM 初始化流程由可信的 TSM 主导。TSM 初始化时会首先判断当前硬件环境是否满足部署 EasyAda-trust 的最低要求，然后清空 RoT 副本并初始化根安全区域隔离 SM 自身内存，防止 SM 在系统运行过程中被运行在更低特权级的不可信组件非法访问和篡改。

完成 SM 内存隔离后，TSM 初始化流程会调用 SBI 固件实现的接口完成 SBI 运行环境初始化，然后由 TSM 完成剩余模块如验证器等初始化并重新配置 SM 的陷入入口，确保低特权软件的执行在明确定义的入口点陷入到 TSM 和 SM 中，最终转移控制权到下一阶段引导程序或 Hypervisor。当 TSM 初始化完成时，SM 已经通过硬件实现了与不可信组件的强制隔离，可以安全进入更低特权级并转移控制权到 Hypervisor 继续下一阶段初始化。

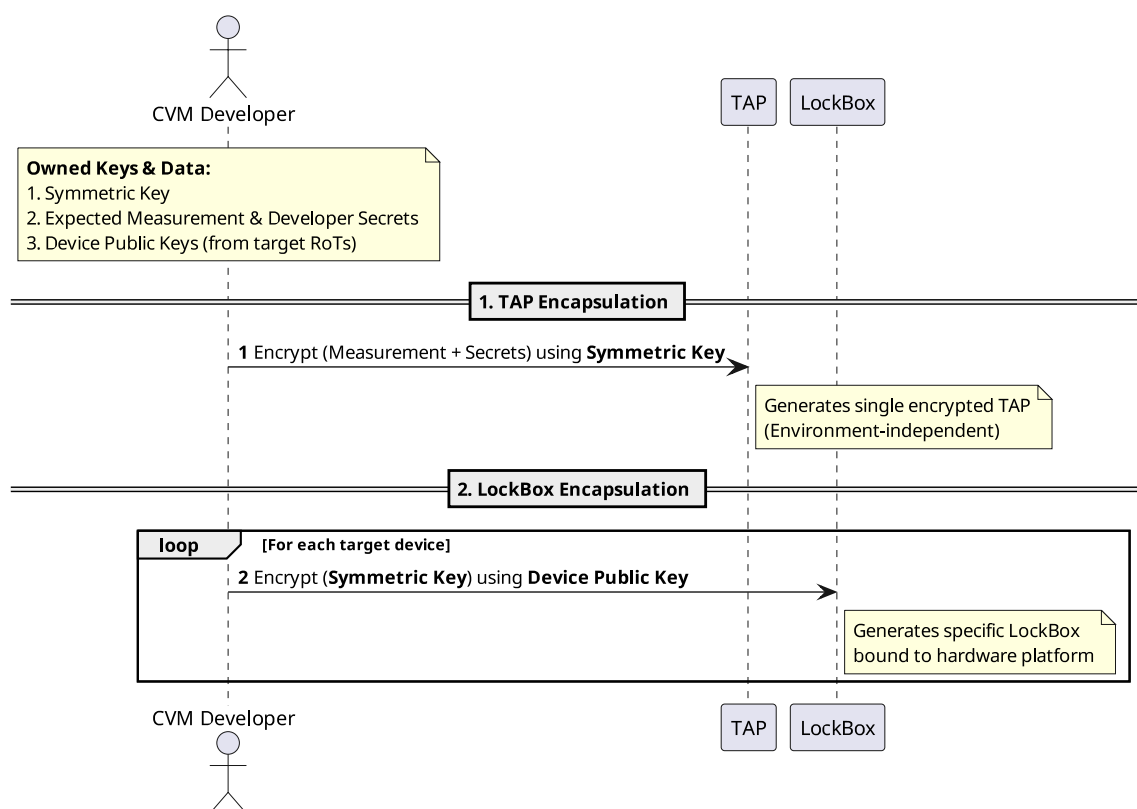


图 5-2 机密虚拟机验证信息封装流程

为了支持 CVM 和硬件环境的本地证明。如图5-2所示，CVM 开发者必须使用开发者提供的密钥对称加密 TEE 证明载荷（TEE Attest Payload, TAP）文件。该文件用于存储 CVM 预期度量值和开发者提供的用于加解密敏感 I/O 数据，代码和参数等的秘密，并且配合一个使用当前设备公钥加密，用于存储加密 TAP 的对称密钥的锁盒（LockBox）。为了支持在多个设备上部署，同一 TAP 可以关联多个使用不同设备公钥加密的锁盒。

在部署 CVM 时，SM 会将 CVM 的初始化状态保存到安全内存和 hart 结构体中并通过验证器模块对关键数据进行度量等到当前度量值，然后使用 RoT 提供的设备私钥解密锁盒，并使用锁盒中的对称密钥解密 TAP，最终得到 CVM 预期度量值和开发者提供的秘密。如果实际和预期度量值对比成功则验证通过。TSM 会将

从 TAP 中解密获得的开发者秘密存储在验证器模块中。CVM 可以通过 SBI 请求获取当前 CVM 开发者提供的秘密，其它 CVM 或不可信组件无法窃取或篡改。

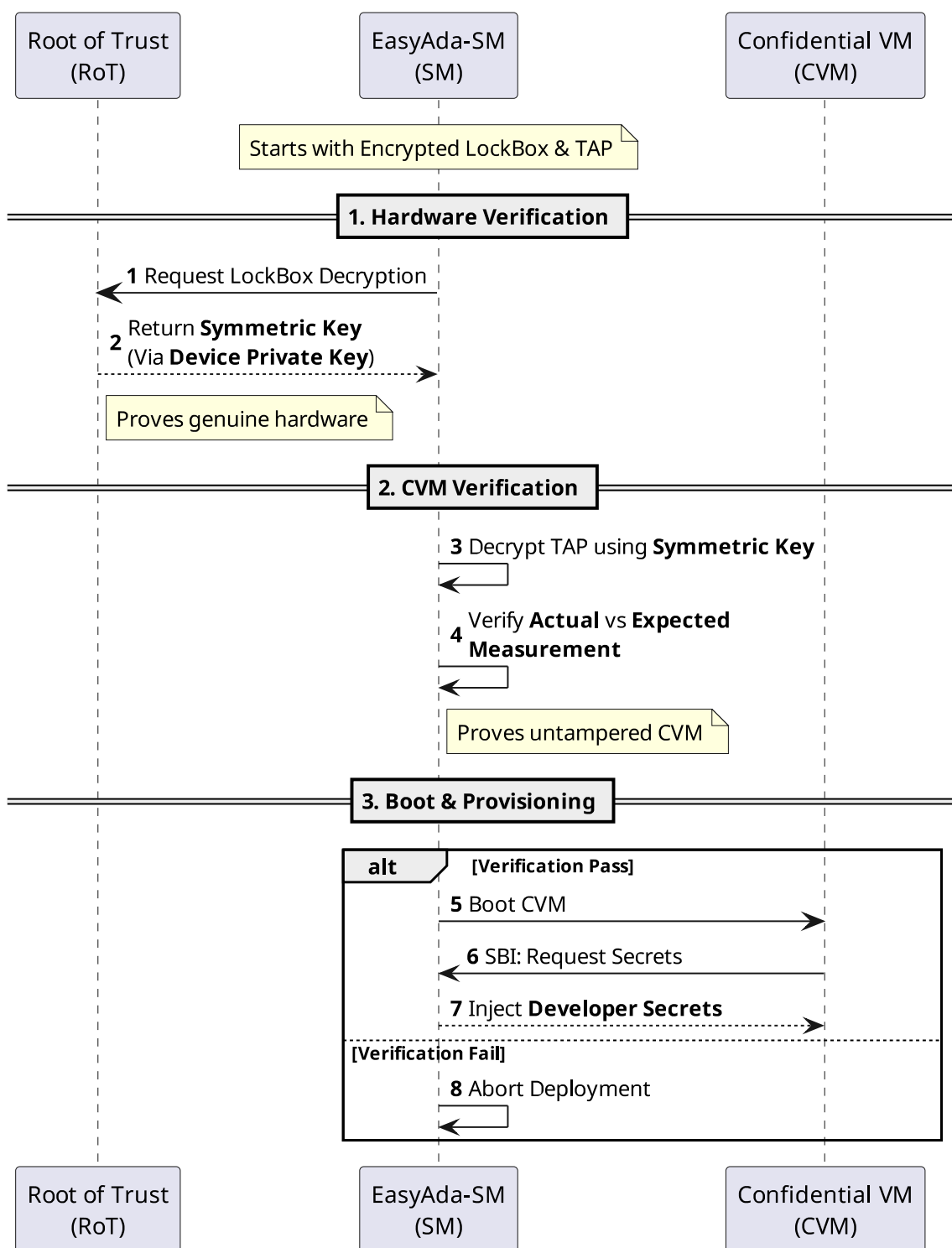


图 5-3 EasyAda-trust 中双向验证流程

本地证明可以实现 CVM 与硬件环境的双向证明，如图5-3所示，因为存储 TAP 加密密钥的锁盒仅能在持有对应硬件私钥的硬件环境上被正确解密，即使 CVM 被

强行拉起，不可信环境仍然无法获取 CVM 开发者提供的特定秘密，无法从 CVM 中解密任何敏感代码或数据；另一方面，任何对 TAP，锁盒或者 CVM 镜像的篡改都会导致最终的度量失败，确保只有可信的，未经过篡改的 CVM 可以被部署。同时，本地证明与远程证明并不冲突，在具备网络条件的环境上仍然可以使用远程证明验证可信环境。

5.2 安全 CPU 虚拟化实现

EasyAda-Hypervisor 将虚拟 CPU (vhart) 抽象为微内核操作系统中的线程进行调度。在其线程控制块 (Thread Control Block, TCB) 中，除包含标准的通用寄存器状态外，还额外记录了当前虚拟机 (VM) 的控制寄存器信息，以确保能够精确恢复 VM 的运行上下文。此外，EasyAda-Hypervisor 为每个 vhart 构建了两个独立的执行上下文：vhart 运行线程与 vhart 服务线程。前者负责存储与调度 VM 的运行时计算上下文，后者则专职处理 VM 的异步 I/O 请求。

为提升系统整体性能，EasyAda-Hypervisor 将物理 hart 静态划分为三类功能区：(1)vhart 运行区：主要承担 VM 及其原生应用的计算任务；(2)vhart 服务区：专用于承载 vhart 服务线程，以异步方式高效响应 VM 的 I/O 请求，最大程度降低对 VM 计算任务的干扰；(3) 系统服务区：专用于运行内存管理、进程管理及文件系统等微内核核心系统服务，旨在消除因任务调度抖动而引发的系统性能衰退。

EasyAda-trust 在此 CPU 虚拟化架构的基础上进行了安全维度的扩展，引入了额外的隔离机制，以实现 CVM 与不可信 Hypervisor 之间上下文的严格物理隔离。

5.2.1 安全上下文管理

在 EasyAda-trust 架构中，底层可信安全监视器 (TSM) 驱动不介入不可信 VM 的 vhart 管理，该职责仍由 Hypervisor 保留。然而对于 CVM 的 vhart，尽管其宏观调度策略仍由 Hypervisor 负责，但其核心上下文状态必须由 TSM 进行机密管理，并对 Hypervisor 呈现完全的不可见性。Hypervisor 仅能通过 TSM 暴露的有限的 CVM 生命周期管理接口，间接完成对 CVM vhart 的调度操作。

如图5-4所示，EasyAda-trust 在原生 hart 分区机制的基础上，将 vhart 运行区进一步细分为 CVM vhart 运行区与不可信 VM 运行区。两者在架构上的核心差异在于跨执行环境时的上下文切换流程。

5.2.1.1 安全上下文切换

为规避非必要的上下文保存与恢复开销，EasyAda-Hypervisor 将 vhart 上下文切换机制划分为“执行环境内轻量级切换”与“跨执行环境安全切换”两个层级。

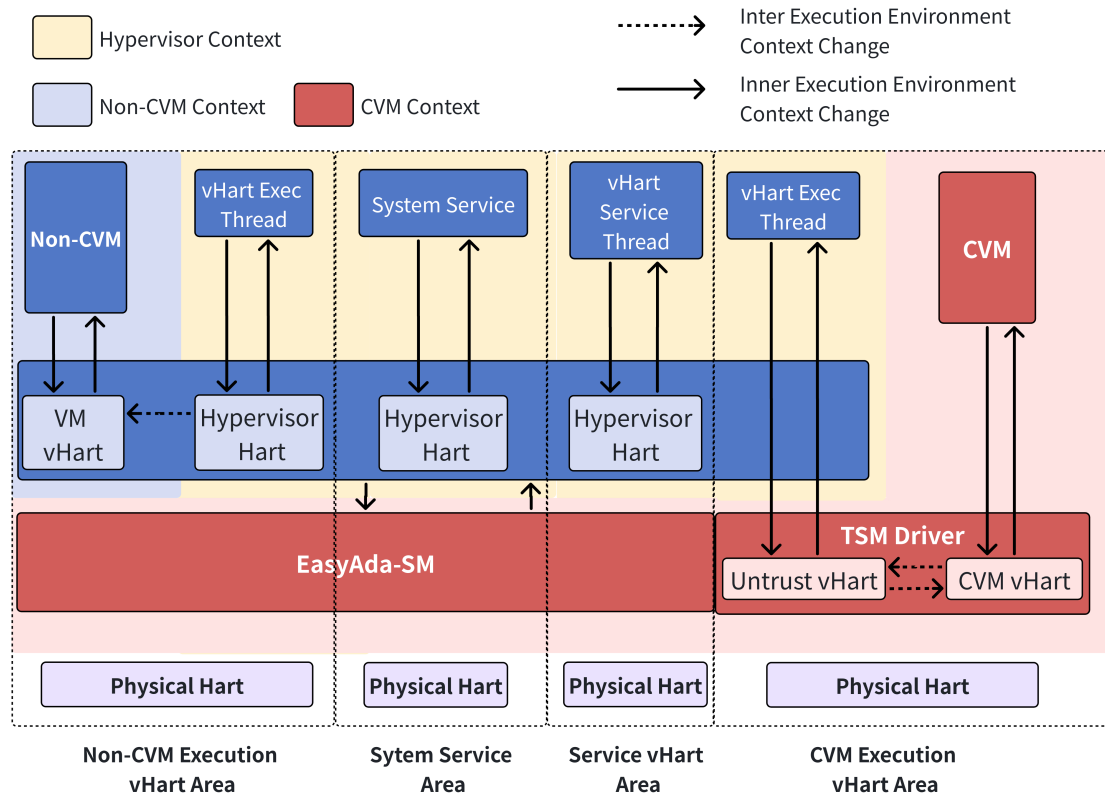


图 5-4 EasyAda-trust 下 hart 分区示意图

对于不可信 VM，Hypervisor 在逻辑上将其上下文解耦为 VM 侧的 VM 上下文与 Hypervisor 侧的 vhart 运行上下文。两者均各自维护一套完整的状态集合，具体包括：(1) 最小架构状态寄存器集合 (含通用寄存器及 TSM 运行时可能变更的控制寄存器)；(2) 虚拟化控制寄存器状态；(3) 虚拟化功能寄存器状态。由于 Hypervisor 在自身运行期间不会修改上述 (2) 与 (3) 中的寄存器状态，因此 VM 与 Hypervisor 之间的上下文切换仅需保存集合 (1) 的信息即可。然而，在不同 VM 实例之间进行切换时，则必须完整保存并清理涵盖 (1)、(2)、(3) 在内的全量上下文信息。为此，Hypervisor 通过 vhart 运行线程统一管理这两类逻辑上下文状态。

相比之下，CVM 的上下文切换呈现出以下显著的安全特征：

- (1) 扩展的上下文保护范围：鉴于 CVM 对 Hypervisor 处于完全不信任状态，除上述三类基础上下文信息外，TSM 还必须额外切换并保护机器模式 (M-Mode) 与硬件虚拟化监管者模式 (HS-Mode) 的控制寄存器 (例如中断与异常委派状态寄存器等)。
- (2) 物理内存视图的严格隔离：CVM 与 Hypervisor 分别运行于相互隔离的物理内存视图中。在触发上下文切换时，TSM 必须动态重配置底层安全硬件 (如 PMP)，以维持安全内存与不可信组件之间的物理隔离边界。

(3) 执行流的安全阻断与切换: CVM 与 Hypervisor 的执行流彼此互斥。在处理特定异常 (如 MMIO 访问异常) 而触发执行流切换时, 必须确保 Hypervisor 无法窥探或获取任何属于 CVM 及 TSM 的上下文残余信息。

针对上述特征, EasyAda-trust 将 CVM 相关的上下文严格划分为“不可信 vhart 上下文”与“机密 vhart 上下文”两类。前者负责暂存 Hypervisor 侧 vhart 运行线程的全部状态, 后者则专用于安全存储 CVM 的全量状态。两者均涵盖所有通用寄存器以及 VS/HS/M 态系统控制寄存器; 若底层硬件使能了浮点或向量指令扩展, 还需额外保护浮点或向量处理单元的上下文信息。

为缓解 CVM 与 Hypervisor 间频繁交互带来的性能损耗, 若当前 hart 控制流未跨越信任边界, 则仅执行轻量级上下文切换; 反之, 则必须执行完整的安全上下文切换流程。

轻量级上下文切换: 当当前 hart 从 Hypervisor 或 CVM 陷入 TSM 执行特定服务时, 由 TSM 直接原地完成请求处理。在此过程中, hart 仅需临时保存当前执行环境的通用寄存器, 待 TSM 处理完毕后直接恢复。由于该过程不涉及执行环境的越界切换, 因此无需动态重构物理内存保护视图, 切换开销极小。

安全上下文切换: 当当前 hart 必须在 Hypervisor 与 CVM 之间转移控制权时, 需严格遵循 CVM 管理模块的状态机约束。在完成必要参数的安全校验与传递后, TSM 会将当前全量上下文状态加密 (或隔离) 存储至对应的 vhart 结构中, 随后彻底清理当前物理环境的残余寄存器状态, 最后加载目标执行环境的上下文, 从而完成执行流的安全迁移。在执行目标指令前, TSM 必须重新配置安全硬件以切换物理内存视图。具体而言, 当控制流由不可信 vhart 切换至机密 vhart 时, TSM 会授权当前 hart 访问除安全监视器 (SM) 自身内存外的所有关联安全区域及不可信内存 (此时依靠二阶段页表机制, TSM 足以严密管控 CVM 的最终物理寻址能力); 反之, 当控制流由机密 vhart 切回不可信 vhart 时, TSM 将立刻剥夺当前 hart 对所有安全区域的物理访问权限。

5.2.2 安全异常处理

如第4.3.3节所述, EasyAda-SM 通过 Rust 语言的静态内存检查机制, 从编译源头消除了因不安全内存访问而导致的敏感数据泄漏隐患, 并凭借构造-处理-析构的规范化生命周期模型, 严格界定了各项处理服务所能触达的资源边界。与此同时, TSM 在架构设计上刻意摒弃了向不可信组件内存空间写入数据的能力, 从根本上阻断了机密数据向不可信域泄漏的物理路径。

然而必须强调的是, 尽管 TSM 在底层机制上保障了不同处理器与执行流之间的物理安全, 但 CVM 内部的客户操作系统仍需肩负起正确调用 TSM 安全接口的

责任。例如，当 TSM 为 CVM 分配不可信的共享物理内存以实现 I/O 半虚拟化时，由于该内存区域暴露于不可信环境中，TSM 本身无法为其提供端到端的机密性与完整性保障。因此，CVM 必须在应用层或客户内核层，自主引入必要的密码学加解密与度量校验机制，以确保共享数据的流转与使用严格符合安全预期。

5.3 安全内存虚拟化实现

内存虚拟化主要涵盖 VM 运行时内存、设备 MMIO 区域以及用于 I/O 半虚拟化的共享内存的二阶段页表映射。二阶段页表的根页表基址作为 VM 配置的核心信息，被保存在 vhart 运行线程的上下文中。EasyAda-Hypervisor 按照以下标准流程构建并维护二阶段页表：（1）用户态 VMM 解析当前 VM 的配置文件（如 FDT），计算 VM 所需的运行时物理内存规模（不包含 MMIO 区域），并通过系统服务申请相应的物理内存页；（2）依据配置文件约束，将 VM 的 Kernel、FDT 等关键镜像加载至满足对齐要求的物理地址中；（3）依据配置文件构建 GPA 至 HPA 的映射关系，由运行在内核态的 Hypervisor 驱动创建初始的二阶段页表，并将其基址存入对应的 vhart 运行线程上下文中。

MMIO 区域的映射主要服务于设备直通（Device Passthrough）场景。在创建 VM 时，运行于内核态的 Hypervisor 驱动会解析配置文件中的物理设备信息，基于 GPA 与 HPA 的映射关系，将目标物理设备的 MMIO 区域直接映射至 VM 的独立地址空间中。此外，Hypervisor 还会将该硬件设备的物理中断号注册并绑定至目标 VM，以确保后续的虚拟中断注入机制能够正确触发，从而保障 VM 对直通设备的正常访问与中断响应。

共享内存映射主要服务于基于半虚拟化技术的虚拟设备管理。其初始化过程采用按需分配机制：当 VM 在系统运行过程中首次访问虚拟设备的 MMIO 区域（GPA）并触发异常时，内核态的 Hypervisor 驱动将拦截该异常。经与虚拟设备注册信息匹配确认后，若该 GPA 属于合法的虚拟设备 MMIO 范围，Hypervisor 驱动将切换至 vhart 运行上下文，并在用户态向微内核系统服务申请对应的物理共享内存。随后，由 Hypervisor 驱动在内核态完成该共享内存到 VM 二阶段地址空间的映射，并从异常处理返回，恢复 VM 的继续执行。在 EasyAda 的底层实现中，共享内存被抽象为 UNIX 风格的文件对象，该文件的所有权归属于 vhart 运行线程，并在常规架构下被映射至 vhart 服务线程中，以支撑高效的异步 I/O 交互。

在 EasyAda-trust 架构中，针对不可信 VM 的内存虚拟化流程保持原貌，但将 CVM 的内存虚拟化控制流与不可信 Hypervisor 进行了深度解耦。具体而言，Hypervisor 仅保留宏观物理层面的安全区域创建与共享内存分配权限，而核心的受

保护内存/设备分配权及二阶段页表映射逻辑，则全权由受信任的 TSM 接管。

5.3.1 基于 TSM 接口的安全区域生命周期管理

如前文所述，安全区域的具体物理抽象、隔离机制与内部内存管理层已在 EasyAda-SM 固件中实现并封装。在 EasyAda-trust 架构中，Hypervisor 无需感知安全区域底层的 PMP 配置与物理鉴权细节，而是作为上层资源调度者，通过 TSM 对外暴露的安全管理扩展接口，完成 CVM 所需核心物理内存的动态构建、分配与回收。

5.3.1.1 安全区域的动态创建触发

在系统运行期，当 Hypervisor 判定需为 CVM 扩充安全环境时，将主动通过微内核系统服务申请满足连续性约束的物理内存或物理外设的 MMIO 区域。随后，Hypervisor 将该物理地址空间作为参数，通过特权级调用触发 TSM 的安全区域创建接口。在此协同流程中，Hypervisor 仅负责宏观物理资源的供给与上层业务的无中断调度，而核心的基于可信硬件配置的合法性校验、物理属性确权以及底层硬件强隔离机制，均交由 EasyAda-SM 在极高特权级下透明完成。一旦 TSM 返回创建成功信号，Hypervisor 即在内部资源表中将该物理区间标记为“不可见”，彻底剥离不可信环境对该区域的寻址能力。

5.3.1.2 基于单次请求的特权提升与按需分配

在初始化阶段，Hypervisor 首先遵循标准 VM 的创建流程，为目标 VM 分配常规的内存与设备资源，并完成其基础逻辑架构的初始配置。当业务场景需要将该标准 VM 提升为 CVM 时，Hypervisor 不再精细化介入底层安全资源的调度过程，而是仅通过一次独立的 SBI 请求，向 TSM 下达特权提升与转换指令。

TSM 在截获该 SBI 请求后，将全权接管后续的安全资源配置工作：其会自主从当前已就绪的全局安全区域池中，为该 CVM 按需划拨容量匹配的内存型安全区域（用于承载 CVM 镜像与二阶段页表等核心元数据），并尝试预分配必需的设备型安全区域。待该 CVM 完成本地度量证明并被系统确认为绝对可信后，由 TSM 独立执行实质性的安全区域物理分配与二阶段页表的安全重映射，从而以最小的 Hypervisor 交互代价完成了 CVM 的资源构建。

5.3.1.3 安全区域的安全回收调度

当系统面临物理内存或外设资源瓶颈，抑或 CVM 生命周期终止时，Hypervisor 作为全局资源管理者，可向 TSM 发起主动的资源回收调度请求。针对内存型安全

区域，Hypervisor 请求 TSM 扫描并释放特定或空闲的安全区域；针对设备型安全区域，Hypervisor 需显式提供目标设备的 MMIO 地址以请求解绑。TSM 在底层安全释放硬件隔离限制并完成必要的擦除后，会将物理控制权交还。Hypervisor 接收到释放成功的确认后，重新将该物理内存或设备 MMIO 区间纳入不可信系统的可用资源池中，从而形成完整的资源调度闭环。

5.4 安全 I/O 虚拟化实现

EasyAda-Hypervisor 原生支持三种 I/O 虚拟化技术：（1）全虚拟化：用于模拟逻辑简单或安全性要求较高的基础组件（如串口与中断控制器等）；（2）半虚拟化：用于支撑大吞吐量 I/O 数据流的设备（如网卡、块存储设备等）；（3）设备直通：用于管理驱动逻辑复杂或具备专有计算能力的硬件外设（如 AI 硬件加速器等）。

在原生架构中，VM 的设备初始化被设计为 VM 无感的被动触发流程，旨在最大程度减少对 Guest OS 代码的侵入。系统采用静态分区与动态分配相结合的硬件发现机制，其完整流程涵盖三个阶段：（1）系统静态配置阶段：硬件供应商提供包含全局视角的 FDT，应用开发者依此划分直通设备，并在 VM 的专属硬件配置文件中建立静态绑定关联；（2）VM 初始化阶段：Hypervisor 解析该配置文件，若存在匹配的直通外设则将其逻辑分配给目标 VM，否则降级为全虚拟化模拟；（3）首次访存触发阶段：设备被分配后，VM 的首次 MMIO 访问将触发缺页异常并陷入至 Hypervisor。针对全虚拟化设备，由 Hypervisor 直接接管处理；针对半虚拟化设备，Hypervisor 将建立不可信共享内存及主从异步事件通知信道（VM 主动发起访问触发事件，vhart 服务线程异步处理后通过中断注入返回结果）；针对直通设备，则由 Hypervisor 完成物理 MMIO 至 GPA 的最终映射，交由 VM 自主完成驱动初始化。

针对 CVM 的高安全需求，EasyAda-trust 在复用上述全虚拟化与半虚拟化基础路由逻辑的同时，重点对设备直通与数据通道进行了深度的安全扩展。安全设备的使用寿命依然由 Hypervisor 驱动宏观调度，但其底层鉴权与物理隔离全权由 TSM 把控。本节将重点阐述 EasyAda-trust 如何保障 CVM 使用 I/O 设备过程中的全链路安全性。

5.4.1 安全直通设备生命周期管理

EasyAda-trust 对安全外设实施“主机分配，TSM 鉴权”的协同管理机制。在系统构建阶段，硬件供应商需在底层固化 Trusted FDT，同时应用开发者需提供 CVM 专用的虚拟硬件配置文件。这一机制将复杂的运行时设备发现转化为静态的密码

学信任链校验，极大降低了安全架构的动态攻击面。

在标准 VM 创建阶段，Hypervisor 仍遵循常规逻辑进行设备初始化与逻辑映射，但此时 VM 并不具备对该物理外设的实际读写权限。当 Hypervisor 请求将该普通 VM 提升为 CVM 时，TSM 将介入并执行严格的设备鉴权：其将 CVM 配置文件中声明的外设与底层 Trusted FDT 进行交叉比对。若匹配成功且被标记为可信，TSM 将为该设备分配专属的设备型安全区域；若匹配失败，则强制将该 MMIO 区域降级回退为全虚拟化处理，防范 Hypervisor 恶意挂载仿冒外设。

在 CVM 运行期，当其首次访问 MMIO 区域并陷入至 TSM 时，TSM 将依据 MMIO 地址的属性类别实施安全路由：若属全虚拟化设备，则将请求转换为标准 MMIO 拦截事件并转发至 Hypervisor 侧处理；若属半虚拟化设备，则转换为不可信共享内存映射请求交由 Hypervisor 调度；若属已鉴权的安全直通设备，TSM 将在底层安全构建物理 MMIO 至 CVM 地址空间的二阶段页表映射，随后将控制权交还 CVM 以完成原生的设备初始化。

5.4.2 基于 IOPMP 的设备侧防 DMA 攻击隔离

仅在 CPU 侧通过 MMU/PMP 隔离 MMIO 空间，仍无法抵御具备 DMA（Direct Memory Access）能力的恶意外设越权访问安全内存。为此，EasyAda-trust 引入了 IOPMP 硬件扩展，从总线级构建外设访存的物理安全网关。在硬件开发阶段，供应商需为所有硬件设备静态分配唯一源标识符（Source ID）。

在系统启动初始化阶段，TSM 会默认接管全局 IOPMP 控制器，并为所有已注册的 SID 创建并绑定初始的不可信 MD。该不可信 MD 的基础访问控制策略被严格设定为允许访问不可信物理内存，但直接屏蔽并拦截对任何受保护的安全区域的寻址请求。所有未被显式提升为安全设备的外设，其 SID 均被默认约束于此不可信 MD 内，从而确立了系统初始阶段防范 DMA 攻击的默认拒绝（Default-Deny）物理隔离基线。因为所有设备共享相同的不可信内存视图，所以共享同一个不可信 MD。

在系统运行期，伴随 CVM 生命周期的演进，每当 Hypervisor 通过 TSM 成功创建一个全新的安全区域时，TSM 将立即触发全局 IOPMP 状态同步机制。具体而言，TSM 会遍历并修改不可信 MD 的内部规则，将针对该新增安全区域的限制访问条目动态注入其中。这一严格的实时同步策略确保了即使安全内存是动态分配与衍生的，所有驻留于不可信环境的常规外设亦无法通过 DMA 途径对其进行非法的探测、读取或篡改。

当 Hypervisor 请求将特定物理外设提升为安全设备以供 CVM 独占使用时，TSM 首先校验该设备 SID 对应的可信硬件配置属性。鉴权通过后，TSM 会为该设

备分配一个处于隔离状态的可信 MD。唯有当该安全设备被实质性地挂载并分配给目标 CVM 时，TSM 才会动态更新该可信 MD，将当前 CVM 所关联的所有内存型安全区域的授权条目注入其中，并将该设备的 SID 从不可信 MD 解绑，转而与该可信 MD 强绑定。至此，IOPMP 硬件强制限定该直通设备仅能对其从属 CVM 的受保护内存发起 DMA 操作，不仅防范了外部入侵，亦从物理总线层面彻底杜绝了设备内部敏感数据向不可信内存泄漏的风险。

反之，当 CVM 生命周期终止并触发安全设备释放时，TSM 将立即重置并安全擦除该设备所对应可信 MD 中的所有授权规则。在 Hypervisor 请求回收该外设资源时，TSM 会将其 SID 重新降级并绑定至不可信 MD，从而在确保物理安全的界限下，完成硬件外设的归还与状态复原。

5.4.3 I/O 数据平面安全与端到端保护

确立了设备控制平面的物理隔离后，必须进一步保障 I/O 数据平面的机密性与完整性。机密应用（如部署于 CVM 中的 AI 推理引擎）是高价值 I/O 数据（如 AI 模型权重与用户输入）的核心消费者。

5.4.3.1 信任根推演与密钥供应

如图5-5所示,I/O 数据保护的基石在于密钥的安全供应。在系统启动阶段,CVM 与底层硬件环境之间将执行双向本地证明。确认环境可信后，TSM 利用硬件信任根（RoT）内置的私钥解锁受保护的锁盒（Lockbox），获取对称密钥并进一步解密目标应用载荷（TAP），从而安全提取出开发者预置的机密素材（Secrets）并暂存于 TSM 内部。在 CVM 运行时，Guest OS 可通过安全的 SBI 请求，从 TSM 处按需获取相应的开发者密钥或会话密钥。

5.4.3.2 端到端数据传输保护机制

获取密钥后，CVM 需针对不同类型的 I/O 通道采取差异化的安全策略：

- （1）**安全直通设备的数据流：**得益于 PMP 与 IOPMP 构建的双向物理隔离屏障，不可信组件（包括 Hypervisor）无法以任何形式窥探直通外设的 MMIO 与 DMA 区域。因此，机密应用可直接以明文形态向安全设备（如直通硬件加速器）下发控制指令与 AI 模型参数，而无需承受额外的软件加解密性能损耗。
- （2）**半虚拟化设备的数据流：**对于基于不可信共享内存构建的半虚拟化信道（如网络与磁盘 I/O），数据必须穿越不可信 Hypervisor 的控制边界。在此场景下，CVM 必须利用预先供应的对称/非对称密钥，在软件栈顶层实施

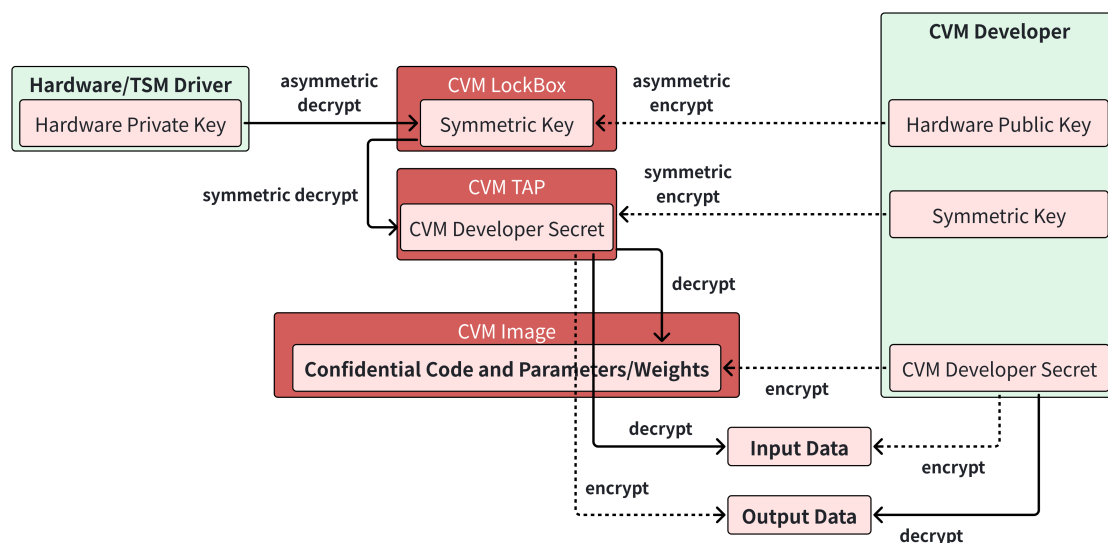


图 5-5 EasyAda-trust 下 CVM 的 I/O 密钥管理

端到端的密码学加解密与报文认证码校验，坚决拒绝接收及执行任何被 Hypervisor 篡改、重放或伪造的 I/O 载荷。

5.5 机密异构计算环境实现

EasyAda-trust 一个重要的目标是保障用户 AI 模型和敏感数据的机密性和完整性。该目标可以进一步细分为两个目标：

- (1) 保障用户运行环境的可信和隔离：为了保护用户 AI 模型，CVM 仅能在可信的硬件环境上对用户模型进行解密和部署，并且由 EasyAda-trust 确保 CVM 运行环境与不可信组件的隔离。
- (2) 保障用户 I/O 数据的机密性和完整性：为了确保用户敏感数据不被窃取或篡改，EasyAda-trust 需要对用户 I/O 数据采取必要的加密和校验，并确保用户模型使用硬件加速器过程中的安全。

同时，EasyAda-trust 为了增强系统的可扩展性，需要实现硬件加速器在不同执行环境间的复用，为了实现以上目标，EasyAda-trust 实现了专用的硬件加速器管理 CVM，并通过 I/O 半虚拟化技术实现硬件加速器在不同执行环境间的复用。下面将详细介绍 EasyAda-trust 如何实现机密异构计算环境。

5.5.1 硬件加速器的保护

为了确保 CVM 使用硬件加速器过程中的安全性，Hypervisor 必须显式将硬件加速器提升为安全设备，TSM 在加速器管理 CVM 启动时，会检查并尝试分配安全设备，只有经过验证的安全硬件加速器可以映射到 CVM 的地址空间中进行管理。

同时，硬件加速器使用的内存缓冲区由加速器管理 CVM 内的驱动从当前 CVM 的安全内存中进行分配，不可信组件无法访问安全的内存缓冲区和 MMIO 区域。

5.5.2 加速器管理 CVM 初始化

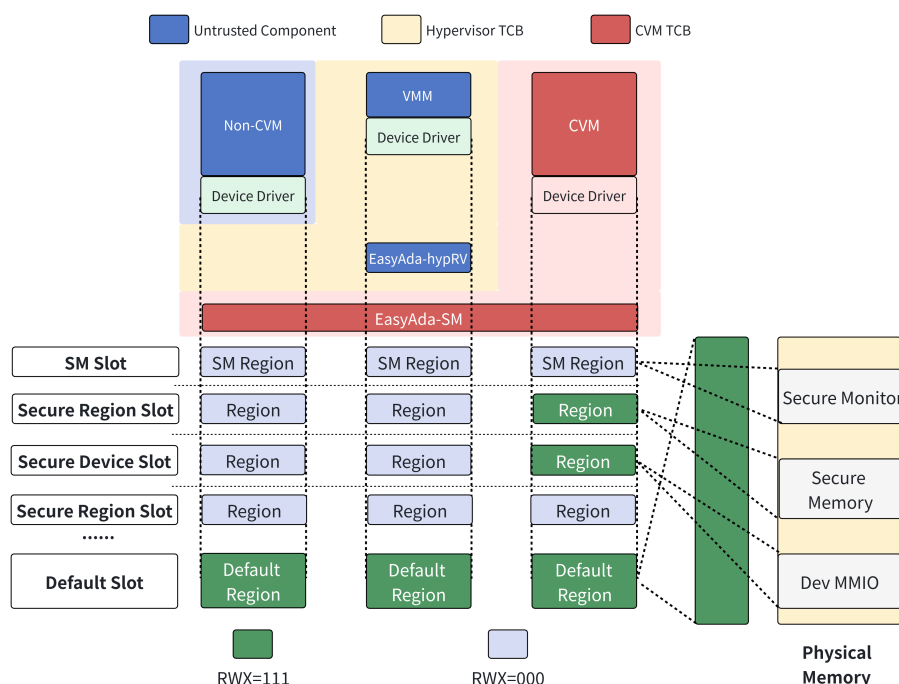


图 5-6 引入安全设备后的 CVM 安全内存视图

在未引入 CVM 的架构下，EasyAda-Hypervisor 为了实现对现有驱动的复用，避免增大微内核操作系统的攻击面，会在启动阶段将硬件加速器的 MMIO 地址透传到硬件加速器管理 VM 中，由该 VM 加载驱动并完成硬件加速器初始化，后续通过 I/O 半虚拟化技术（如 VirtI/O 协议）实现硬件加速器在 VM 间以及 host 间的时分复用。

加速器管理 CVM 的创建与其他 CVM 类似，在创建加速器管理 CVM 前，Hypervisor 需要主动将可信的硬件加速器提升为安全硬件，因为提升时机由 EasyAda-Hypervisor 决定，应当由其自身确保无任何组件正在访问硬件加速器，避免访问流程的异常中断。当硬件加速器被提升为安全设备后，不可信组件无法访问硬件加速器的 MMIO 区域，

接下来由 Hypervisor 负责创建 CVM，在 TSM 为 CVM 分配安全设备时，会同时为硬件加速器创建授权 MD 用于授予当前硬件加速器访问安全区域的能力，因为端侧硬件加速器多以协处理器的形式集成在 SoC 中，因此其数据缓冲区需要动态进行分配，为了降低 IOPMP 管理的开销，TSM 配置 IOPMP 授权硬件加速器访问加速器管理 CVM 关联的所有安全区域以确保可以正常访问动态数据缓冲区，因

为配置 DMA 时会通过 IOMMU 或 TSM 软件翻译 IPA 为物理地址，避免了硬件加速器因为 DMA 配置错误写入其它安全内存中。隔离完成后，不可信组件无法访问硬件加速器的 MMIO 区域，硬件加速器也无法写入不可信的内存区域。

加速器管理 CVM 创建成功后，如图5-6所示，此时加速器管理 CVM 是除 TSM 外唯一可以访问硬件加速器软件实体。当加速器管理 CVM 实际运行时，需要按照安全启动流程，CVM 从 TSM 获取开发者密钥并对开发者提供的 AI 模型和参数进行解密和部署，并对不可信的半虚拟化和全虚拟化设备进行初始化。

加速器管理 CVM 创建成功之后，可以直接对其上的机密应用提供服务。如果需要与不可信环境间进行复用，则需要与请求使用硬件加速器的不可信组件建立信道，下面将介绍信道建立的过程。

5.5.3 建立信道

加速器管理 CVM 其它组件均通过 I/O 半虚拟化技术——VirtI/O 协议实现硬件加速器的时分复用。

虚拟设备的发现在开发是通过静态的硬件配置文件如 FDT 文件静态完成，EasyAda-trust 应用开发者需要根据硬件供应商提供的可信物理硬件配置完成设备静态分区，并在加速器管理 CVM 的硬件配置文件中预留内存区域用于建立映射共享内存。

加速器管理 CVM 会对设备进行初始化，当访问到预留的内存区域时会陷入 TSM，由 TSM 检查 MMIO 区域类型后，将共享内存申请请求转发到 EasyAda-Hypervisor 处理。EasyAda-Hypervisor 负责完成共享内存分配并返回地址给 TSM，由 TSM 完成共享内存到 CVM 地址空间的映射以及 CVM 上下文的恢复，并准备进行驱动后端初始化。同时，EasyAda-Hypervisor 会标记该共享内存为虚拟硬件加速器设备区域，等待映射到驱动前端中。

当不可信 VM 尝试初始化虚拟硬件加速器时，会陷入到 EasyAda-Hypervisor 中，由 EasyAda-Hypervisor 将会获取之前加速器管理 CVM 申请的共享内存，并将其映射到请求方 VM 对应的虚拟设备地址中。之后 VM 和加速器管理 CVM 可以通过该共享内存完成数据交换，由 TSM 和 EasyAda-Hypervisor 转发和注入中断进行 I/O 请求通知和回复。EasyAda-trust 并不要求加速器管理 CVM 和请求方 VM 的初始化时序，双方可以由任意一方进行初始化并完成共享内存的申请和标记，EasyAda-trust 不感知实际的进一步的协议交互和信道建立流程，通信双方可以基于共享内存，通过轮询或中断完成通信协议的初始化。

5.6 本章小结

本章详细论述了基于微内核的可信虚拟化方案的工程实现，重点阐述了如何适配 EasyAda-Hypervisor 以实现对 CVM 的安全管理与资源虚拟化。在 CPU 虚拟化方面，方案通过扩展 EasyAda-Hypervisor 的 hart 分区架构，引入了机密与非机密 vhart 运行区，并利用 TSM 接管 CVM 的两阶段安全上下文切换流程，确保了 CVM 的 CPU 状态对不可信 Hypervisor 完全隐匿。在内存与 I/O 虚拟化层面，方案将安全资源的直接控制权从 EasyAda-Hypervisor 中剥离，确立了由 Hypervisor 负责物理内存分配、设备发现与宏观生命周期调度，进而委托 TSM 完成安全区域动态构建、二阶段页表隔离映射及安全设备鉴权配置的职责解耦机制。此外，针对端侧异构计算需求，方案利用 EasyAda-Hypervisor 辅助建立不可信共享内存通信信道，成功实现了专用硬件加速器管理 CVM 的安全部署与跨环境复用。综上所述，本章通过对 EasyAda-Hypervisor 实施职责解耦与非侵入式安全扩展，在充分复用微内核统一资源管理优势的同时，为 CVM 提供了一个高安全、高可靠的可信虚拟化运行底座。

第六章 系统测试与分析

6.1 测试目标

本章旨在对基于微内核的可信虚拟化架构 EasyAda-trust 及其核心组件 EasyAda-SM 进行系统性的定量评估与安全性验证。评测过程重点围绕以下四个核心科学问题展开：1. 生态兼容性：采用 Rust 语言重构并引入安全区域抽象的 EasyAda-SM 是否能够以尽可能小的代价兼容现有 RISC-V TEE 生态系统？2. 基础性能开销：消除不安全内存访问并引入强隔离机制后，EasyAda-trust 在核心计算任务中引入的额外性能损耗是否在可接受范围内？

6.2 测试软硬件环境

鉴于当前 RISC-V 硬件生态的客观限制，目前量产的 RISC-V 商用芯片在安全性上存在明显的路线分化：面向端侧的芯片普遍支持 PMP 和 IOPMP 扩展，但是缺乏对 H 扩展的支持；而面向高性能计算场景的芯片虽然支持 H 扩展，但是通常集成较复杂的 IOMMU 和高级中断架构（AIA）。然而，这类复杂组件高昂的硬件面积开销及基于总线事务的中断延迟抖动，使其难以在成本与实时性敏感的嵌入式 SoC 中广泛落地。。

6.3 SM 生态兼容性测试

参考 EasyAdaZone 的实验设计^[49]。为验证系统的生态兼容性，本文在极小化代码修改的前提下，除引导程序与安全监视器外完全复用了既有 Keystone 与 Penglai 框架中的全量组件。复用范围涵盖宿主机用户态接口、编译工具链、软件依赖库、测试用例以及受保护侧的 TA，并以 Linux（内核版本 4.15）作为 Host OS。在性能评估方面，本文选用 Coremark 和 RV8 benchmark 作为基准测试集，并以原生 Linux 环境的执行表现作为对比基线。

6.3.0.1 Coremark 测试套

如图6-1所示,所有测试结果均以非 EasyAda-SM 环境的原生 Linux 环境(Native Linux)的得分为基线(100%)进行归一化处理。实验数据表明,在引入 EasyAda-SM 后,系统的计算性能得到了极大的保留。在 Linux、Keystone 与 Penglai 三种工作负载下与未引入安全防护的原生环境相比, EasyAda-SM 环境的相对性能衰减均被严格控制在 2% 以内。

因为 Coremark 属典型的 CPU 密集型载荷，其执行路径高度集中于用户态及 Enclave 内部。由于本文所提架构充分利用了 RISC-V 硬件级的 PMP 扩展，安全隔离校验在微架构流水线中隐式完成，且 Rust 语言的内存安全检查均发生在编译期。因此，只要未触发跨特权级的上下文切换，EasyAda-SM 不会对核心运算流水线引入任何运行时开销。

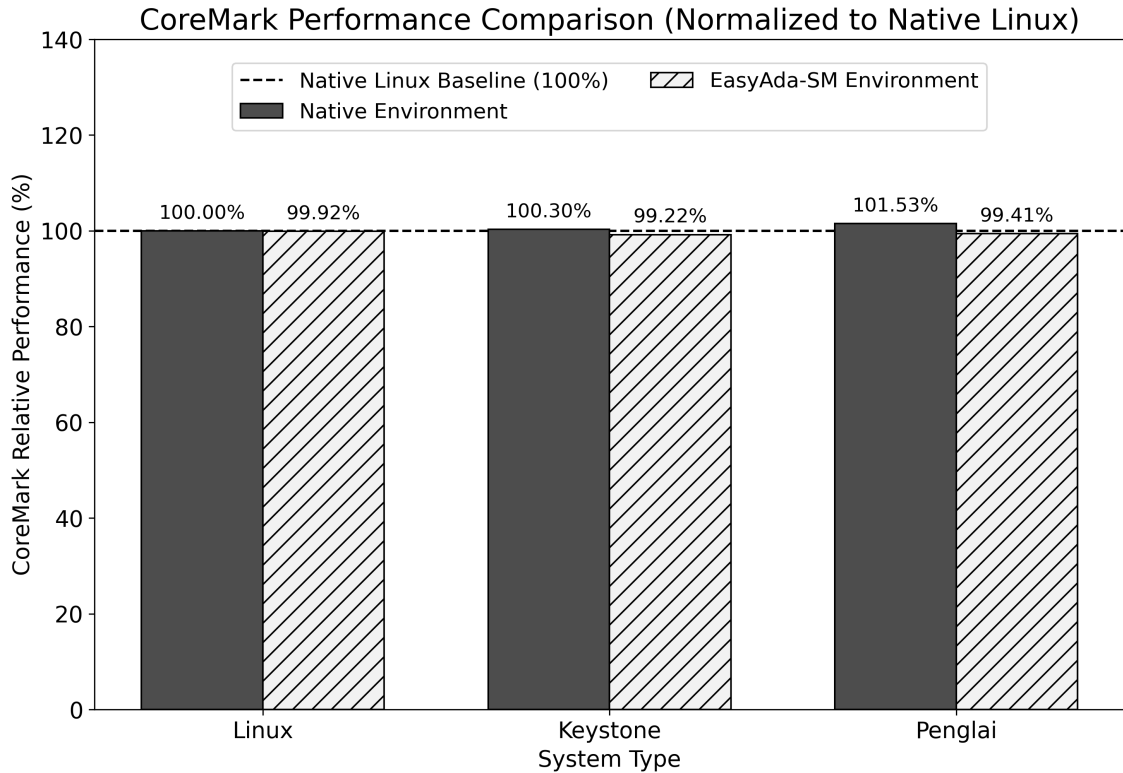
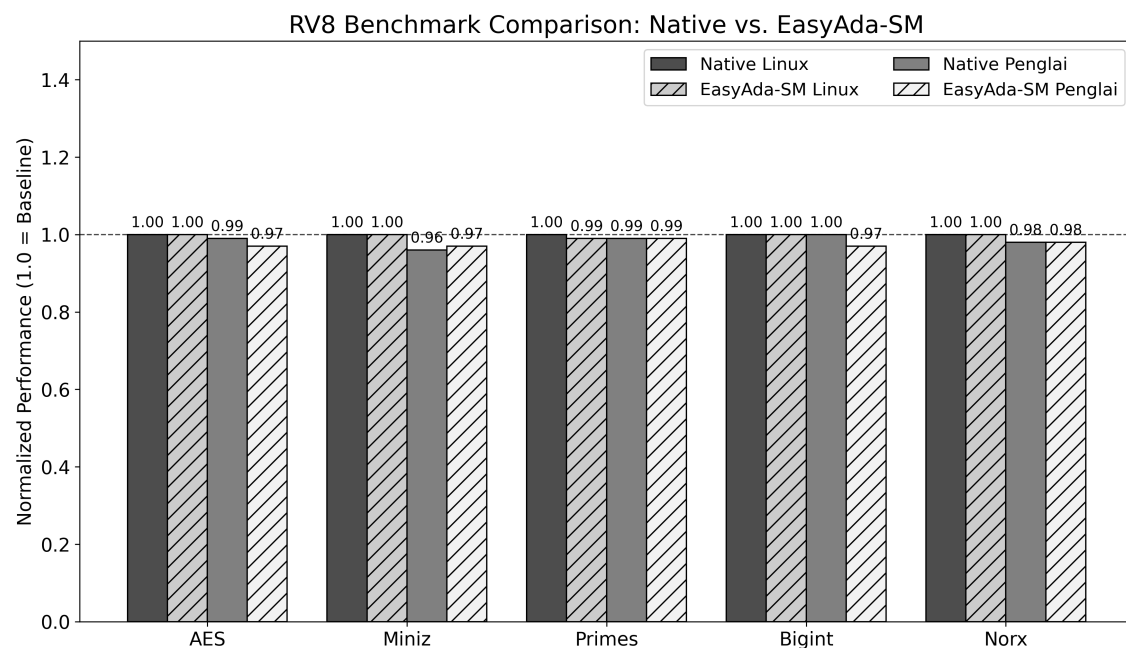


图 6-1 原生环境与 EasyAda-SM 环境下 Coremark 的对比测试

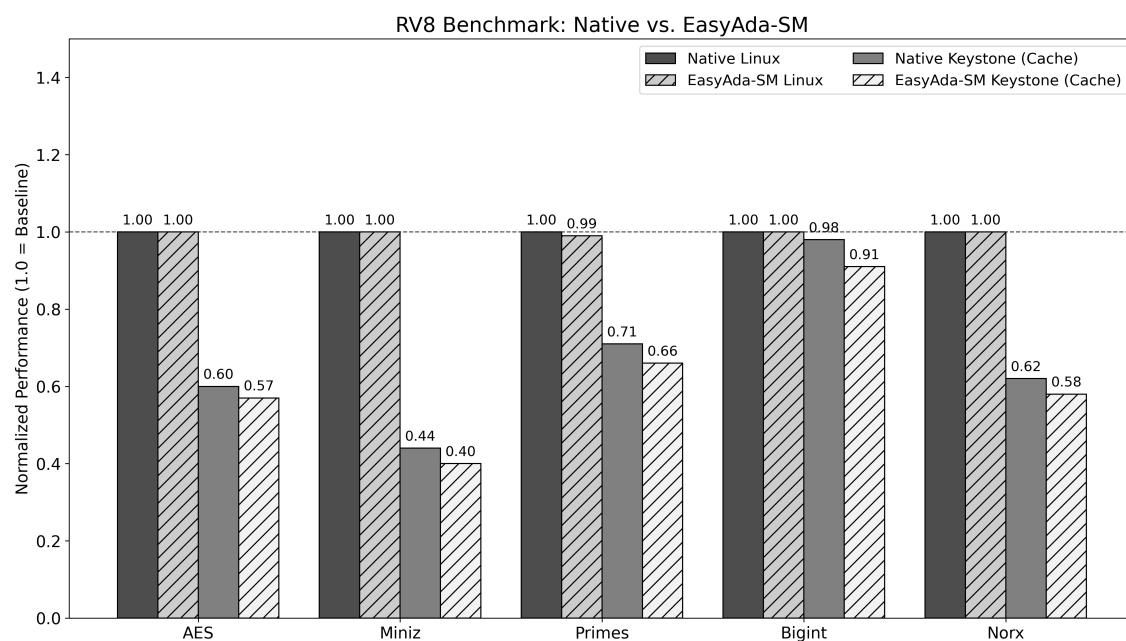
6.3.0.2 RV8 基准测试套

为了进一步探究 EasyAda-SM 环境在面对具有不同指令分布与复杂访存行为特征的应用时的性能表现，本文引入了 RV8 benchmark 测试集。该测试集包含 AES（加密）、Miniz（数据压缩）、Primes（素数计算）、Bigint（大数运算）以及 Norx（认证加密）等五项典型负载。所有测试结果均以原生 Linux 环境为归一化基准（1.0）。

如图6-2(a)所示，在未显式开启微架构侧信道防御的 Penglai 轻量级 Enclave 场景下，Penglai 在原生环境和 EasyAda-SM 环境下均展现出卓越的执行效率，各项负载归一化得分均不低于 0.96。以访存密集型的 Miniz 为例，二者得分分别为 0.96 与 0.97，证实了引入基于 Rust 的 EasyAda-SM 及安全区域内存抽象后，系统的额外性能损耗被严格控制在 1% 至 2% 的极小区间内，底层特权级调度对上层复杂业务逻辑的执行干扰极微。



(a) Penglai 对比测试结果



(b) Keystone 对比测试结果

图 6-2 原生环境与 EasyAda-SM 环境下 RV8 benchmark 测试结果对比

然而，在适配 Keystone 框架的测试中，系统性能呈现出显著的差异化衰减：如图6-2(b)所示，Miniz 与 AES 的相对性能分别断崖式下跌至约 0.40 与 0.57，而计算密集型的 Bigint 受影响较小，分析可能是因为 Keystone 为防御缓存侧信道攻击，在物理层面强制启用了硬件缓存分区隔离机制，严格限制了 Enclave 可用的末级缓存路数，导致高度依赖数据局部性的算法缓存缺失率剧增。在未启用缓存分区的环境中，横向对比原生环境和 EasyAda-SM 环境可知，EasyAda-SM 引入的绝对软

件开销仅为 3% 至 7%，这一处于合理区间的微小性能折损，主要源于 EasyAda-SM 模块化分层抽象所导致的底层执行链路延长。

综合上述交叉验证与基准测试结果表明，本文提出的基于 Rust 构建的内存安全监视器 EasyAda-SM 在确立坚实底层信任根的同时，兼具高度的执行效率与架构弹性。具体而言，在纯计算密集型负载下，得益于 PMP 机制的流水线隐式校验以及 Rust 语言编译期的零成本抽象特性，该安全架构所引入的运行时额外开销与原生实现相比趋近于零。

6.4 EasyAda-trust 测试

6.4.1 系统启动测试

系统的启动流程可严格划分为基础固件引导与安全监控器接管两个关键阶段。如图6-3所示，本系统将 RustSBI 作为静态库集成至 EasyAda-SM 中。在系统上电初期，执行流首先跳转至 RustSBI 以完成底层硬件环境的配置。在交出控制权前，RustSBI 激活了前 8 个 PMP 条目，将其自身驻留的物理内存区域配置为无访问权限，从而构建防御低特权级代码篡改的基础安全边界。

当 RustSBI 完成底层硬件初始化后，系统控制权交还至 EasyAda-SM 以推进核心初始化逻辑。如图6-4所示，多 hart 的初始化日志交替输出，表明系统已成功唤醒并构建了 SMP 环境。鉴于 RISC-V 架构中各 hart 均拥有独立的控制与状态寄存器，EasyAda-SM 确保了所有核心严格同步执行一致的安全策略配置，从而有效规避跨核越权访问等硬件级侧信道漏洞。

在全局内存隔离策略的构建上，测试环境中的 Hart 均支持 16 个 PMP 条目。EasyAda-SM 利用最高优先级的条目划定 SM 的根安全区域，通过写入配置值 0x18（即 NAPOT 匹配模式且权限为 NONE），对预留的 16MB 安全监控器专有内存实施强制硬件隔离；同时，利用最低优先级的条目制定背景策略，授予系统默认访问公共物理内存与 MMIO 区域的权限。此外，为实现对 RustSBI 的最小化侵入式修改，系统保留了其独立的堆内存区；前述 16MB 预留空间的主体则被接管为 EasyAda-SM 的全局堆内存。上述初始化序列执行完毕后，控制权将正式移交至 EasyAda-hypervisor，以开启虚拟化层的引导流程。

6.4.2 CVM 安全启动测试

在普通 VM 提升为安全 CVM 时，需要完成 CVM 与 EasyAda-SM 的双向本地证明。本文模拟了以下三种边界情况进行安全启动流程测试：（1）CVM 可信，平台可信；（2）CVM 不可信，平台可信；（3）CVM 可信，平台不可信。

```

[RustSBI] INFO - Hello RustSBI!
[RustSBI] INFO - RustSBI version 0.4.0
[RustSBI] INFO -
[RustSBI] INFO - | _ \ | | | | / | | | | / | | | | _ \ | | |
[RustSBI] INFO - | |_) | | | | | (---|---| (---|---| |_) | | |
[RustSBI] INFO - | / | | | | \ \ | | | | \ \ | | | < | | |
[RustSBI] INFO - | | \---| |---| |---) | | | |---) | | |_) | | |
[RustSBI] INFO - |_| \_| |_/ |_/ |_/ |_/ |_/ |_/ |_/ |_/
[RustSBI] INFO - Initializing RustSBI machine-mode environment.
[RustSBI] INFO - Platform Name : riscv-virtio,qemu
[RustSBI] INFO - Platform HART Count : 4
[RustSBI] INFO - Enabled HARTs : [0, 1, 2, 3]
[RustSBI] INFO - Platform IPI Extension : SiFiveClint (Base Address: 0x2000000)
[RustSBI] INFO - Platform Console Extension : Uart16550U8 (Base Address: 0x1000000)
[RustSBI] INFO - Platform Reset Extension : Available (Base Address: 0x100000)
[RustSBI] INFO - Platform HSM Extension : Available
[RustSBI] INFO - Platform RFence Extension : Available
[RustSBI] INFO - Platform SUSP Extension : Available
[RustSBI] INFO - Platform PMU Extension : Available
[RustSBI] INFO - Memory range : 0x80000000 - 0x180000000
[RustSBI] INFO - Platform Status : Platform initialization complete and ready.
[RustSBI] INFO - PMP Configuration
[RustSBI] INFO - PMP Range Permission Address
[RustSBI] INFO - 0 OFF NONE 0x0000000000000000
[RustSBI] INFO - 1 TOR RWX 0x0000000080000000
[RustSBI] INFO - 2 TOR RWX 0x0000000080000000
[RustSBI] INFO - 3 TOR R 0x0000000080027000
[RustSBI] INFO - 4 TOR NONE 0x0000000080036000
[RustSBI] INFO - 5 TOR RW 0x0000000080071000
[RustSBI] INFO - 6 TOR RWX 0x0000000180000000
[RustSBI] INFO - 7 TOR RWX 0xfffffffffffffc
[RustSBI] INFO - Boot HART ID : 3
[RustSBI] INFO - Boot HART Privileged Version: : Version1_12
[RustSBI] INFO - Boot HART MHPM Mask: : 0x07ffff
[RustSBI] INFO - The patched dtb is located at 0x8005c000 with length 0x2258.

```

图 6-3 SBI 固件初始化

图6-5展示了可信 CVM 在可信平台上启动的基准情况。在该流程中，EasyAda-SM 在接管 VM 提升请求后，对 CVM 的内存布局、内核镜像及 FDT 进行完整性度量。日志显示，系统计算得出的 PCR4 度量值与 TAP 中预置的参考值完全一致。本地证明成功后，SM 向 CVM 安全注入 TAP 中存储的秘密，随后 Linux 内核成功引导。该结果验证了系统在理想状态下能够正确鉴权并启动合法 CVM。

图6-6展示了不可信 CVM（例如内核镜像被恶意篡改）在可信平台上的启动被拦截过程。当 EasyAda-SM 对篡改后的 CVM 进行度量时，生成的 PCR4 值与预期参考值发生偏移。本地证明机制敏锐捕捉到该完整性破坏，抛出 LocalAttestationFailed 错误。随后，SM 立即阻断引导流，拒绝为其分配执行权限，并将其从控制数据结构中安全移除。该测试证明了平台对恶意或受损 CVM 的有效识别与拦截能力。

图6-7则验证了双向证明中的反向约束，即 CVM 可信但底层平台（EasyAda-SM 或硬件运行环境）不可信的情况。在该场景下，当系统尝试读取并执行 TAP 以验证平台身份时，由于平台无法获取预期的用于解密的密钥，TAP 解密失败并触发固件恐慌。SM 随即中止启动流程并执行环境清理。此结果表明，EasyAda-trust

```
#EasyAda-SM: Setting up hardware hart id=1
#EasyAda-SM: Number of PMPs=16
#EasyAda-SM: pmp0 value: 40000000, shifted 100000000
#EasyAda-SM: pmp0 cfg: 11000
#EasyAda-SM: pmp15 value: 0, shifted ffffffff
#EasyAda-SM: pmp15 cfg: 11111
#EasyAda-SM: Setting up hardware hart id=0
#EasyAda-SM: Number of PMPs=16
#EasyAda-SM: pmp0 value: 40000000, shifted 100000000
#EasyAda-SM: pmp0 cfg: 11000
#EasyAda-SM: pmp15 value: 0, shifted ffffffff
#EasyAda-SM: pmp15 cfg: 11111
#EasyAda-SM: Setting up hardware hart id=2
#EasyAda-SM: Number of PMPs=16
#EasyAda-SM: pmp0 value: 40000000, shifted 100000000
#EasyAda-SM: pmp0 cfg: 11000
#EasyAda-SM: pmp15 value: 0, shifted ffffffff
#EasyAda-SM: pmp15 cfg: 11111
#EasyAda-SM: Setting up hardware hart id=3
#EasyAda-SM: Number of PMPs=16
#EasyAda-SM: pmp0 value: 40000000, shifted 100000000
#EasyAda-SM: pmp0 cfg: 11000
#EasyAda-SM: pmp15 value: 0, shifted ffffffff
#EasyAda-SM: pmp15 cfg: 11111
```

图 6-4 EasyAda-SM 初始化测试

[illegible]

图 6-5 可信 CVM 在可信平台上启动测试

架构有效防止了合法 CVM 在被攻陷或未授权的底层环境中运行，确保了机密计算环境的双向信任根基。

[illegible]

图 6-6 不可信 CVM 在可信平台上启动测试

```
#EasyAda-SM: Promoting a VM to a confidential VM
#EasyAda-SM: Reading flatten device tree (FDT) at memory address 0xbfe00000
#EasyAda-SM: Virtual machine's memory layout: ConfidentialVmMemoryLayout { kernel: (2147483648, 3221225472), fdt: (3219128320, 3219132833), initrd: None }
#EasyAda-SM: Number of confidential harts: 2
#EasyAda-SM: Reading TEE attestation payload (TAP) at memory address 0x81503000
#EasyAda-SM: Ops security monitor panicked!
#EasyAda-SM: Line 115, file rust-crates/riscv_cove_tap/src/spec.rs: called `Result::unwrap()` on an `Err` value: Error
#EasyAda-SM: Cleaning up...
```

图 6-7 可信 CVM 在不可信平台上启动测试

6.4.3 CVM 性能测试

本节旨在全面评估 EasyAda-trust 架构在承载 CVM 时，TSM 的引入对客户操作系统核心原语执行效率的影响。为了精准量化各项系统级操作的开销，本文选用 LEBench^[50] 作为核心测试套件。该套件能够精细评估进程调度、内存管理及基础 I/O 等系统调用的时延表现。

鉴于仿真环境硬件时钟精度的客观限制，本测试对 **LEBench** 进行了批处理改造，将单次操作修改为批量循环执行后取平均值，以有效放大测量时延并确保数据的统计学精度。本文引入了以下三种对比基准：（1）原生 **Linux**，未启用虚拟化与 **TEE** 保护的裸金属物理执行环境，作为绝对性能上限的参照组；（2）普通虚拟机：运行于 **EasyAda-Hypervisor** 之上，受标准虚拟化调度但不具备 **EasyAda-SM** 内存隔离保护的虚拟机环境；（3）**CVM**，运行于 **EasyAda-trust** 架构之上，受 **EasyAda-SM** 全生命周期管理、内存强隔离及机密上下文切换保护的执行环境。

如图6-8所示，所有测试结果均以原生 Linux 环境的执行时延为基线进行归一化处理。数值越低代表操作时延越短、性能越优。具体分析如下：

在读取系统时钟的测试 (ref) 中, 普通 VM 与 CVM 的归一化延迟几乎与物理裸机性能一致。这一表现归功于实验环境底层硬件对 S 态时钟中断扩展的完备支持, 允许 VM 直接在 VS 态配置定时器并接收虚拟中断, 规避了传统架构中需要陷入到 M 态进行注入的开销, 这符合 TSM 直接将 CVM 时钟中断委派到 VS 态进行

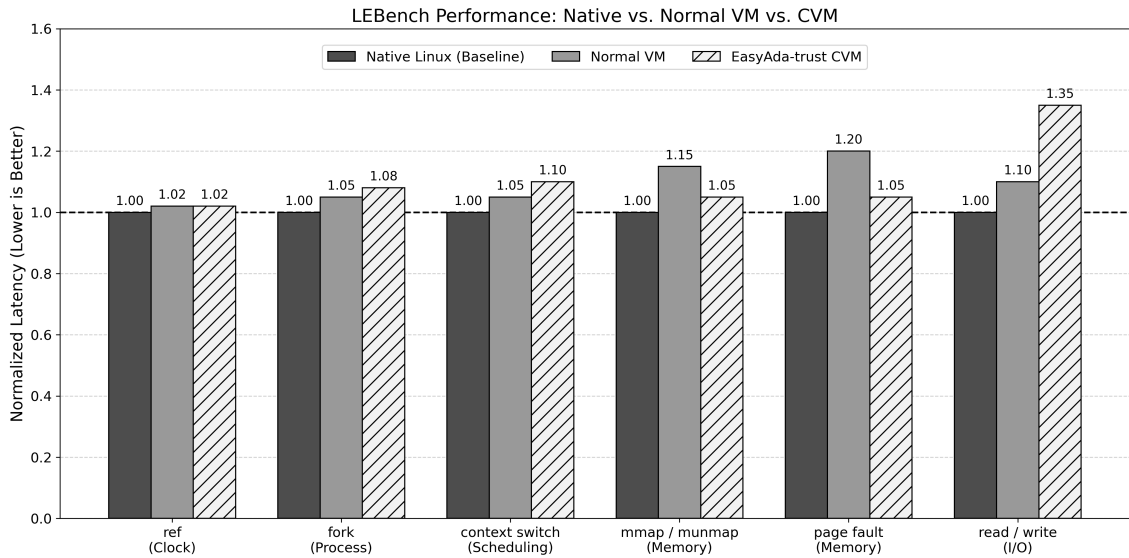


图 6-8 LEBench 测试结果

处理的设计。

在内存分配与缺页异常处理上，普通虚拟机与 CVM 均表现出极高的效率，因为普通 VM 和 CVM 的运行期内存已在启动时一次性完成分配，由于二阶段页表在初始化时已建立完毕且不进行换出，虚拟机在运行期的缺页异常完全收敛于客户系统内部闭环处理，除了硬件固有的两阶段地址翻译开销外，并未在系统关键访存路径上引入任何额外的软件级运行时惩罚。

在进程调度层面，CVM 的时延略高于普通 VM。LEBench 测量的核心是虚拟机内部（VS 与 VU 态之间）的用户态进程切换。虽然常规切换不会触发陷入 TSM 的行为，但在涉及跨执行环境的上下文协作时，需陷入 TSM 执行严苛的两阶段安全上下文切换、清理微架构残余状态，并动态重配安全内存视图，这导致了 CVM 侧可预期的性能衰减。

基础 I/O 测试结果表明，底层设备路由架构对 CVM 性能具有决定性影响。在传统的全/半虚拟化架构下，为遵循严密的内存隔离模型，TSM 阻断了不可信 Hypervisor 的 DMA 路径。这迫使 CVM 采用高开销的安全中转机制：I/O 数据必须经历内部密码学加解密、向不可信共享缓冲区的多重拷贝，以及高频触发的 TSM 安全域上下文切换。此类数据冗余拷贝、微架构缓存失效与纯软件计算的叠加，共同导致该模式下 I/O 性能显著衰减。

相比之下，图6-9展示了安全设备直通架构与半虚拟化机制（不可信共享内存）下的网卡 I/O 对比测试结果。该架构将受 PMP 保护的外设 MMIO 直接映射至 CVM 二阶段地址空间，并协同 IOPMP 扩展在总线级别实施抗 DMA 攻击的物理内存鉴权。机密应用得以合法旁路不可信软件中转栈，在免除高昂软件流加密开销的同

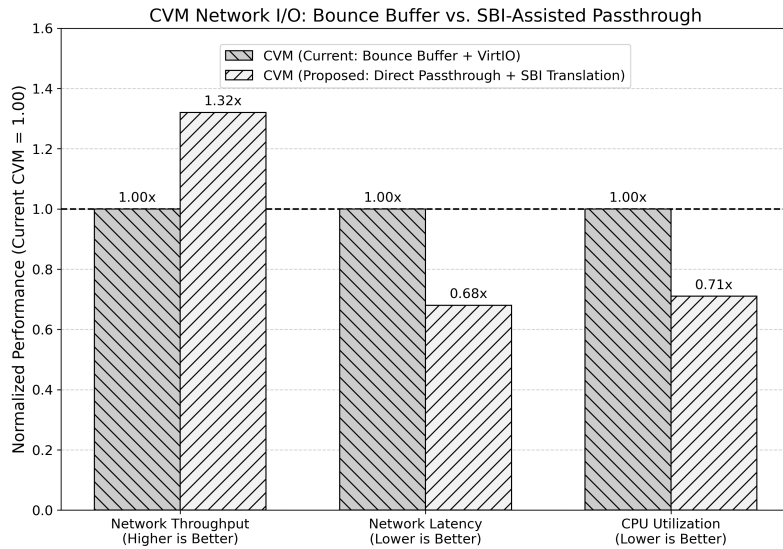


图 6-9 安全设备直通架构与半虚拟化机制（不可信共享内存）下的网卡 I/O 对比测试结果

时，实现了与受信异构硬件的零拷贝明文高速通信。

6.5 本章小结

本章对所提出的 EasyAda-trust 微内核可信虚拟化架构及其核心固件 EasyAda-SM 进行了系统性的定量评估与安全性验证。首先，通过 Coremark 与 RV8 基准测试证明了引入 Rust 内存安全机制后的 EasyAda-SM 具备极高的生态兼容性，且在核心计算场景下引入的额外运行时开销趋近于零。其次，验证了系统安全启动与双向本地证明机制的有效性，结果表明该机制能精准识别并拦截受损的虚拟机镜像与不可信的底层运行环境，确保了信任链的完整性。

此外，通过 LEBench 微基准测试与网络 I/O 测试对 CVM 的全生命周期开销进行了深度剖析。数据表明，得益于静态安全内存预分配策略与 Sstc 硬件扩展的支持，CVM 在系统时钟与内存管理等核心原语上实现了近似零损耗的极致性能。而在面临物理隔离边界导致的 I/O 性能衰减时，本文提出的基于 IOPMP 扩展的安全设备直通架构，成功绕过了高昂的软件密码学计算与不可信内存中转栈，在捍卫系统极高隔离强度的同时，实现了异构外设的零拷贝明文高速复用。综合而言，本系统在极致的内存安全保障与总体执行效能之间取得了科学且理想的架构折中。

第七章 总结和展望

7.1 全文总结

虚拟化技术与可信执行环境作为优化资源利用、保障系统安全的核心技术手段，在物联网与端侧异构计算领域扮演着愈加重要的角色。机密虚拟机（CVM）作为现代 TEE 的演进方向，直接关系到用户敏感数据与 AI 模型资产的机密性、完整性及系统实时性。当前，基于 x86 和 ARM 架构的机密计算方案已广泛部署，但面向开源 RISC-V 架构的微内核机密虚拟化研究仍处于起步阶段，且传统 C 语言构建的底层固件面临严峻的内存安全威胁，因此该方向具有重大的学术与工程研究价值。

本文围绕基于 RISC-V 架构的机密虚拟化方案实现与底层安全隔离机制展开系统性研究。基于 RISC-V 硬件虚拟化扩展（H 扩展）、物理内存保护机制（PMP/IOPMP）以及自研微内核操作系统 EasyAda 及微内核虚拟化架构 EasyAda-Hypervisor，设计并实现了基于微内核虚拟化架构的机密虚拟化架构 EasyAda-trust 及内存安全固件 EasyAda-SM，并在 QEMU 仿真环境及多种基准测试套件下对其核心性质与性能开展了系统性评估。总体而言，本文主要贡献可概括为以下几点：

1. 系统阐述了微内核虚拟化、TEE 架构演进及其在异构计算环境下面临的安全挑战。在端侧与物联网系统中，硬件算力复用与 AI 数据隐私保护需求日益凸显。本文分析了 CVM 在实际系统中的关键作用，并指出引入 Rust 内存安全语言与 RISC-V 开放架构对于削减可信计算基（TCB）、消除内存漏洞及降低 TEE 准入门槛的重要意义。

2. 针对底层安全监视器易受内存破坏漏洞威胁的痛点，本文提出并使用 Rust 语言从零构建了内存安全固件 EasyAda-SM。该方案创新性地结合 Rust 的所有权机制设计了页分配器（Page Allocator），从编译期彻底杜绝了内存管理元数据被破坏的可能。同时，通过模块化分层设计，抽象出统一的物理内存保护原语与安全区域管理层，实现了不同类型 Enclave 与不可信组件间的物理级强隔离。

3. 针对 CVM 全生命周期管理与异构安全扩展的需求，本文在微内核虚拟化方案 EasyAda-Hypervisor 的基础上设计实现了 EasyAda-trust 架构。通过将物理资源调度与安全内存映射鉴权进行深度解耦，实现了安全的 CPU 机密流上下文切换与二阶段页表隔离。在此基础上，本文结合 IOPMP 技术构建了防御恶意 DMA 攻击的总线屏障，并创新性地设计了加速器管理 CVM，利用 I/O 半虚拟化技术实现了底层硬件加速器在不可信与机密环境间的安全时分复用。

4. 本文在 QEMU 环境中对 EasyAda-trust 及其核心组件开展了多维度的定量评估与功能验证。测试结果表明, EasyAda-SM 具备极高的生态兼容性, 在核心计算负载(如 Coremark 与 RV8 benchmark)下引入的性能损耗不到 2%; 在安全机制验证方面, 本文构建的基于本地证明的双向验证测试, 成功验证了 EasyAda-trust 架构在 CVM 不可信或平台不可信等多种边界情况下的防御拦截能力。

通过上述研究工作, 本文提供了一种将微内核虚拟化技术与 Rust 内存安全固件深度融合的 RISC-V 机密计算解决方案。与已有的 Keystone 或 Penglai 等纯应用级/系统级 TEE 框架相比, 本文聚焦于支持 CVM 抽象的虚拟机级强隔离, 并在异构外设的机密复用维度提出了完整的软硬协同范式, 为基于 RISC-V 架构的下一代端侧机密计算底座提供了一种高可靠、高可扩展的工程实现框架。

7.2 后续工作展望

尽管本文在 RISC-V 微内核机密虚拟化及安全隔离机制上取得了一定成果, 但面向未来复杂系统的演进, 仍有以下四个方向亟待深入研究与改进:

1. 多核机密并发调度与中断优化: 当前跨环境上下文切换策略较为保守。未来将结合 RISC-V 高级中断架构, 探索细粒度上下文感知与懒加载策略, 优化核间中断与共享资源的并发控制, 提升高并发负载下的吞吐量与实时响应能力。

2. 真实硬件部署与直通开销优化: 受限于硬件生态, 本文评估主要基于 QEMU。未来拟将系统移植至高阶 RISC-V 物理开发板进行全面压测。同时, 针对 IOPMP 高频总线鉴权引入的 DMA 瓶颈, 将探索基于硬件表项缓存或动态规则折叠的软硬协同算法, 以降低物理访存损耗。

3. 核心可信组件的形式化验证: 尽管 Rust 从编译期消除了内存漏洞, 但 TSM 的逻辑正确性仍需理论证明。未来将引入 Isabelle/HOL 等证明工具, 对 TSM 的异常嵌套处理、二阶段页表映射等开展形式化验证, 并扩展无干涉性证明, 从理论层面夯实信任根。

4. 异构计算生态与端到端总线安全: 面向未来基于 PCIe/CXL 互连的片外独立加速器, 现有机制难以防御总线窥探与重放攻击。未来将研究集成机密计算总线协议与内存加密引擎构建硬件级端到端加密链路, 并拓展加速器 CVM 驱动生态, 以原生支撑端侧大语言模型的机密推理。

致 谢

参考文献

- [1] OpenAMP. Welcome to the openamp project documentations[EB/OL]. 2026-01-11, <https://openamp.readthedocs.io/en/latest/index.html>.
- [2] Yan J, Chen Q, Zhou S, et al. Ekc: a portable and extensible kernel compartment for de-privileging commodity os[C]. Proceedings of the 34th USENIX Conference on Security Symposium, USENIX Association.
- [3] ARM. Arm security technology building a secure system using trustzone technology[EB/OL]. 2009-4-1, <https://developer.arm.com/documentation/PRD29-GENC-009492/latest>.
- [4] ARM. Intel® architecture instruction set extensions programming reference[EB/OL]. 2025-11-12, <https://www.intel.com/content/www/us/en/content-details/869288/intel-architecture-instruction-set-extensions-programming-reference.html>.
- [5] AMD. Using sev with amd epyc™ processors[EB/OL]. 2023-10-03, <https://docs.amd.com/v/u/en-US/58207-using-sev-with-amd-epyc-processors>.
- [6] Feng E, Lu X, Du D, et al. Scalable memory protection in the PENGDAI enclave[C]. 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21), 2021: 275–294.
- [7] Lee D, Kohlbrenner D, Shinde S, et al. Keystone: an open framework for architecting trusted execution environments[C]. Proceedings of the Fifteenth European Conference on Computer Systems, 2020: 1-16.
- [8] GlobalPlatform. Tee system architecture v1.3[EB/OL]. 2022-05-01, <https://globalplatform.org/specs-library/tee-system-architecture/#>.
- [9] Limited A. Learn the architecture - introducing arm confidential compute architecture version 4.0[EB/OL]. 2025-03-19, <https://developer.arm.com/documentation/den0125/latest>.
- [10] AMD. Intel® trust domain extensions[EB/OL]. 2025-06-01, <https://cdrdv2.intel.com/v1/dl/getContent/690419>.
- [11] Cerdeira D, Santos N, Fonseca P, et al. Sok: Understanding the prevailing security vulnerabilities in trustzone-assisted tee systems[C]. 2020 IEEE Symposium on Security and Privacy (SP), 2020: 1416-1432.
- [12] Matsakis N D, Klock F S. The rust language[C]. Association for Computing Machinery, 2014: 103–104.

- [13] Wang H, Wang P, Ding Y, et al. Towards memory safe enclave programming with rust-sgx[C]. Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019: 2333–2350.
- [14] Wan S, Sun M, Sun K, et al. Rustee: Developing memory-safe arm trustzone applications[C]. Proceedings of the 36th Annual Computer Security Applications Conference, 2020: 442–453.
- [15] Popek G J, Goldberg R P. Formal requirements for virtualizable third generation architectures[J]. Commun. ACM, 1974, 17(7): 412–421.
- [16] Sahoo J, Mohapatra S, Lath R. Virtualization: A survey on concepts, taxonomy and associated security issues[C]. 2010 Second International Conference on Computer and Network Technology, 2010: 222–226.
- [17] Qumranet A, Qumranet Y, Qumranet D, et al. Kvm: The linux virtual machine monitor[J]. Proceedings Linux Symposium, 2007, 15.
- [18] Barham P, Dragovic B, Fraser K, et al. Xen and the art of virtualization[J]. SIGOPS Oper. Syst. Rev., 2003, 37(5): 164–177.
- [19] Dall C, Li S W, Nieh J. Optimizing the design and implementation of the linux ARM hypervisor[C]. 2017 USENIX Annual Technical Conference (USENIX ATC 17), 2017: 221–233.
- [20] Hwang J Y, Suh S B, Heo S K, et al. Xen on arm: System virtualization using xen hypervisor for arm-based secure mobile phones[C]. 2008 5th IEEE Consumer Communications and Networking Conference, 2008: 257–261.
- [21] Martins J, Tavares A, Solieri M, et al. Bao: A Lightweight Static Partitioning Hypervisor for Modern Multi-Core Embedded Systems[C]. Workshop on Next Generation Real-Time Embedded Systems (NG-RES 2020), 2020: 3:1–3:14.
- [22] Li H, Xu X, Ren J, et al. Acrn: a big little hypervisor for iot development[C]. Proceedings of the 15th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments, 2019: 31–44.
- [23] Trujillo S, Crespo A, Alonso A. Multipartes: Multicore virtualization for mixed-criticality systems[C]. 2013 Euromicro Conference on Digital System Design, 2013: 260–265.
- [24] Matos E d, Ahvenjärvi M. sel4 microkernel for virtualization use-cases: Potential directions towards a standard vmm[J]. Electronics, 2022, (24).
- [25] Limited B. Qnx hypervisor 8.0 empowering embedded systems with trusted virtualization and mixed-criticality support[EB/OL]. 2025-10-01, <https://qnx.software/content/dam/qnx-xwalk/pdf/product-briefs/qnx-hypervisor-8-product-brief.pdf>.
- [26] Steinberg U, Kauer B. Nova: a microhypervisor-based secure virtualization architecture[C]. Proceedings of the 5th European Conference on Computer Systems, 2010: 209–222.

-
- [27] Jia Y, Liu S, Wang W, et al. HyperEnclave: An open and cross-platform trusted execution environment[C]. 2022 USENIX Annual Technical Conference (USENIX ATC 22), 2022: 437–454.
- [28] Pereira S, Sousa J, Pinto S, et al. Bao-enclave: Virtualization-based enclaves for arm[C]. 2022 IEEE 8th World Forum on Internet of Things (WF-IoT), 2022: 1–6.
- [29] Brasser F, Gens D, Jauernig P, et al. Sanctuary: Arming trustzone with user-space enclaves[C]. 2019: .
- [30] Wang J, Li A, Li H, et al. Rt-tee: Real-time system availability for cyber-physical systems using arm trustzone[C]. 2022 IEEE Symposium on Security and Privacy (SP), 2022: 352–369.
- [31] Feng E, Feng D, Du D, et al. siopmp: Scalable and efficient i/o protection for tees[C]. Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, 2024: 1061–1076.
- [32] Du D, Yang B, Xia Y, et al. Accelerating extra dimensional page walks for confidential computing[C]. Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture, 2023: 654–669.
- [33] Ozga W. Towards a formally verified security monitor for vm-based confidential computing[C]. Proceedings of the 12th International Workshop on Hardware and Architectural Support for Security and Privacy, 2023: 73–81.
- [34] AMD. Amd sev-snp: Strengthening vm isolation with integrity protection and more[EB/OL]. 2020-01-01, <https://docs.amd.com/v/u/en-US/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more>.
- [35] Sahita R, Shanbhogue V, Bresticker A, et al. Cove: Towards confidential computing on risc-v platforms[C]. Proceedings of the 20th ACM International Conference on Computing Frontiers, 2023: 315–321.
- [36] Wu X, Tian D J, Kim C H. Building gpu tees using cpu secure enclaves with gevisor[C]. Proceedings of the 2023 ACM Symposium on Cloud Computing, 2023: 249–264.
- [37] Deng Y, Wang C, Yu S, et al. Strongbox: A gpu tee on arm endpoints[C]. Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, 2022: 769–783.
- [38] Fan S, Hua Z, Xia Y, et al. Xputee: A high-performance and practical heterogeneous trusted execution environment for gpus[J]. ACM Trans. Comput. Syst., 2025, 43(1–2).
- [39] Wang C, Zhang F, Deng Y, et al. Cage: Complementing arm cca with gpu extensions[C]. Proceedings of the 31st Annual Network and Distributed System Security Symposium.
- [40] Wang C, Tang D, Ci C, et al. ccAI: A Compatible and Confidential System for AI Computing[M], Association for Computing Machinery, 2025, 340–353.

- [41] Lu H, Deng Y, Mertoguno S, et al. Mole: Breaking gpu tee with gpu-embedded mcu[C]. Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, 2025: 693–707.
- [42] International R V. The risc-v instruction set manual: Volume ii[EB/OL]. 2025-05-08, https://docs.riscv.org/reference/isa/_attachments/riscv-privileged.pdf.
- [43] Li X, Zhao B, Yang G, et al. A survey of secure computation using trusted execution environments[J]. arXiv preprint arXiv:2302.12150, 2023.
- [44] Consortium C C. Confidential computing – the next frontier in data security[EB/OL]. 2021-10-19, https://confidentialcomputing.io/wp-content/uploads/sites/10/2023/03/Everest_Group_-_Confidential_Computing_-_The_Next_Frontier_in_Data_Security_-_2021-10-19.pdf.
- [45] Wang X, Shen W, Bu Y, et al. Dmaauth: a lightweight pointer integrity-based secure architecture to defeat dma attacks[C]. Proceedings of the 33rd USENIX Conference on Security Symposium.
- [46] Hunt G D H, Pai R, Le M V, et al. Confidential computing for openpower[C]. Proceedings of the Sixteenth European Conference on Computer Systems, 2021: 294–310.
- [47] Lee J, Wang Y, Rajat R, et al. Characterization of gpu tee overheads in distributed data parallel ml training[J]. arXiv preprint arXiv:2501.11771, 2025.
- [48] RustSBI. rustsbi[EB/OL]. 2026-02-10, <https://github.com/rustsbi/rustsbi>.
- [49] Jinzhe L, Kun X, Lei L, et al. Easy adazone: A memory-safe physical memory isolation framework for risc-v[C]. 2025 22nd International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP), 2025: 1-4.
- [50] Ren X J, Rodrigues K, Chen L, et al. An analysis of performance evolution of linux’s core operations[C]. Proceedings of the 27th ACM Symposium on Operating Systems Principles, Association for Computing Machinery, SOSP ’19, 2019: 554–569.

攻读硕士学位期间取得的成果